Глубокое обучение с подкреплением

Модуль 2: введение в Q-обучение

Курс Deep RL от Hugging Face

Полное учебное издание на основе исходных MDX-файлов

Исходный материал: курс Deep RL от Hugging Face.

Содержание

1 Введение в Q-обучение **2**

2 Что такое RL? Краткое повторение **3**

3 Два типа методов на основе ценности **4**
- 3.1 Функция ценности состояния . . . 7
- 3.2 Функция ценности действия . . . 8

4 Уравнение Беллмана: упрощаем оценку ценности **10**

5 Методы Монте-Карло и обучение с временными разностями **13**
- 5.1 Монте-Карло: обучение в конце эпизода . . . 13
- 5.2 Обучение с временными разностями: обновление на каждом шаге . . . 17

6 Промежуточное повторение **20**

7 Промежуточный тест **21**
- 7.1 Ответы и объяснения . . . 22

8 Знакомство с Q-обучением **24**
- 8.1 Что такое Q-обучение? . . . 24
- 8.2 Алгоритм Q-обучения . . . 27
- 8.3 Off-policy и on-policy: различия . . . 31

9 Пример Q-обучения **32**
- 9.1 Шаг 1. Инициализируем Q-таблицу . . . 34
- 9.2 Шаг 2. Выбираем действие с помощью ε-жадной стратегии . . . 34
- 9.3 Шаг 3. Выполняем A_t, получаем R_{t+1} и S_{t+1} . . . 34
- 9.4 Шаг 4. Обновляем $Q(S_t, A_t)$ . . . 35
- 9.5 Шаг 2. Выбираем действие с помощью ε-жадной стратегии . . . 36
- 9.6 Шаг 3. Выполняем A_t, получаем R_{t+1} и S_{t+1} . . . 36
- 9.7 Шаг 4. Обновляем $Q(S_t, A_t)$ . . . 37

10 Повторение: Q-обучение **37**

11 Практическая часть **39**

12 Модуль 2: Q-обучение для FrozenLake-v1 и Taxi-v3 **41**
- 12.1 Цели этого ноутбука . . . 42
- 12.2 Этот ноутбук относится к курсу Deep Reinforcement Learning . . . 42
- 12.3 Предварительные требования . . . 42
- 12.4 Краткое повторение Q-обучения . . . 42

13 Давайте напишем наш первый алгоритм обучения с подкреплением **44**
13.1 Установка зависимостей и создание виртуального экрана . . . 44
13.2 Импорт библиотек . . . 45

14 Часть 1. FrozenLake без скольжения **46**
14.1 Создание и изучение среды FrozenLake . . . 46
14.2 Создание и инициализация Q-таблицы . . . 47
14.3 Определение жадной политики . . . 49
14.4 Определение ε-жадной политики . . . 49
14.5 Определение гиперпараметров . . . 50
14.6 Создание цикла обучения . . . 51
14.7 Обучение Q-агента . . . 52
14.8 Посмотрим на получившуюся Q-таблицу . . . 53
14.9 Метод оценки качества . . . 53
14.10Оценка качества Q-агента . . . 53
14.11Публикация обученной модели в Hub . . . 54

15 Часть 2. Taxi-v3 **58**
15.1 Создание и изучение среды Taxi-v3 . . . 58
15.2 Определение гиперпараметров . . . 60
15.3 Обучение нашего Q-агента . . . 61
15.4 Создание словаря модели и публикация обученной модели в Hub . . . 62

16 Часть 3. Загрузка модели из Hub **62**
16.1 Дополнительные задания . . . 63
16.2 Продолжайте учиться и развиваться . . . 65

17 Второй тест **65**
17.1 Ответы и пояснения . . . 66

18 Глоссарий **68**

19 Заключение **70**

20 Дополнительная литература **70**
20.1 Обучение Монте-Карло и с временными разностями . . . 70
20.2 Q-обучение . . . 70

1 Введение в Q-обучение

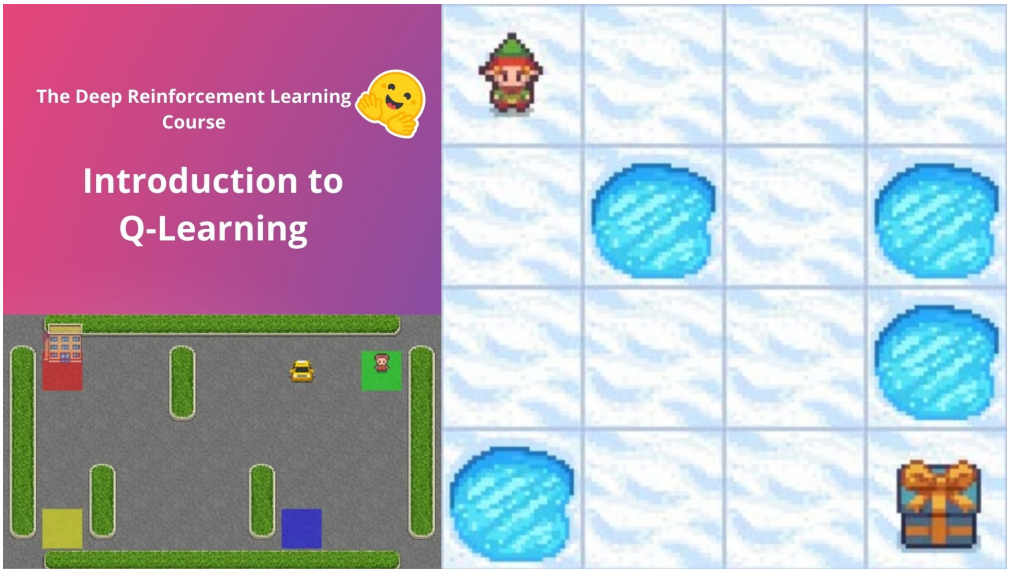


Рис. 1: Обложка модуля 2 [оригинал]

В первом модуле курса мы познакомились с обучением с подкреплением (RL), процессом взаимодействия агента со средой и различными подходами к решению задач RL. Кроме того, мы **обучили первых агентов и загрузили их в Hugging Face Hub.**

В этом модуле мы **подробнее изучим один из подходов к обучению с подкреплением — методы на основе функции ценности (value-based methods)** и разберём наш первый алгоритм RL: **Q-обучение (Q-Learning).**

Мы также **реализуем первого RL-агента с нуля** на основе Q-обучения и обучим его в двух средах:

1. FrozenLake-v1 (нескользкий вариант): агенту нужно **добраться от начальной клетки (S) до цели (G)**, перемещаясь только по замёрзшим клеткам (F) и избегая прорубей (H).
2. Автономное такси: агент должен **научиться ориентироваться** в городе, чтобы **перевозить пассажиров из пункта A в пункт B.**



Рис. 2: Две учебные среды: FrozenLake и Taxi [оригинал]

В частности, мы:

- Изучим **методы на основе функции ценности**.
- Разберём **различия между методами Монте-Карло и обучением с временными разностями (Temporal Difference Learning)**.
- Изучим и реализуем **наш первый алгоритм RL** — Q-обучение.

Этот модуль **необходим для понимания глубокого Q-обучения (Deep Q-Learning)** — первого алгоритма глубокого RL, который научился играть в игры Atari и в некоторых из них превзошёл уровень человека (Breakout, Space Invaders и др.).

Итак, приступим!

2 Что такое RL? Краткое повторение

В RL мы создаём агента, способного **принимать разумные решения**. Например, агента, который **учится играть в видеоигру**, или торгового агента, который **учится максимизировать прибыль**, выбирая, **какие акции покупать и когда их продавать.**

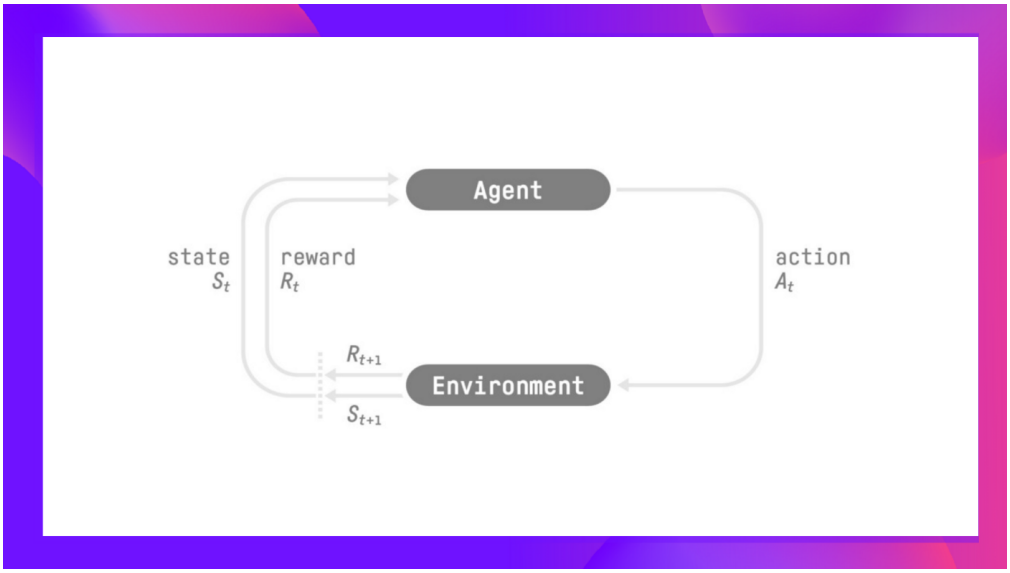


Рис. 3: Цикл обратной связи между агентом и средой [оригинал]

Чтобы принимать осмысленные решения, агент учится, **взаимодействуя со средой методом проб и ошибок**, и получает вознаграждения (положительные или отрицательные) **как единственный сигнал обратной связи.**

Его цель — **максимизировать ожидаемое суммарное вознаграждение** (согласно гипотезе вознаграждения).

Способ принятия решений агентом называется политикой π: по заданному состоянию политика выдаёт действие или распределение вероятностей по действиям. Иными словами, на основании наблюдения среды политика определяет, какое действие должен выполнить агент (либо вероятности выбора каждого действия).



Рис. 4: Политика сопоставляет наблюдениям действия [оригинал]

Наша цель — найти оптимальную политику π^***,** то есть политику, обеспечивающую наилучшее ожидаемое суммарное вознаграждение.

Для нахождения оптимальной политики (а значит, решения задачи RL) существуют **два основных подхода**:

- *Методы на основе политики (policy-based):* **обучают непосредственно политику**, чтобы выбирать действие по заданному состоянию.
- *Методы на основе ценности (value-based):* **обучают функцию ценности**, определяющую, **какие состояния предпочтительнее**, и затем используют её, **чтобы выбрать ведущее к ним действие.**

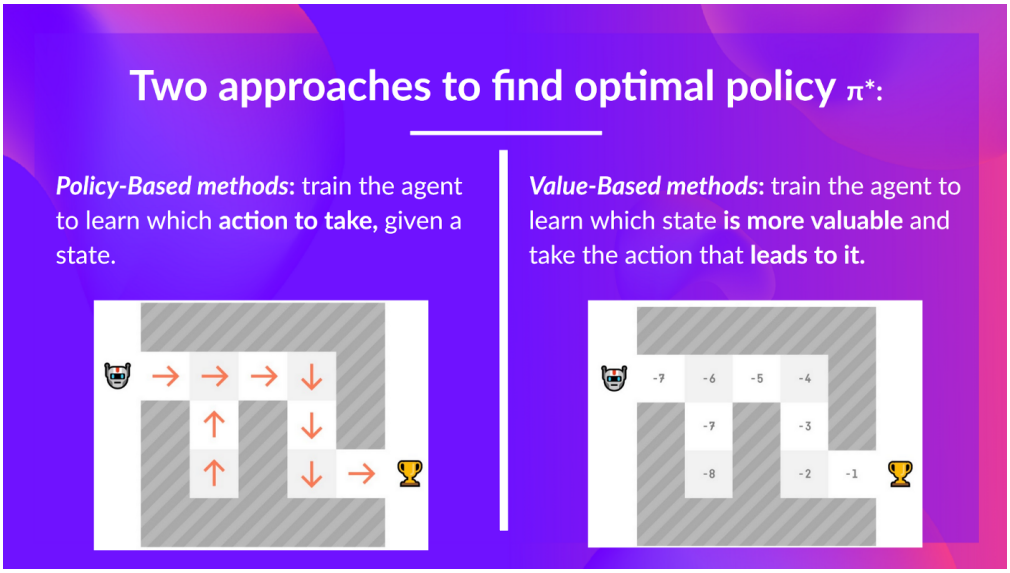


Рис. 5: Два основных подхода: непосредственное обучение политики и обучение функции ценности [оригинал]

В этом модуле мы **подробнее рассмотрим методы на основе функции ценности.**

3 Два типа методов на основе ценности

В методах на основе ценности **мы обучаем функцию ценности**, которая **сопоставляет состоянию ожидаемую ценность нахождения в этом состоянии.**

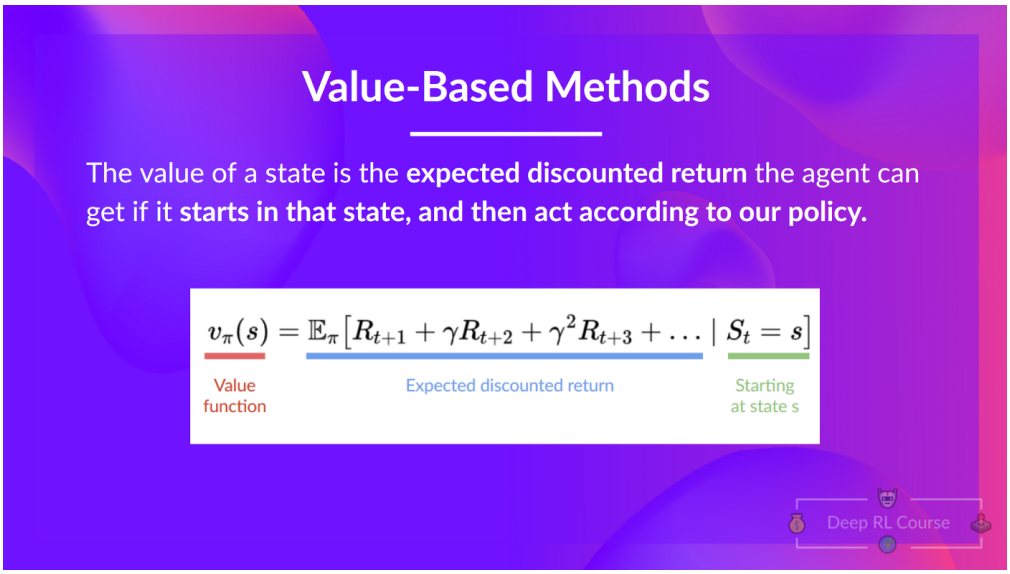


Рис. 6: Общая схема обучения с подкреплением на основе функции ценности [оригинал]

Ценность состояния — это **ожидаемая дисконтированная отдача**, которую агент может получить, если **начнёт из этого состояния и далее будет действовать согласно своей политике.**

> **Примечание к курсу.** Но что значит «действовать согласно политике»? Ведь в методах на основе ценности мы обучаем функцию ценности, а не саму политику.

Напомним, цель **RL-агента — получить оптимальную политику** π^*.

Для поиска оптимальной политики мы рассмотрели два подхода:

- *Методы на основе политики:* **обучают политику напрямую**, чтобы выбрать действие по состоянию (или распределение вероятностей действий в этом состоянии). В таком варианте **функция ценности отсутствует.**

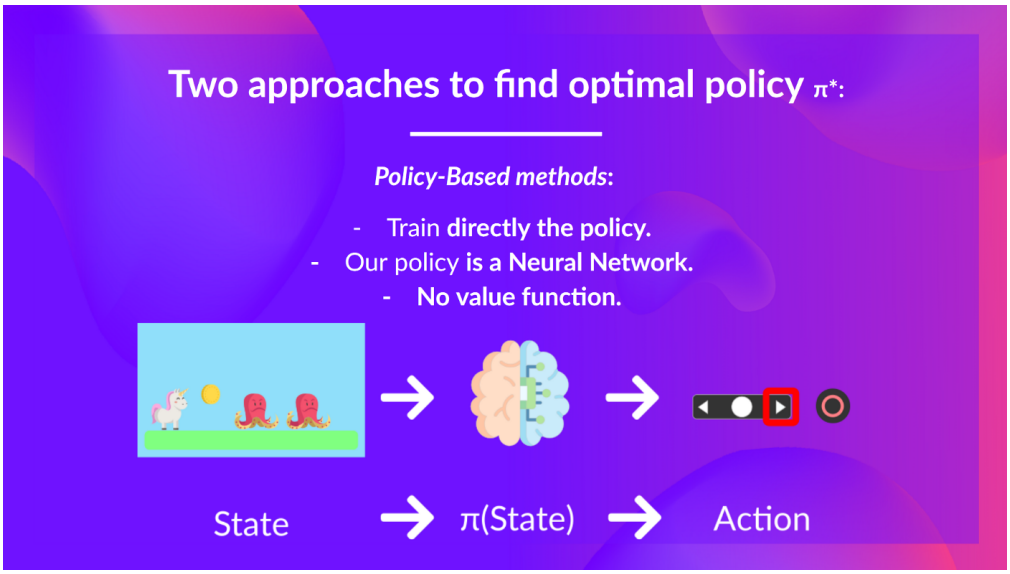


Рис. 7: Обучение на основе политики: действие непосредственно предсказывается политикой [оригинал]

Политика принимает состояние и выдаёт действие. Детерминированная политика выдаёт для состояния одно действие, в отличие от стохастической, которая выдаёт распределение вероятностей по действиям.

Следовательно, **мы не задаём поведение политики вручную: оно определяется в**

процессе обучения.

- *Методы на основе ценности:* **обучают функцию ценности косвенным способом**, которая выдаёт ценность состояния или пары «состояние–действие». На основании этой функции наша политика **выбирает действие.**

Поскольку политика сама не обучается, **её поведение необходимо задать заранее.** Например, если требуется всегда выбирать действие с наибольшей ожидаемой выгодой согласно функции ценности, **мы используем жадную политику (Greedy Policy).**

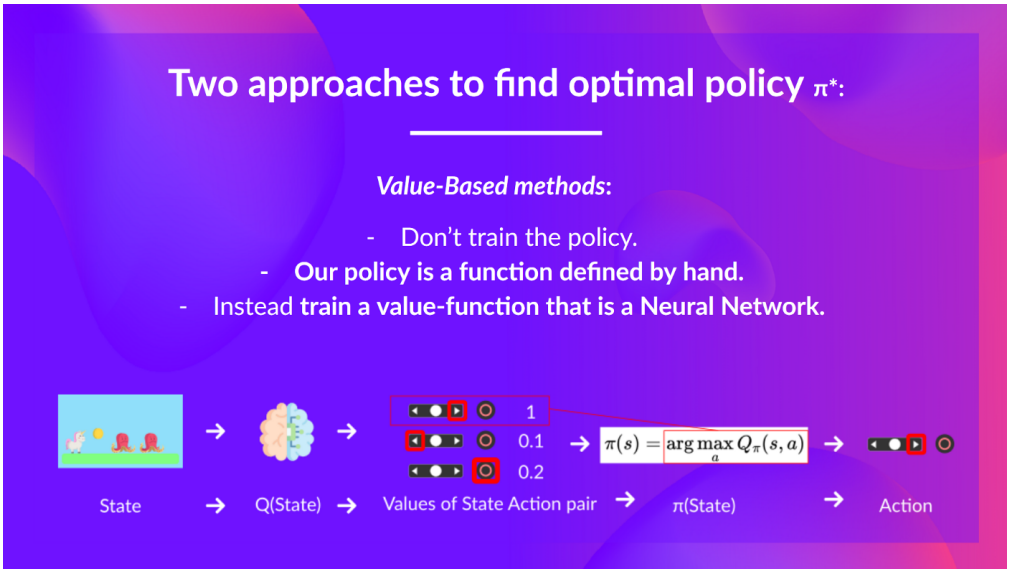


Рис. 8: Обучаемая функция ценности действий выдаёт значения для всех действий в заданном состоянии, после чего заранее определённая жадная политика выбирает действие с максимальной ценностью [оригинал]

Таким образом, какой бы метод решения задачи вы ни использовали, **политика у агента будет всегда**. В методах на основе ценности политика не обучается: она представляет собой **простую заранее заданную функцию** (например, жадную политику), использующую значения функции ценности для выбора действий.

Итак, различие состоит в следующем:

- В методах на основе политики **оптимальная политика (обозначаемая π^*) находится непосредственным обучением политики.**
- В методах на основе ценности **нахождение оптимальной функции ценности (Q^* или V^*, различие между ними рассмотрим ниже) позволяет получить оптимальную политику.**

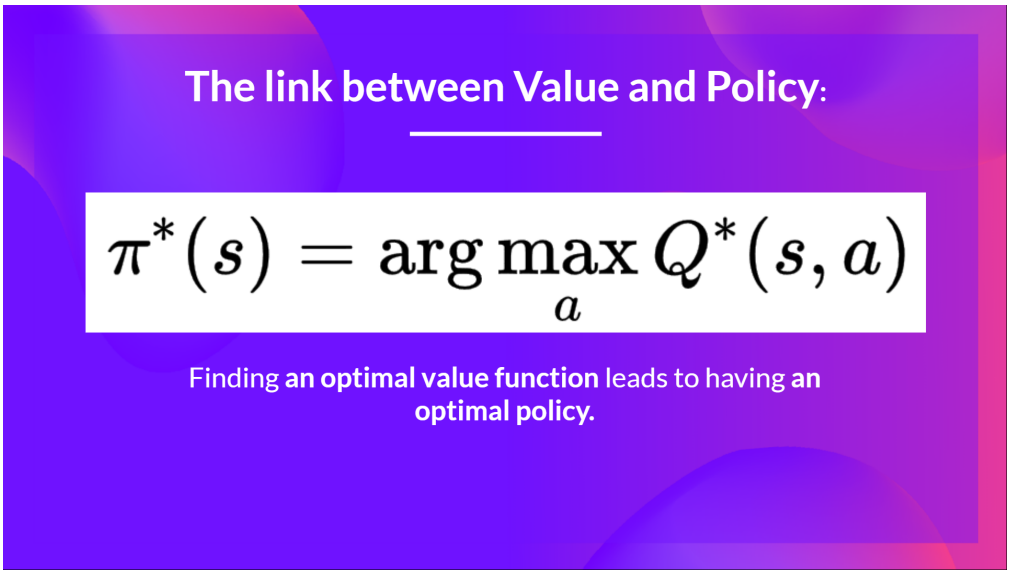


Рис. 9: Получение оптимальной политики из оптимальной функции ценности действий [оригинал]

На практике в методах на основе ценности чаще всего используют ε-**жадную политику**, которая балансирует исследование среды (exploration) и использование известной информации (exploitation). Подробнее об этом поговорим во второй части модуля при изучении Q-обучения.

Как отмечалось выше, существуют два типа функций ценности:

3.1 Функция ценности состояния

Функция ценности состояния для политики π записывается следующим образом:

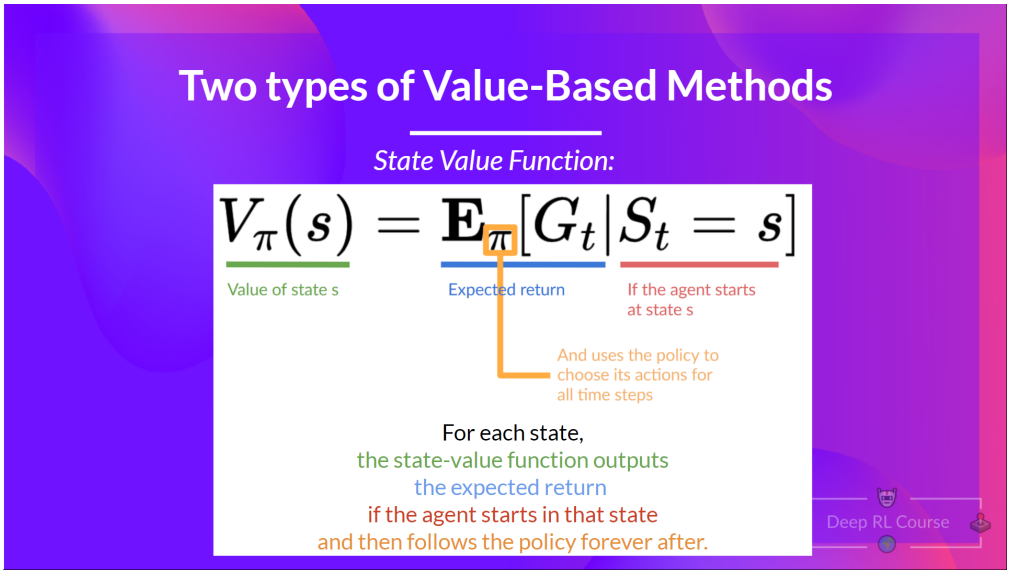


Рис. 10: Математическое определение функции ценности состояния [оригинал]

Для каждого состояния функция ценности состояния выдаёт ожидаемую отдачу, если агент **начинает из этого состояния**, а затем всё время следует политике (на всех последующих шагах).

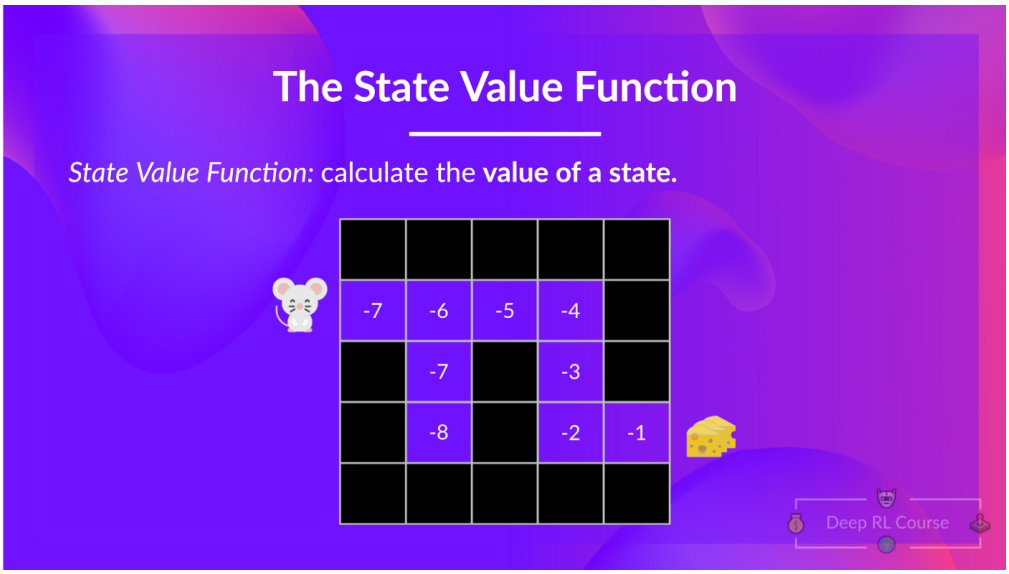


Рис. 11: Если взять состояние со значением -7, это ожидаемая отдача при старте из него и выборе действий согласно нашей (жадной) политике: вправо, вправо, вправо, вниз, вниз, вправо, вправо [оригинал]

3.2 Функция ценности действия

Функция ценности действия для каждой пары «состояние–действие» **выдаёт ожидаемую отдачу**, если агент начинает в этом состоянии, выполняет данное действие, а затем всё время следует политике.

Ценность выполнения действия a в состоянии s при политике π равна:

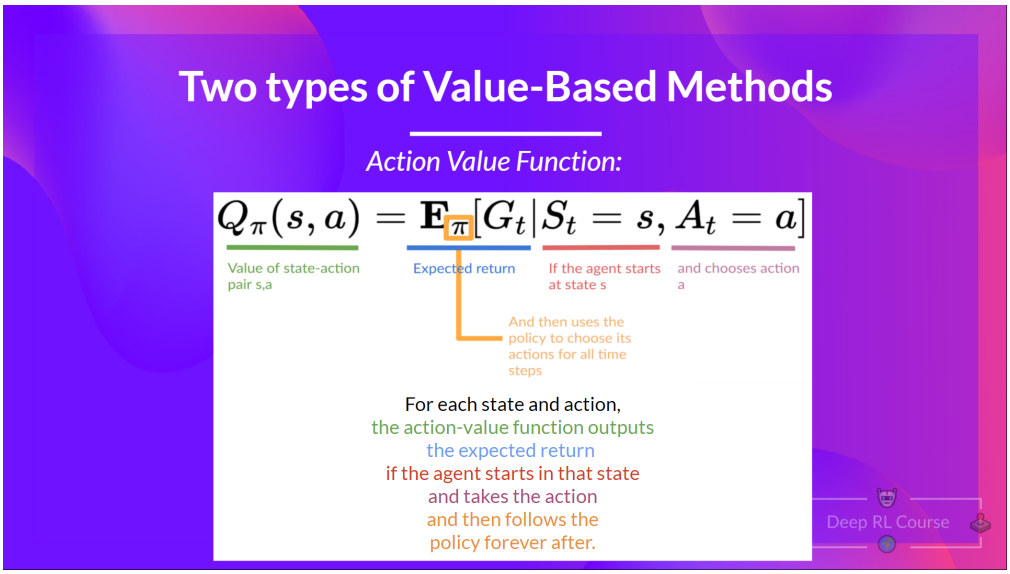


Рис. 12: Математическое определение функции ценности действия [оригинал]

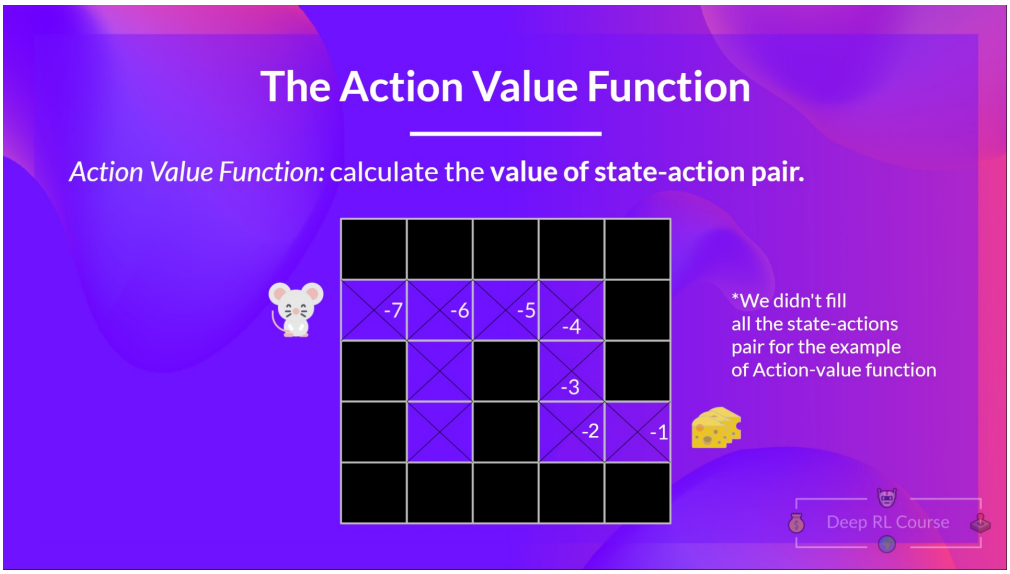


Рис. 13: Ценности действий для выбранного состояния и действия [оригинал]

Мы видим следующее различие:

- Для функции ценности состояния вычисляется **ценность состояния** S_t.
- Для функции ценности действия вычисляется **ценность пары «состояние–действие»** (S_t, A_t), то есть ценность выбора данного действия в данном состоянии.

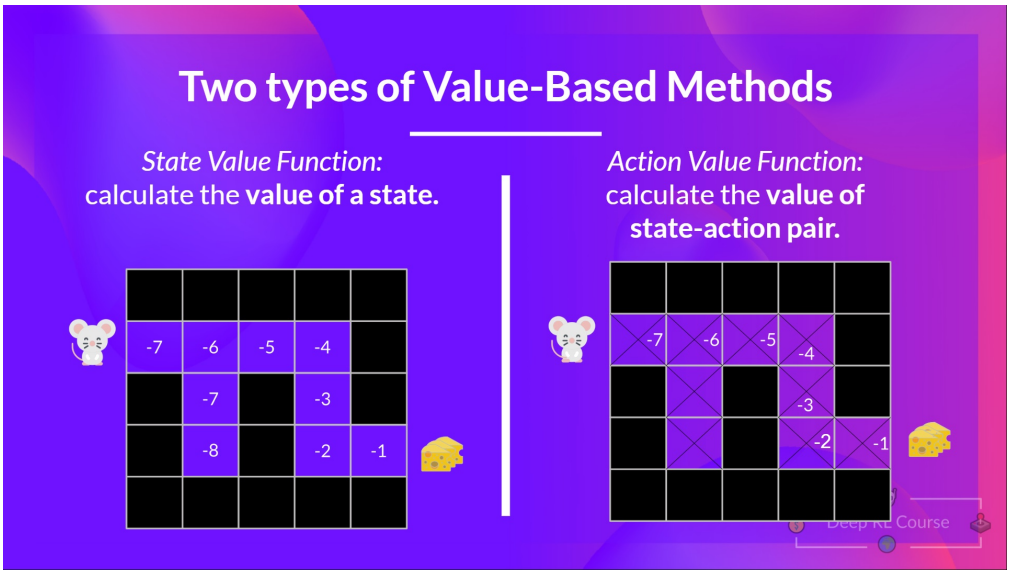


Рис. 14: Примечание: в примере функции ценности действий показаны не все пары «состояние–действие» [оригинал]

В обоих случаях, какую бы функцию ценности мы ни выбрали (состояний или действий), **возвращаемое значение представляет собой ожидаемую отдачу.**

Однако проблема заключается в том, что **для вычисления КАЖДОЙ ценности состояния или пары «состояние–действие» необходимо суммировать все вознаграждения, которые агент может получить, начиная из этого состояния.**

Это может потребовать больших вычислительных затрат. **Именно здесь нам помогает уравнение Беллмана.**

4 Уравнение Беллмана: упрощаем оценку ценности

Уравнение Беллмана **упрощает вычисление ценности состояния или пары «состояние–действие».**

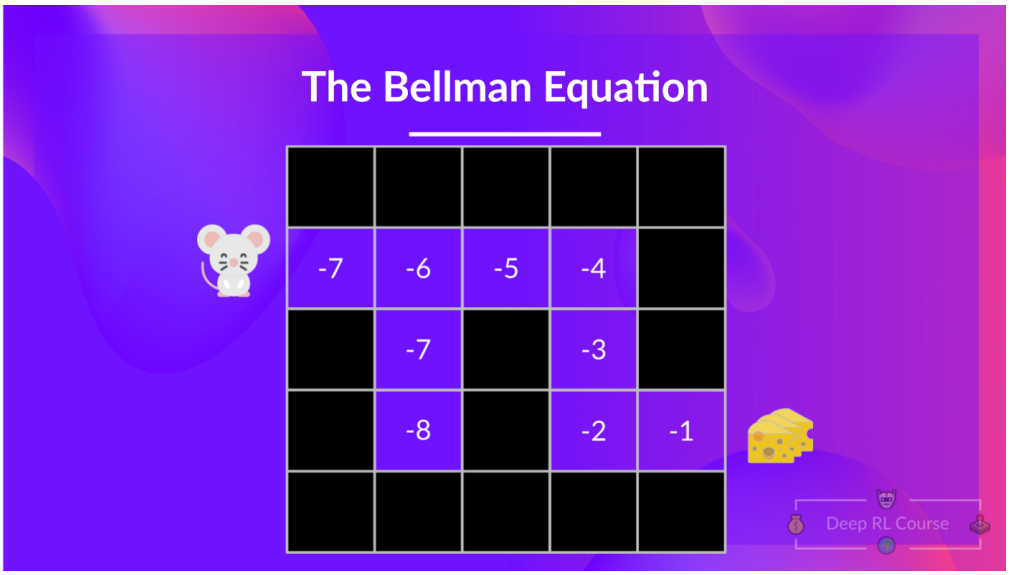


Рис. 15: Уравнение Беллмана и рекурсивная оценка ценности [оригинал]

Из уже изученного мы знаем, что для вычисления $V(S_t)$ (ценности состояния) нужно найти отдачу при старте из этого состояния и дальнейшем бесконечном следовании политике. **(В следующем примере политика является жадной; для простоты мы не дисконтируем вознаграждения.)**

Итак, чтобы вычислить $V(S_t)$, нужно найти сумму ожидаемых вознаграждений. Следовательно:

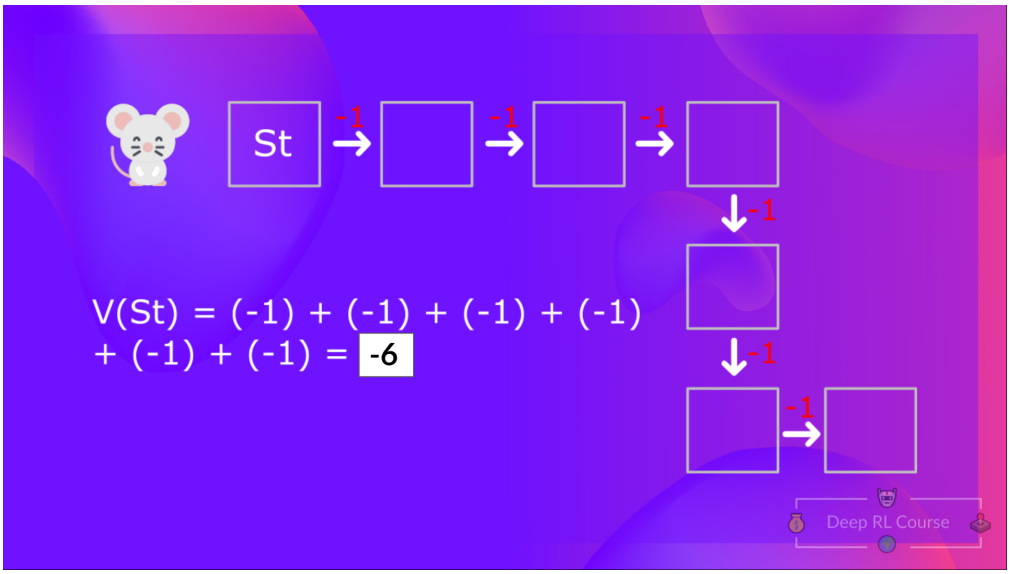


Рис. 16: Ценность состояния 1: сумма вознаграждений, если агент начинает из него и на всех шагах следует жадной политике, выбирая действия, ведущие к состояниям с наибольшей ценностью [оригинал]

Чтобы вычислить $V(S_{t+1})$, необходимо найти отдачу при старте из состояния S_{t+1}.

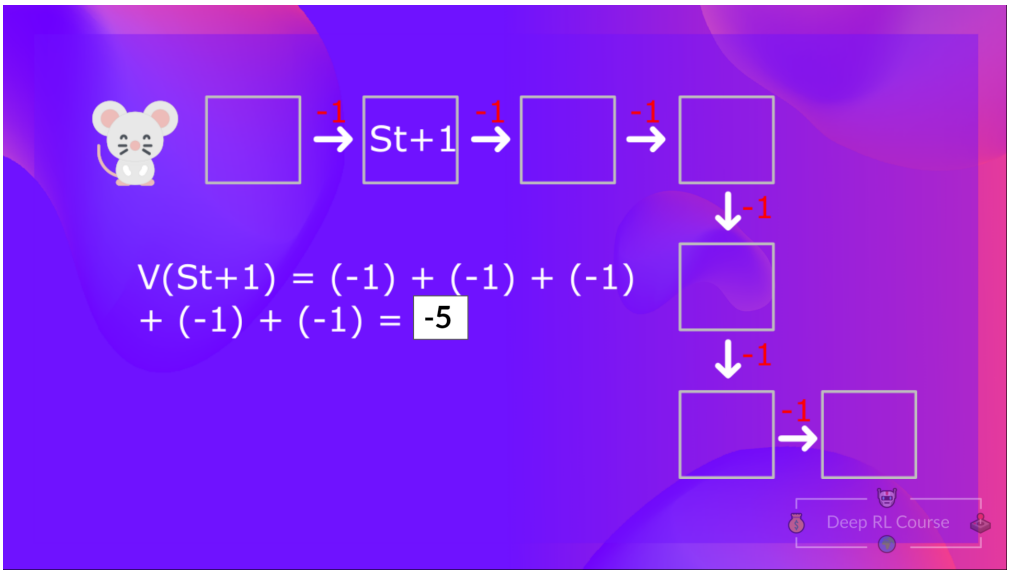


Рис. 17: Ценность состояния 2: сумма вознаграждений, если агент начинает из него и следует политике на всех шагах [оригинал]

Как вы, возможно, заметили, мы повторяем вычисление ценностей разных состояний. Это становится трудоёмким, когда нужно оценить каждое состояние или пару «состояние–действие».

Вместо прямого вычисления ожидаемой отдачи для каждого состояния или каждой пары **можно использовать уравнение Беллмана.** (Подсказка: если вы знакомы с динамическим программированием, здесь используется очень похожая идея. Если нет — ничего страшного!)

Уравнение Беллмана задаёт рекурсивную зависимость: вместо того чтобы каждый раз заново вычислять отдачу для состояния, мы можем представить его ценность следующим образом:

Непосредственное вознаграждение R_{t+1} **+ дисконтированная ценность следующего состояния** $(\gamma * V(S_{t+1}))$**.**

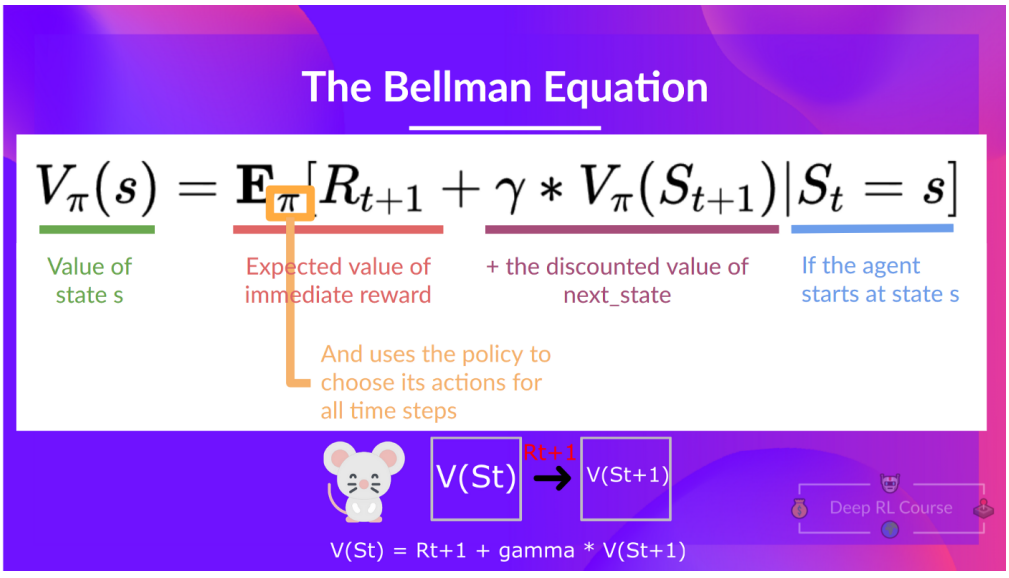


Рис. 18: Разложение по Беллману: непосредственное вознаграждение и дисконтированная ценность следующего состояния [оригинал]

Вернёмся к примеру: ценность состояния 1 равна ожидаемой суммарной отдаче, если начать именно в этом состоянии.

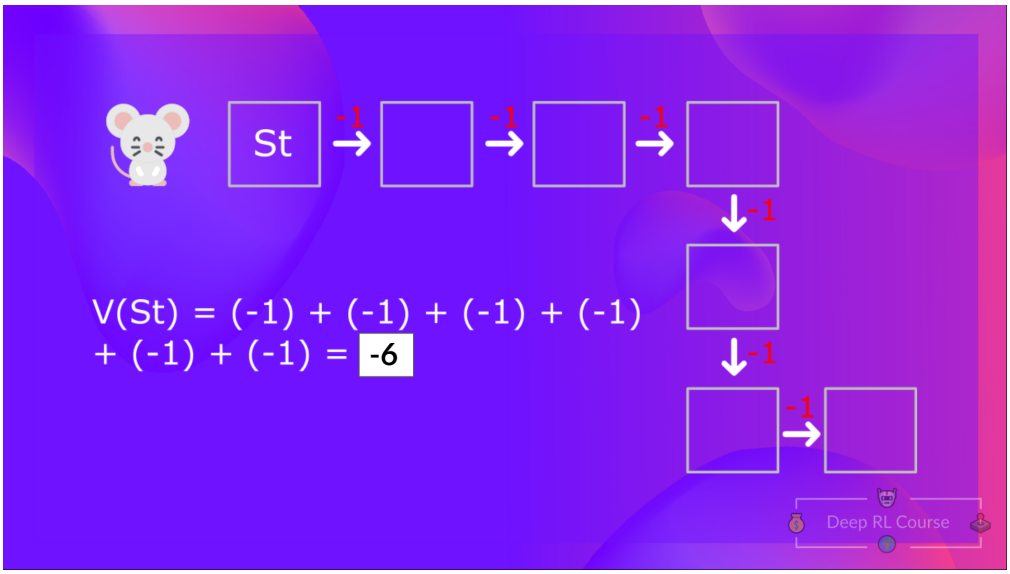


Рис. 19: Суммарная отдача при старте из первого состояния траектории [оригинал]

Чтобы вычислить ценность состояния 1, нужно сложить вознаграждения, **полученные при старте из состояния 1**, и дальнейшем следовании **политике на всех шагах.**

Это равносильно записи: $V(S_t)$ = непосредственное вознаграждение R_{t+1} + дисконтированная ценность следующего состояния $\gamma * V(S_{t+1})$.

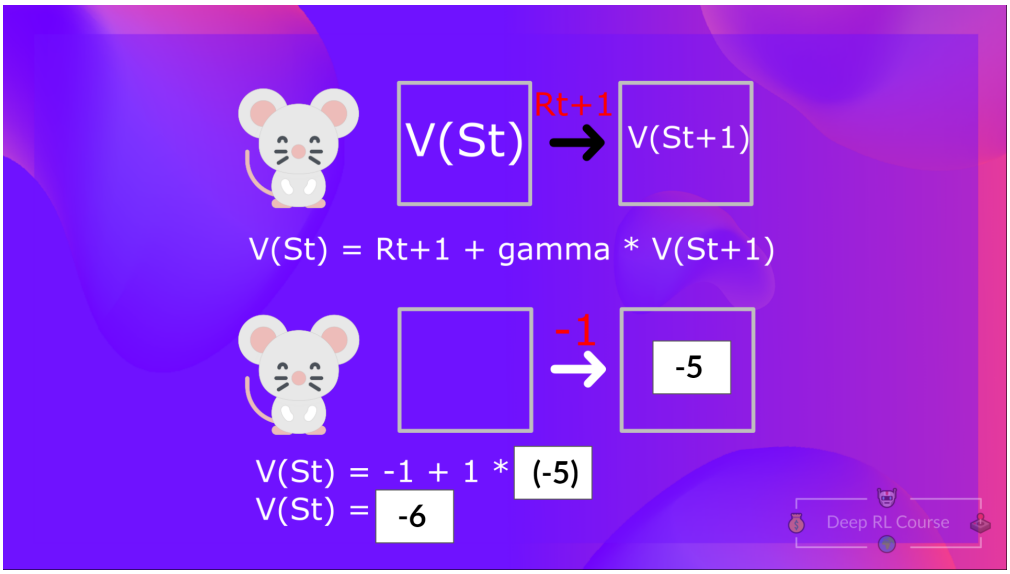


Рис. 20: Для простоты дисконтирование не применяется: $\gamma = 1$ [оригинал]

Ради простоты здесь мы не дисконтируем вознаграждения, поэтому $\gamma = 1$. Однако в разделе о Q-обучении этого модуля мы рассмотрим пример с $\gamma = 0.99$.

- Ценность $V(S_{t+1})$ = непосредственное вознаграждение R_{t+2} + дисконтированная ценность следующего состояния $(\gamma * V(S_{t+2}))$.
- И так далее.

Подведём итог: идея уравнения Беллмана состоит в том, что вместо вычисления каждой ценности как суммы ожидаемых вознаграждений, **что требует длительных вычислений**, мы представляем её как **сумму непосредственного вознаграждения и дисконтированной ценности следующего состояния.**

Прежде чем переходить к следующему разделу, подумайте о роли коэффициента γ в уравнении Беллмана. Что произойдёт при очень малом γ (например, 0.1 или даже 0)? Что будет при $\gamma = 1$?

А если он очень большой, например миллион?

5 Методы Монте-Карло и обучение с временными разностями

Прежде чем перейти непосредственно к Q-обучению, обсудим две стратегии обучения.

Напомним, RL-агент **учится через взаимодействие со средой**. Идея заключается в том, что **на основании полученного опыта и вознаграждения агент обновляет функцию ценности или политику.**

Методы Монте-Карло (Monte Carlo) и временных разностей (Temporal Difference, TD) — две разные **стратегии обучения функции ценности или политики**. Обе **используют накопленный опыт для решения задачи RL.**

С одной стороны, метод Монте-Карло использует **целый эпизод взаимодействия перед обновлением**. С другой стороны, TD-обучению достаточно **одного шага** $(S_t, A_t, R_{t+1}, S_{t+1})$ **для обновления.**

Объясним оба подхода **на примере метода на основе ценности.**

5.1 Монте-Карло: обучение в конце эпизода

Метод Монте-Карло ждёт завершения эпизода, вычисляет G_t (отдачу) и использует её **как целевое значение при обновлении** $V(S_t)$**.**

Таким образом, для обновления функции ценности нужен **полный эпизод взаимодействия со средой.**

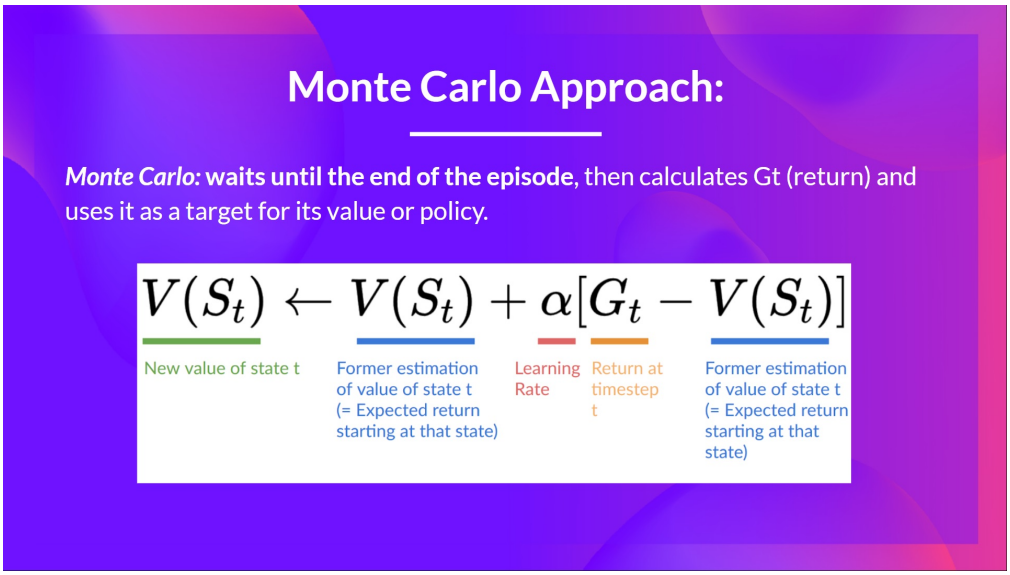


Рис. 21: Оценка ценности методом Монте-Карло по отдаче полного эпизода [оригинал]

Рассмотрим пример:

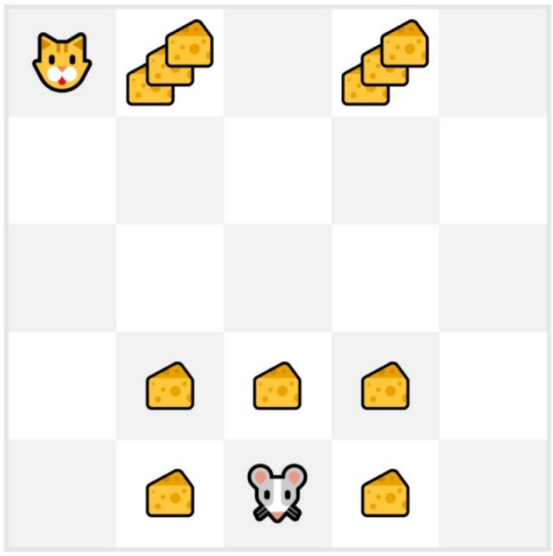


Рис. 22: Обучение Монте-Карло: пошаговый пример (этап 2) [оригинал]

- Каждый эпизод начинается **в одной и той же начальной точке.**
- **Агент выбирает действия согласно политике**. Например, с помощью ε-жадной стратегии, чередующей исследование среды (случайные действия) и использование известных знаний.
- Мы получаем **вознаграждение и следующее состояние.**
- Эпизод завершается, если кошка съедает мышь или мышь делает > 10 шагов.
- В конце эпизода **у нас есть список кортежей «состояние, действие, вознаграждение, следующее состояние».**

Например: [[нижняя клетка 3, двигаться влево, +1, нижняя клетка 2], [нижняя клетка 2, двигаться влево, +0, нижняя клетка 1]...]

- **Агент суммирует все вознаграждения в отдачу** G_t, чтобы оценить успешность своих действий.
- Затем **обновляет** $V(s_t)$ **по следующей формуле:**

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$

New value of state t

Former estimation of value of state t (= Expected return starting at that state)

Learning Rate

Return at timestep t

Former estimation of value of state t (= Expected return starting at that state)

Рис. 23: Обучение Монте-Карло: пошаговый пример (этап 3) [оригинал]

- После этого **начинается новая игра с учётом полученных знаний.**

С каждым новым эпизодом **агент постепенно учится играть всё лучше.**

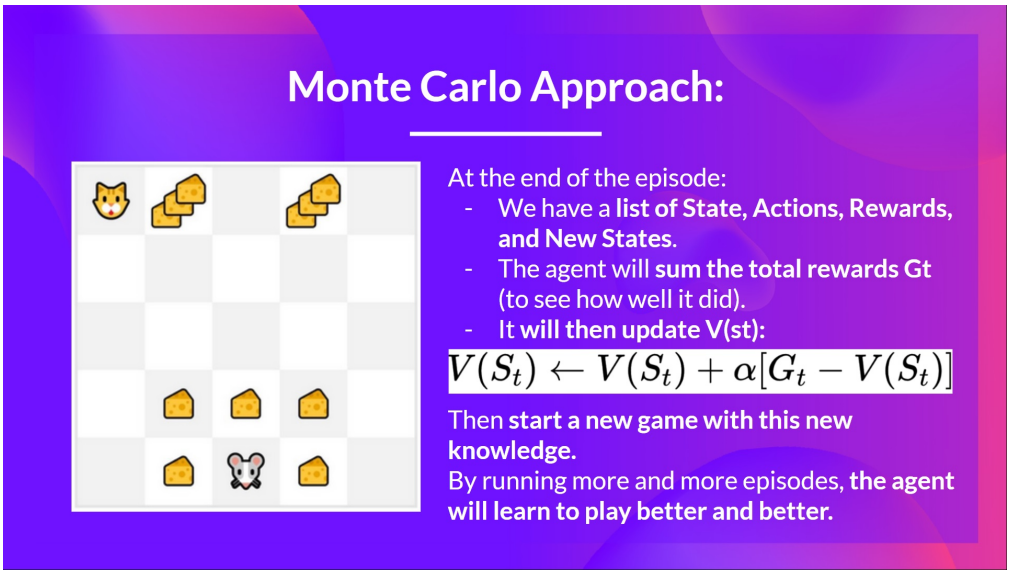


Рис. 24: Обучение Монте-Карло: пошаговый пример (этап 3) [оригинал]

Например, если обучать функцию ценности состояния методом Монте-Карло:

- Сначала мы инициализируем функцию ценности так, чтобы **она возвращала 0 для каждого состояния**.
- Скорость обучения (learning rate, lr) равна 0.1, а коэффициент дисконтирования равен 1 (то есть дисконтирования нет).
- Мышь **исследует среду и совершает случайные действия**.

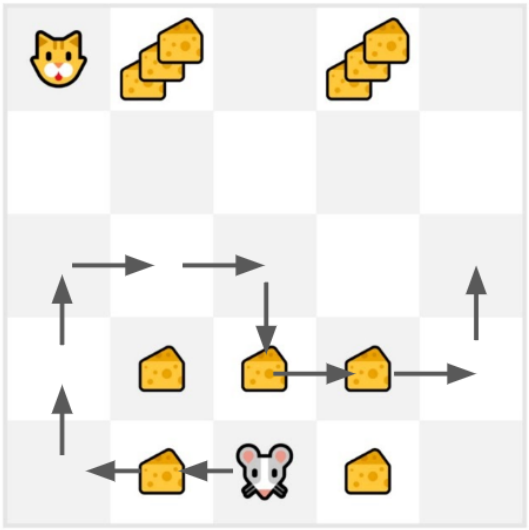


Рис. 25: Обучение Монте-Карло: пошаговый пример (этап 4) [оригинал]

- Мышь выполнила более 10 шагов, поэтому эпизод завершился.

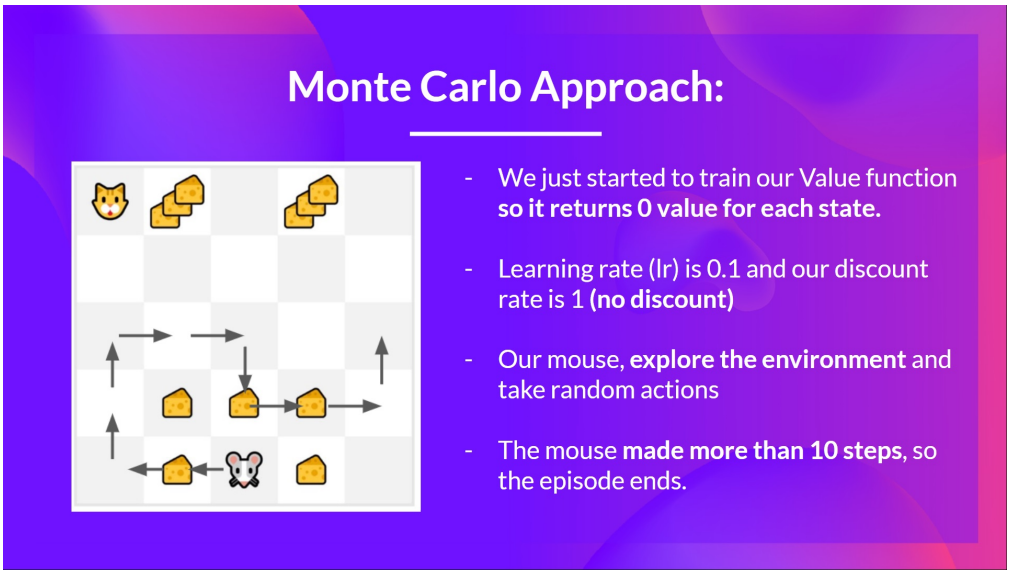


Рис. 26: Обучение Монте-Карло: пошаговый пример (этап 4) [оригинал]

- Имея список state, action, rewards, next_state, **нам нужно вычислить отдачу** $Gt = 0$.

$G_t = R_{t+1} + R_{t+2} + R_{t+3}$... (для простоты вознаграждения не дисконтируются).
$G_0 = R_1 + R_2 + R_3 \dots$
$G_0 = 1 + 0 + 0 + 0 + 0 + 0 + 1 + 1 + 0 + 0$
$G_0 = 3$

- Теперь можно вычислить **новое** значение $V(S_0)$:

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$

New value of state t | Former estimation of value of state t (= Expected return starting at that state) | Learning Rate | Return at timestep t | Former estimation of value of state t (= Expected return starting at that state)

Рис. 27: Обучение Монте-Карло: пошаговый пример (этап 5) [оригинал]

$V(S_0) = V(S_0) + lr * [G_0 - V(S_0)]$
$V(S_0) = 0 + 0.1 * [3 - 0]$
$V(S_0) = 0.3$

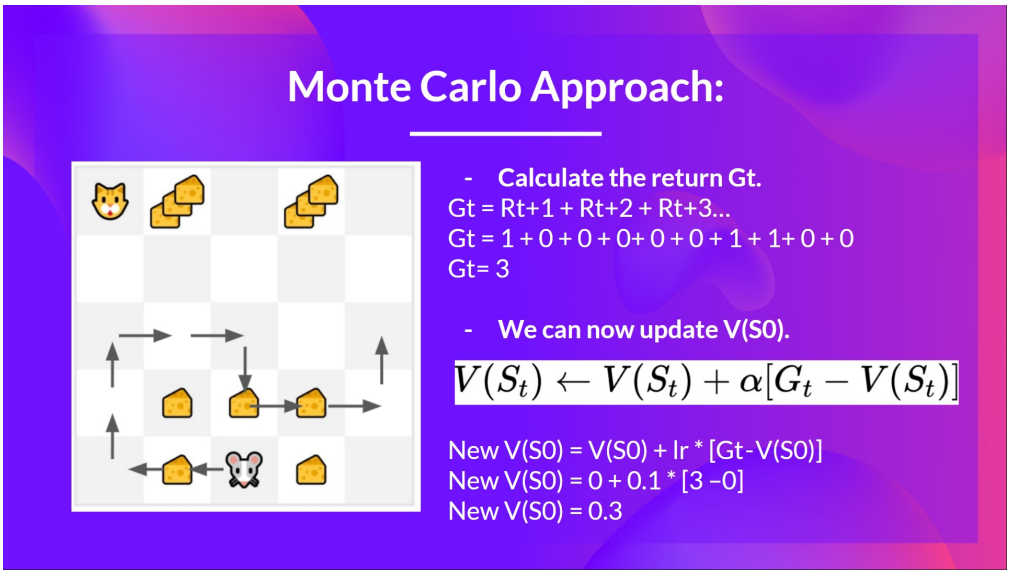


Рис. 28: Обучение Монте-Карло: пошаговый пример (этап 5) [оригинал]

5.2 Обучение с временными разностями: обновление на каждом шаге

При обучении с временными разностями достаточно одного взаимодействия (одного шага), дающего S_{t+1}, чтобы сформировать TD-цель и обновить $V(S_t)$, используя R_{t+1} и $\gamma * V(S_{t+1})$.

Идея **TD состоит в обновлении** $V(S_t)$ **на каждом шаге.**

Но поскольку полный эпизод ещё не пройден, мы не знаем G_t (ожидаемую отдачу). Вместо этого **мы оцениваем** G_t **как сумму** R_{t+1} **и дисконтированной ценности следующего состояния.**

Это называется *bootstrap-оцениванием*. Такое название связано с тем, что **TD-обновление частично опирается на уже имеющуюся оценку** $V(S_{t+1})$**, а не на полную наблюдённую отдачу** G_t**.**

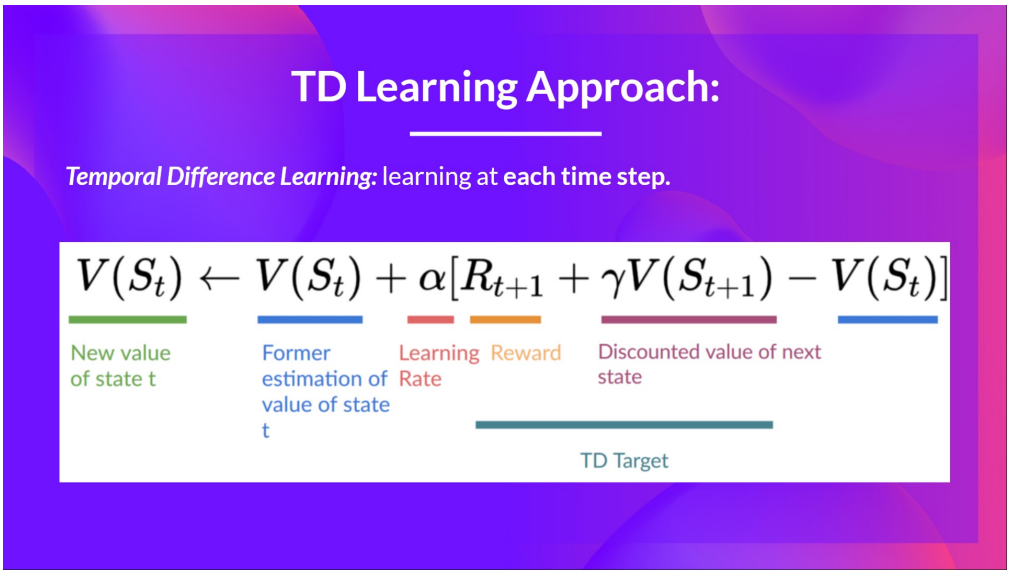


Рис. 29: Обучение с временными разностями: пошаговый пример (этап 1) [оригинал]

Этот метод называют TD(0), или **одношаговым TD (функция ценности обновляется после каждого отдельного шага).**



Рис. 30: Обучение с временными разностями: пошаговый пример (этап 1) [оригинал]

Возьмём тот же пример.

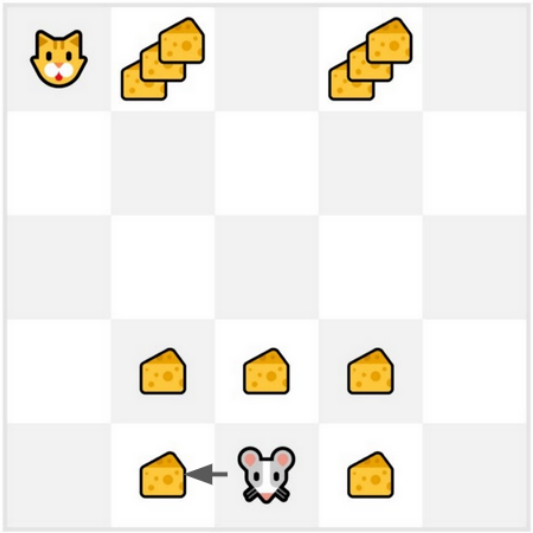


Рис. 31: Обучение с временными разностями: пошаговый пример (этап 2) [оригинал]

- Инициализируем функцию ценности так, чтобы для каждого состояния она возвращала ноль.
- Скорость обучения (lr) равна 0.1, коэффициент дисконтирования — 1 (без дисконтирования).
- Мышь начинает исследовать среду и выполняет случайное действие: **движется влево**.
- Она получает вознаграждение $R_{t+1} = 1$, потому что **съела кусочек сыра**.

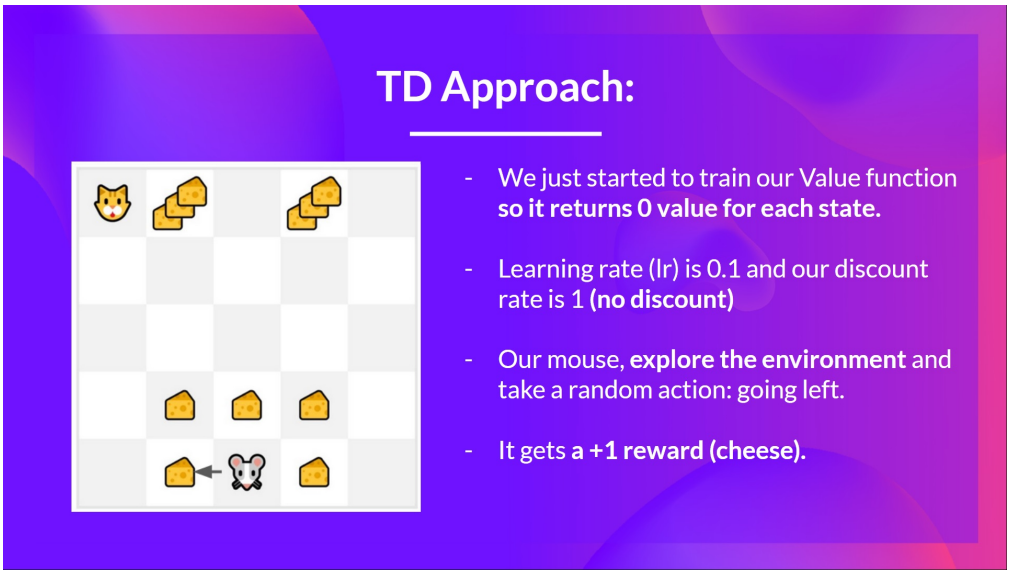


Рис. 32: Обучение с временными разностями: пошаговый пример (этап 2) [оригинал]

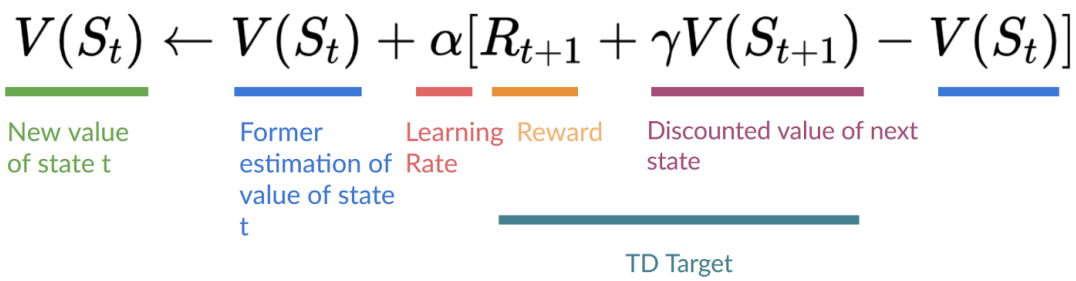


Рис. 33: Обучение с временными разностями: пошаговый пример (этап 3) [оригинал]

Теперь можно обновить $V(S_0)$:

Новое значение $V(S_0) = V(S_0) + lr * [R_1 + \gamma * V(S_1) - V(S_0)]$.

Новое значение $V(S_0) = 0 + 0.1 * [1 + 1 * 0 - 0]$.

Новое значение $V(S_0) = 0.1$.

Таким образом, мы только что обновили функцию ценности для состояния 0.

Теперь **продолжим взаимодействие со средой, используя обновлённую функцию ценности.**

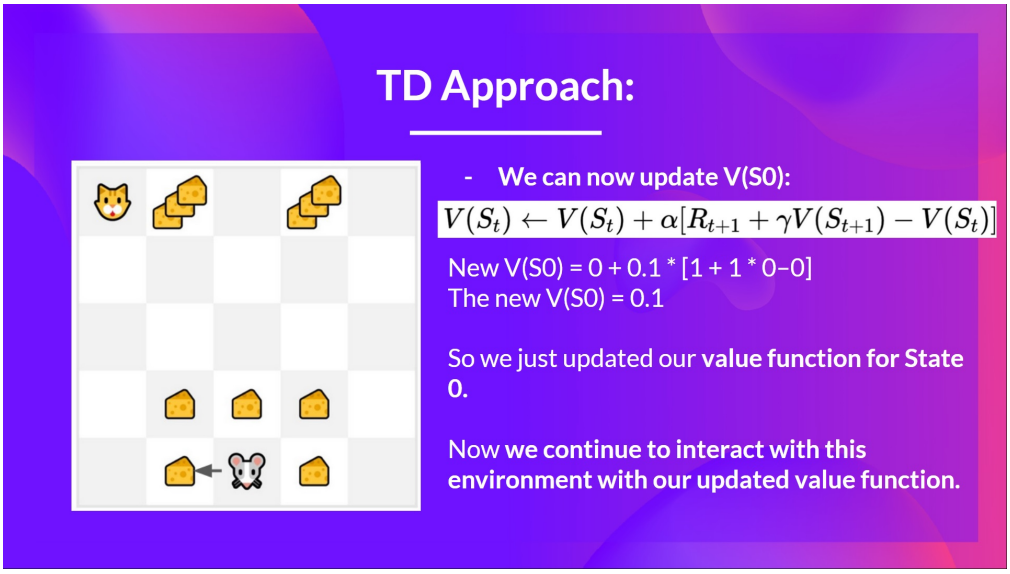


Рис. 34: Обучение с временными разностями: пошаговый пример (этап 3) [оригинал]

Подведём итоги:

- В *методе Монте-Карло* функция ценности обновляется по завершении целого эпизода, поэтому **используется фактическая дисконтированная отдача этого эпизода.**
- При *TD-обучении* функция ценности обновляется после одного шага; неизвестную величину G_t заменяет **оценка отдачи, называемая TD-целью (TD target).**

$$\text{Monte Carlo: } V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$

$$\text{TD Learning: } V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

Рис. 35: Сравнение обучения Монте-Карло и обучения с временными разностями [оригинал]

6 Промежуточное повторение

Прежде чем перейти к Q-обучению, подведём итог изученному.

Существуют два типа функций ценности:

- Функция ценности состояния: возвращает ожидаемую отдачу, если **агент начинает из заданного состояния и затем всегда следует политике.**
- Функция ценности действия: возвращает ожидаемую отдачу, если **агент начинает из заданного состояния, выполняет в нём заданное действие**, а затем всегда следует политике.
- В методах на основе ценности вместо обучения политики **мы задаём политику вручную** и обучаем функцию ценности. Если найдена оптимальная функция ценности, **можно получить оптимальную политику.**

Существуют два способа обновления функции ценности:

- В *методе Монте-Карло* обновление выполняется после целого эпизода, поэтому **используется фактическая дисконтированная отдача этого эпизода.**
- При *TD-обучении* обновление выполняется после одного шага; неизвестное G_t заменяется **оценкой отдачи — TD-целью.**

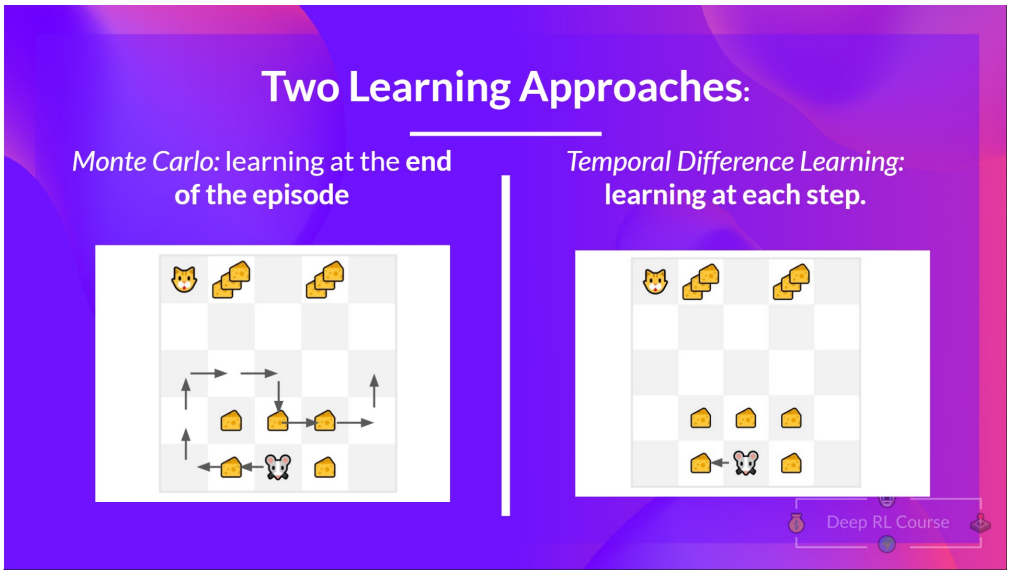


Рис. 36: Краткое сравнение методов обучения [оригинал]

7 Промежуточный тест

Лучший способ учиться и избежать иллюзии компетентности — **проверять себя**. Это поможет выявить **пробелы, требующие повторения**.

Вопрос 1. Какие два основных подхода используются для поиска оптимальной политики?

Выберите все подходящие ответы.

- **A.** Методы на основе политики
- **B.** Методы на основе случайного выбора
- **C.** Методы на основе ценности
- **D.** Методы эволюционных стратегий

Вопрос 2. Что такое уравнение Беллмана?

Вопрос 3. Определите каждую составляющую уравнения Беллмана

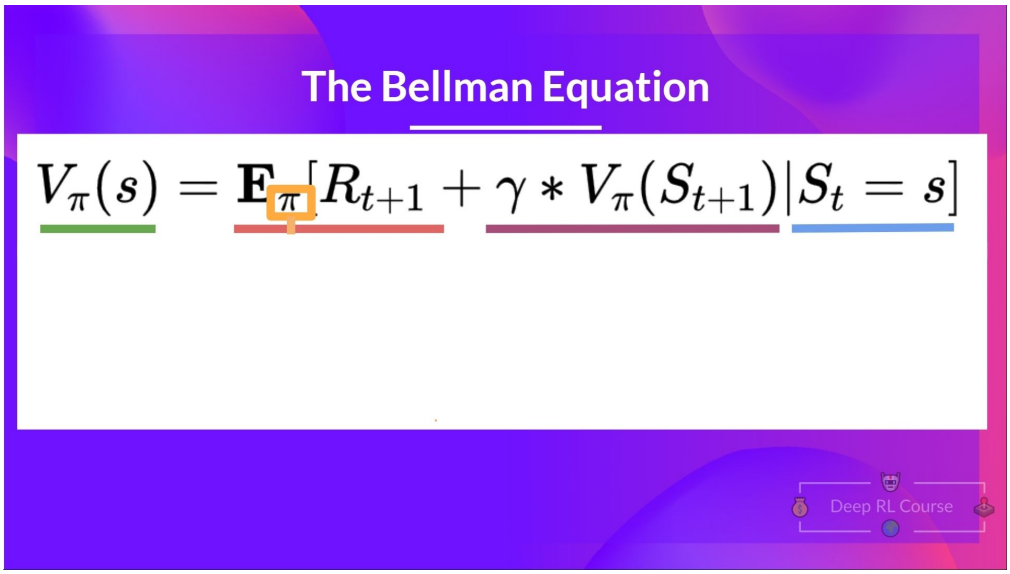


Рис. 37: Задание по уравнению Беллмана [оригинал]

Вопрос 4. Чем различаются методы Монте-Карло и обучения с временными разностями?

Выберите все подходящие ответы.

- **A.** При использовании методов Монте-Карло функция ценности обновляется после полного эпизода
- **B.** При использовании методов Монте-Карло функция ценности обновляется после одного шага
- **C.** При TD-обучении функция ценности обновляется после полного эпизода
- **D.** При TD-обучении функция ценности обновляется после одного шага

Вопрос 5. Объясните каждую часть формулы TD-обучения

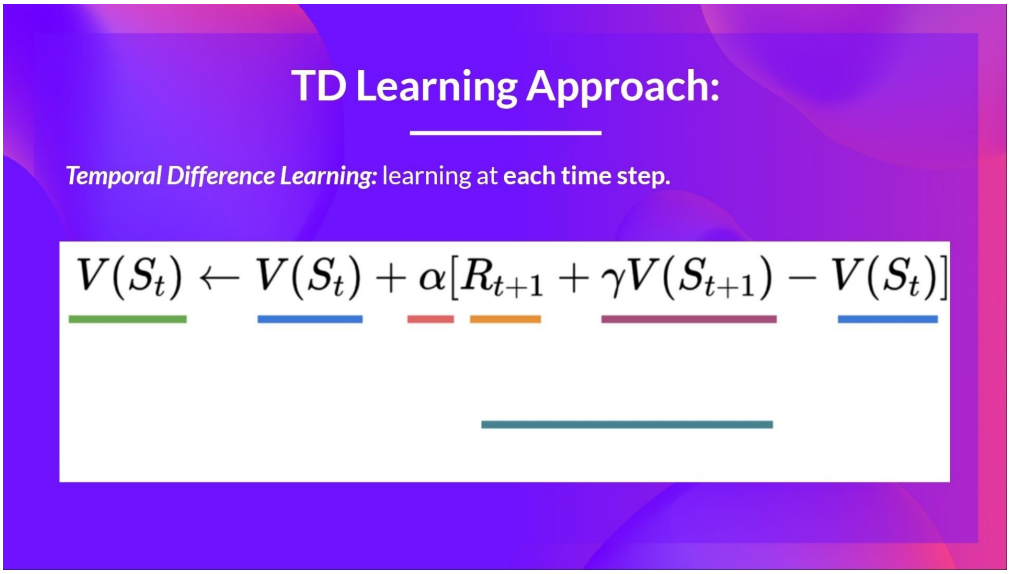


Рис. 38: Упражнение по обучению с временными разностями [оригинал]

Вопрос 6. Объясните каждую часть формулы Монте-Карло

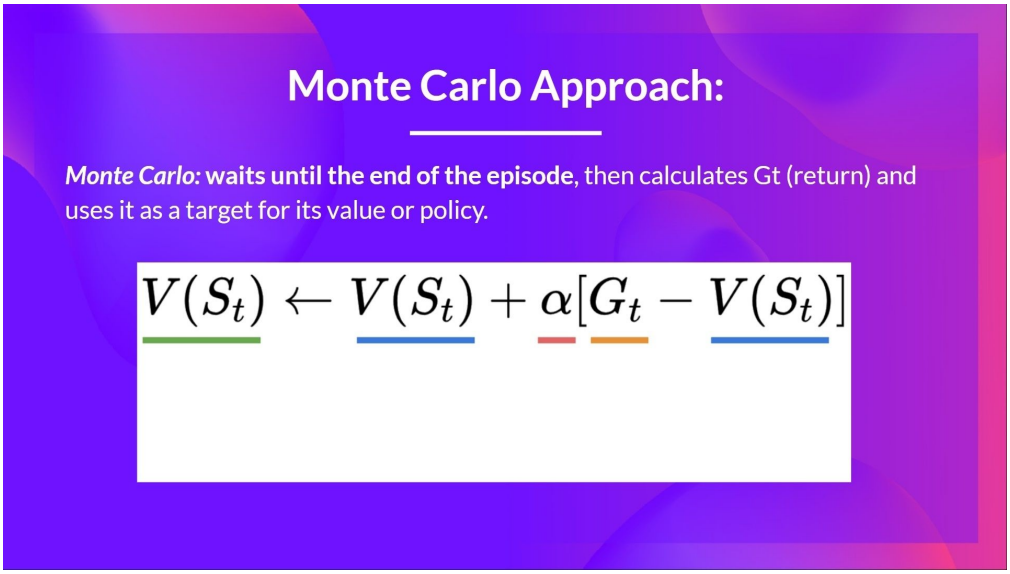


Рис. 39: Упражнение по методу Монте-Карло [оригинал]

7.1 Ответы и объяснения

Вопрос 1. Какие два основных подхода используются для поиска оптимальной политики?

Правильные ответы: A, C

- **Вариант A (верно):** методы на основе политики обучают саму политику, чтобы определять действие по заданному состоянию.
- **Вариант C (верно):** методы на основе ценности обучают функцию ценности, определяющую предпочтительность состояний, а затем используют её для выбора действия, ведущего к более ценному состоянию.

Вопрос 2. Что такое уравнение Беллмана?

Уравнение Беллмана задаёт рекурсивную зависимость: вместо того чтобы отдельно вычислять полную отдачу для каждого состояния, можно представить его ценность как:

Rt+1 + gamma * V(St+1)

Непосредственное вознаграждение + дисконтированная ценность следующего состояния.

Вопрос 3. Объясните каждую составляющую уравнения Беллмана

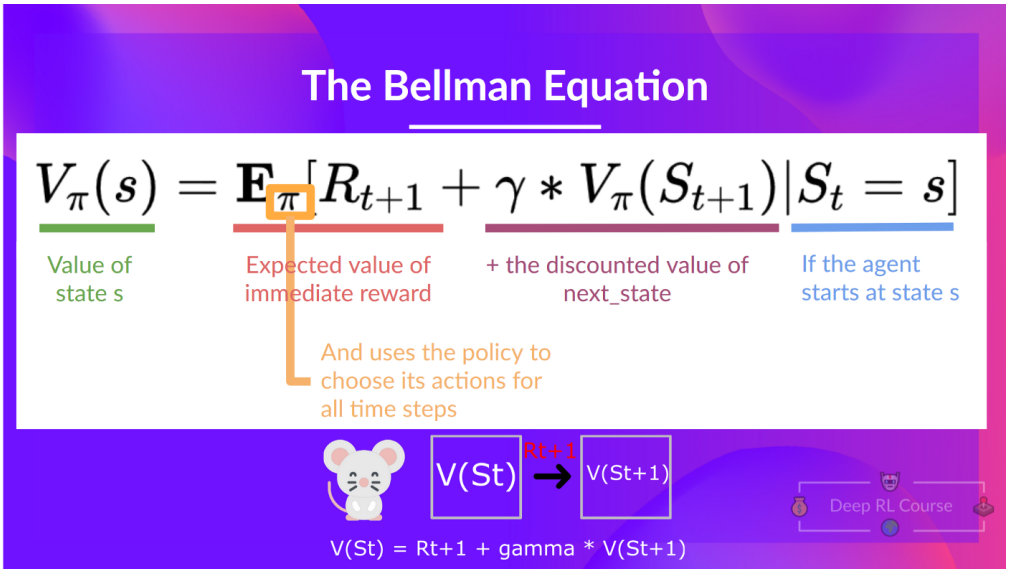


Рис. 40: Разложение Беллмана на непосредственное вознаграждение и дисконтированную ценность следующего состояния [оригинал]

Вопрос 4. Чем различаются методы Монте-Карло и TD-обучения?

Правильные ответы: A, D

Вопрос 5. Объясните каждую часть формулы обучения с временными разностями

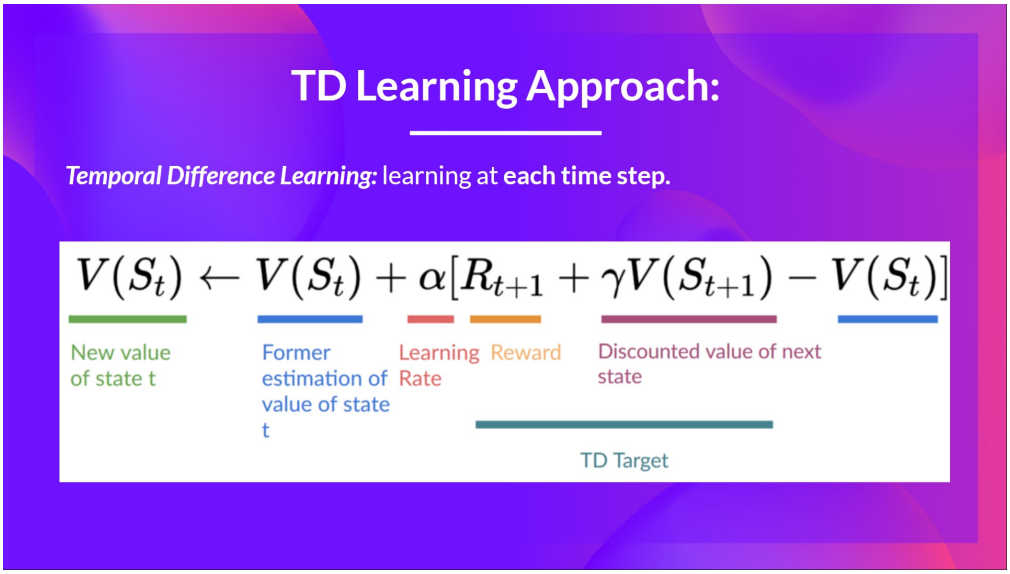


Рис. 41: Обучение с временными разностями: пошаговый пример (этап 1) [оригинал]

Вопрос 6. Объясните каждую часть формулы Монте-Карло

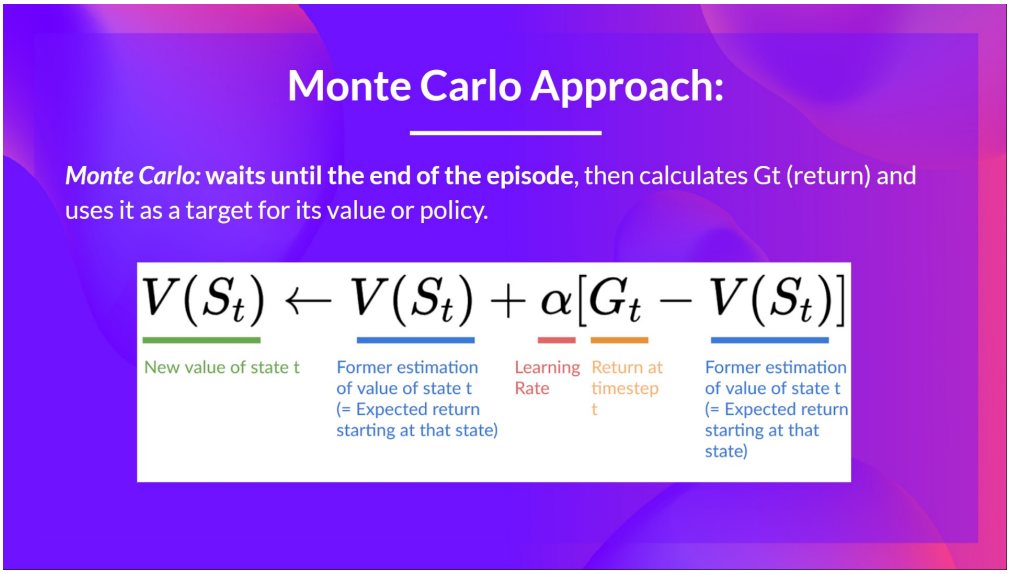


Рис. 42: Оценка ценности методом Монте-Карло по полной отдаче эпизода [оригинал]

Поздравляем с завершением теста! Если вы ответили не на всё, вернитесь к предыдущим разделам и ещё раз разберите сложные моменты, чтобы закрепить знания.

8 Знакомство с Q-обучением

8.1 Что такое Q-обучение?

Q-обучение — это **off-policy метод на основе ценности, использующий TD-подход для обучения функции ценности действий:**

- *Off-policy*: об этом понятии мы поговорим в конце модуля.
- *Метод на основе ценности*: находит оптимальную политику косвенно, обучая функцию ценности состояний или действий, которая определяет **ценность каждого состояния или пары «состояние–действие».**
- *TD-подход:* **обновляет функцию ценности действий на каждом шаге, а не в конце эпизода.**

Q-обучение — это алгоритм обучения Q-функции, то есть **функции ценности действий**, определяющей ценность нахождения в заданном состоянии и выполнения в нём заданного действия.

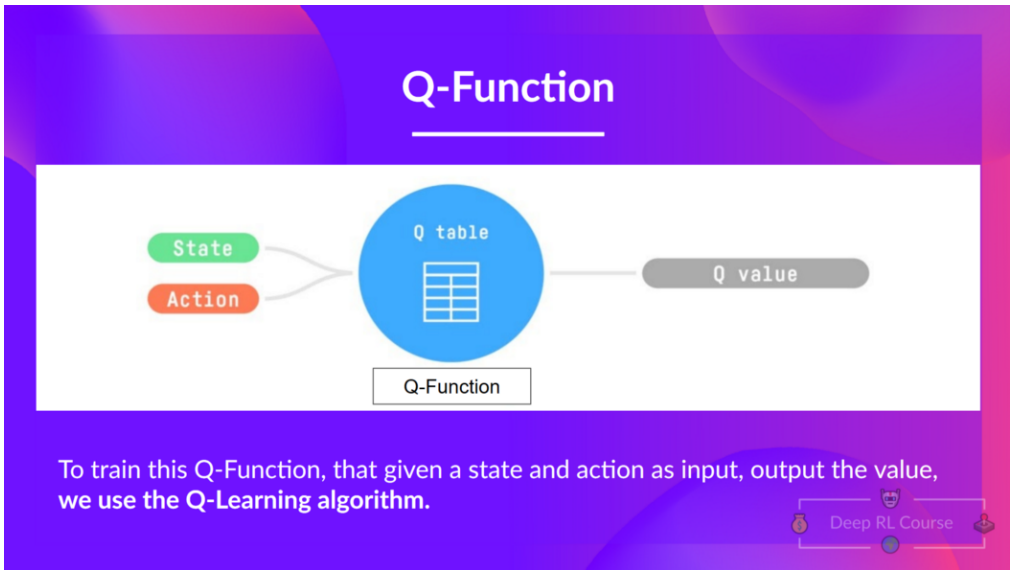


Рис. 43: По состоянию и действию Q-функция возвращает ценность пары «состояние–действие», также называемую Q-значением [оригинал]

Буква Q происходит от слова «Quality» — качества (ценности) данного действия в данном состоянии.

Напомним различие между ценностью и вознаграждением:

- *Ценность состояния* или *пары «состояние–действие»* — это ожидаемое суммарное вознаграждение, которое получит агент, начиная из данного состояния (или пары) и далее следуя политике.
- *Вознаграждение* — это **сигнал обратной связи от среды** после выполнения действия в конкретном состоянии.

Внутри Q-функция представляется **Q-таблицей: каждая её ячейка соответствует ценности пары «состояние–действие».** Можно считать Q-таблицу **памятью или шпаргалкой нашей Q-функции.**

Рассмотрим пример с лабиринтом.



Рис. 44: Q-таблица на примере лабиринта с мышью и сыром (схема 1) [оригинал]

Q-таблица только что инициализирована, поэтому все значения равны нулю. **Для каждого состояния и действия в ней хранится соответствующая ценность.** В этом простом

примере состояние определяется только положением мыши. Следовательно, в Q-таблице будет 2∗3 строки: по одной на каждую возможную позицию мыши. В более сложных задачах состояние может содержать гораздо больше информации, чем положение агента.

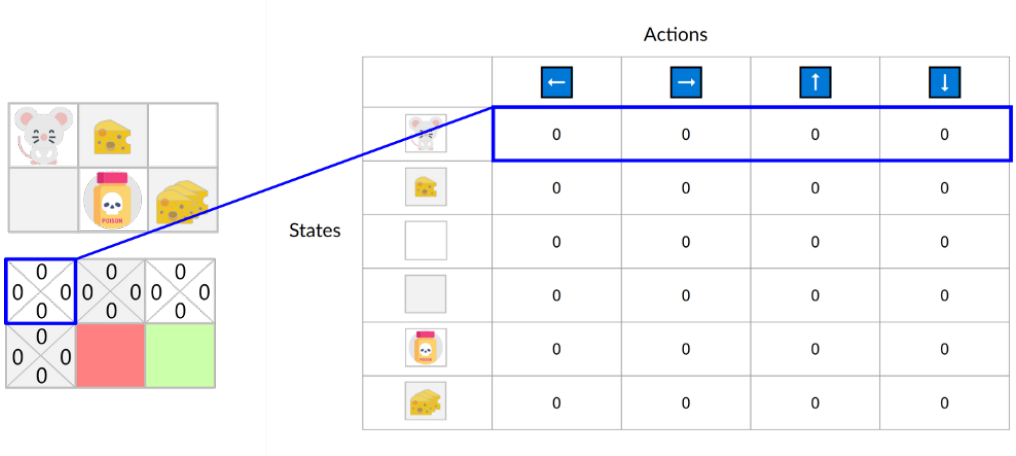


Рис. 45: Q-таблица на примере лабиринта с мышью и сыром (схема 2) [оригинал]

Здесь видно, что **ценность пары «начальное состояние, действие вверх» равна 0:**

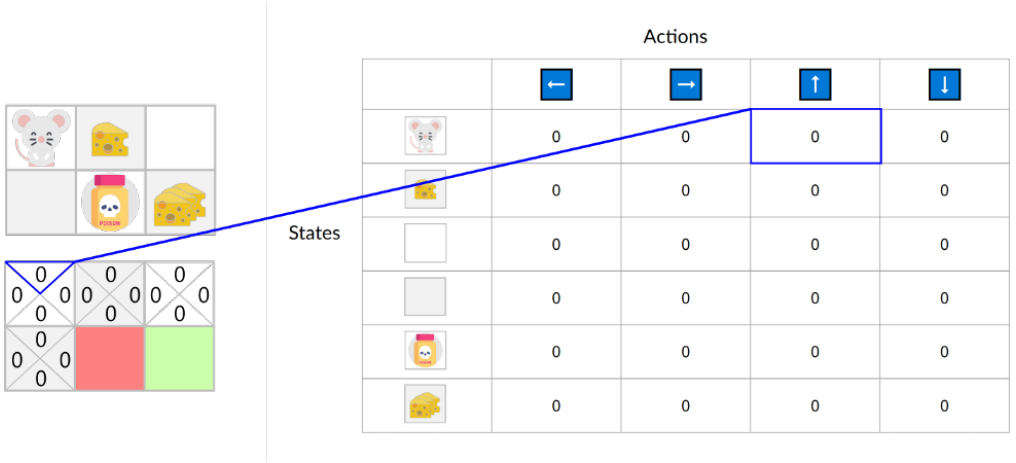


Рис. 46: Q-таблица на примере лабиринта с мышью и сыром (схема 3) [оригинал]

Таким образом, Q-функция использует Q-таблицу, **содержащую ценность каждой пары «состояние–действие».** Получив состояние и действие, **Q-функция ищет в таблице соответствующее значение и возвращает его.**

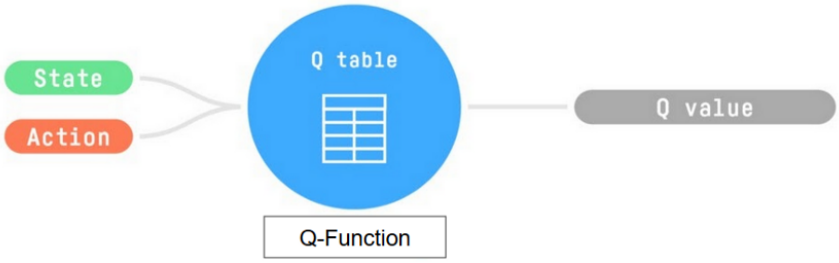


Рис. 47: Поиск ценности пары «состояние–действие» в Q-таблице [оригинал]

Итак, *Q-обучение* — это **алгоритм RL, который:**

- Обучает *Q-функцию* (**функцию ценности действий**), представленную внутри **Q-таблицей со значениями для всех пар «состояние–действие»**.
- По заданным состоянию и действию **находит в Q-таблице соответствующую ценность**.

- После завершения обучения **даёт оптимальную Q-функцию, а значит, и оптимальную Q-таблицу**.
- Если **Q-функция оптимальна**, то **оптимальна и политика**: мы **знаем наилучшее действие для каждого состояния**.

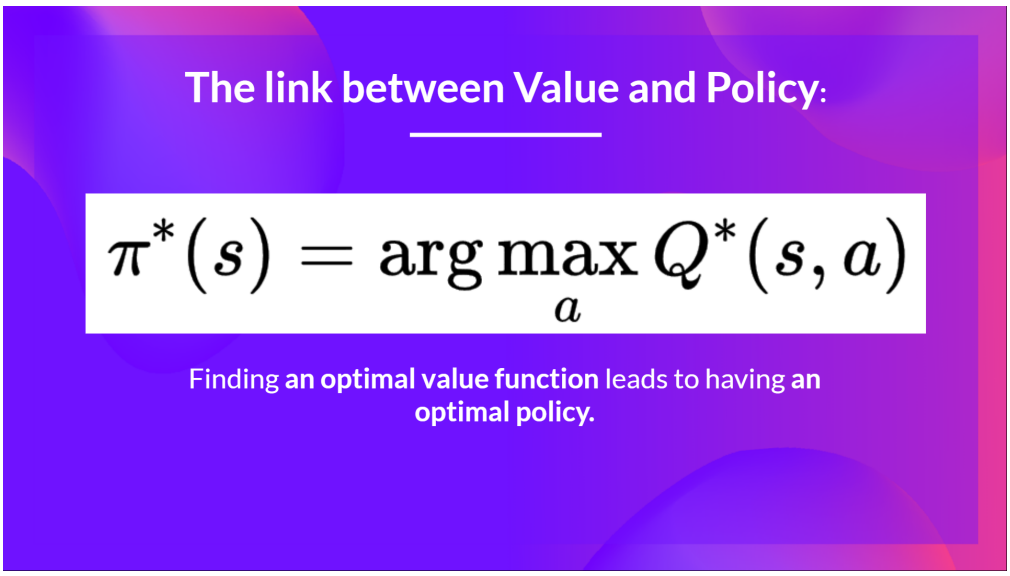


Рис. 48: Восстановление оптимальной политики из оптимальной функции ценности действий [оригинал]

Сначала **Q-таблица бесполезна, поскольку содержит произвольные значения для всех пар «состояние–действие»** (обычно мы заполняем её нулями). По мере того как агент **исследует среду, а мы обновляем Q-таблицу, она всё лучше приближает** оптимальную функцию ценности и соответствующую политику.

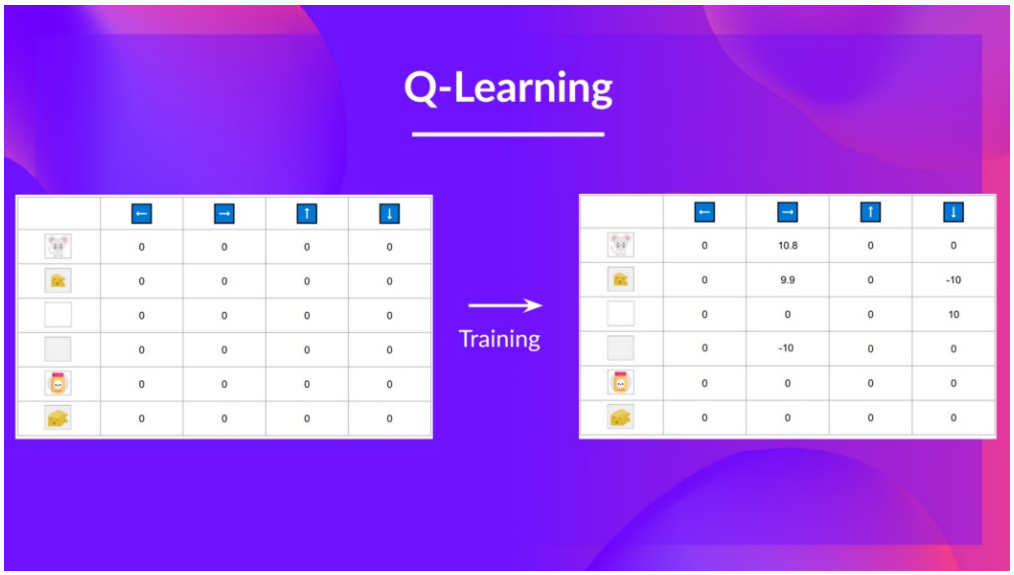


Рис. 49: По мере обучения Q-таблица становится точнее и позволяет оценивать полезность каждой пары «состояние–действие» [оригинал]

Теперь, когда мы разобрались с Q-обучением, Q-функцией и Q-таблицей, **рассмотрим сам алгоритм Q-обучения подробнее**.

8.2 Алгоритм Q-обучения

Перед нами псевдокод Q-обучения. Разберём каждую его часть и **посмотрим на простом примере, как он работает, прежде чем реализовывать его.** Не пугайтесь: всё проще, чем

кажется! Пройдём по шагам.

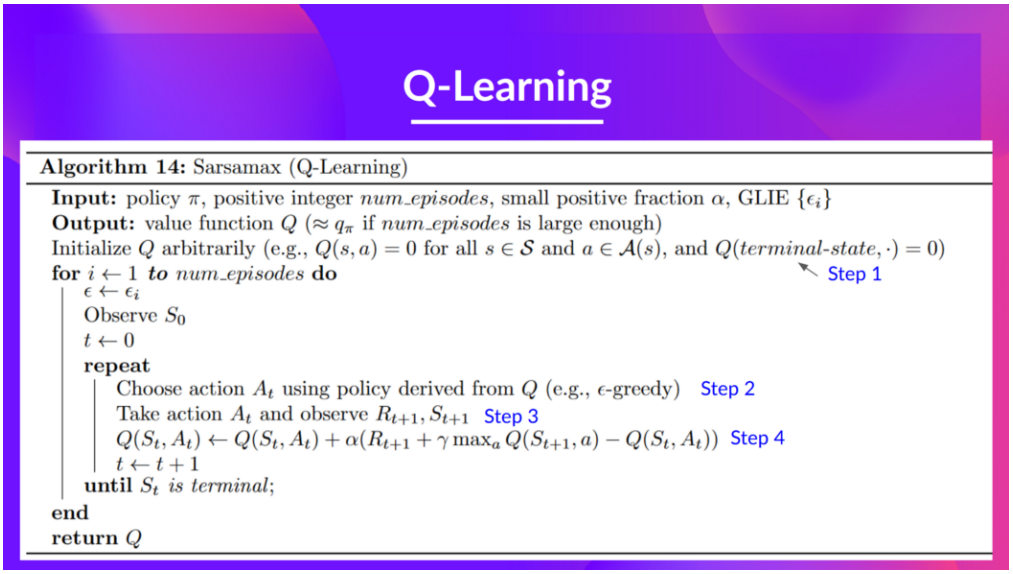


Рис. 50: Алгоритм Q-обучения и псевдокод обновления [оригинал]

Шаг 1. Инициализация Q-таблицы

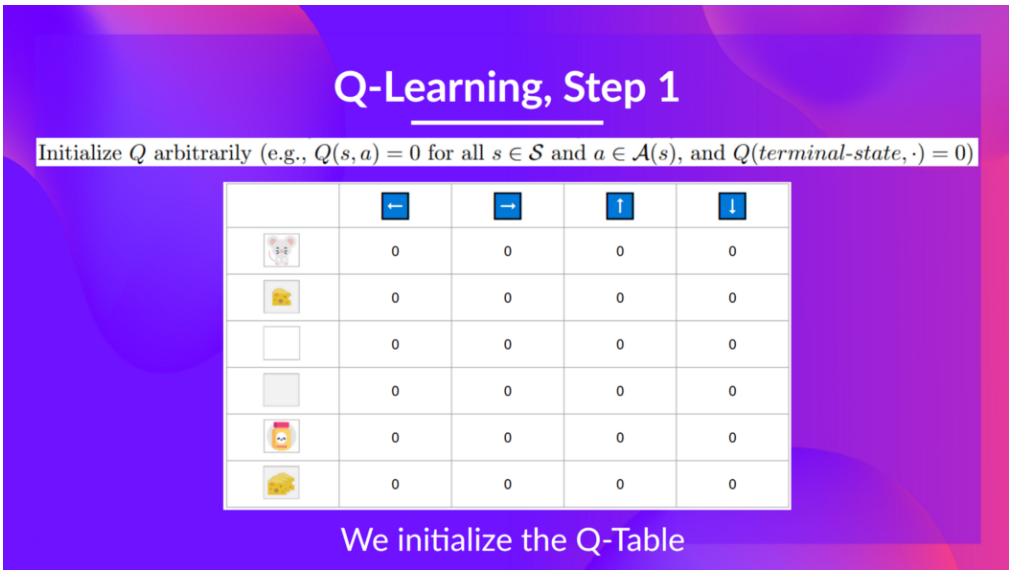


Рис. 51: Q-обучение: выбор действия с помощью ε-жадной политики [оригинал]

Необходимо инициализировать Q-таблицу для всех пар «состояние–действие». **Чаще всего все значения инициализируют нулями.**

Шаг 2. Выбор действия с помощью ε-жадной стратегии

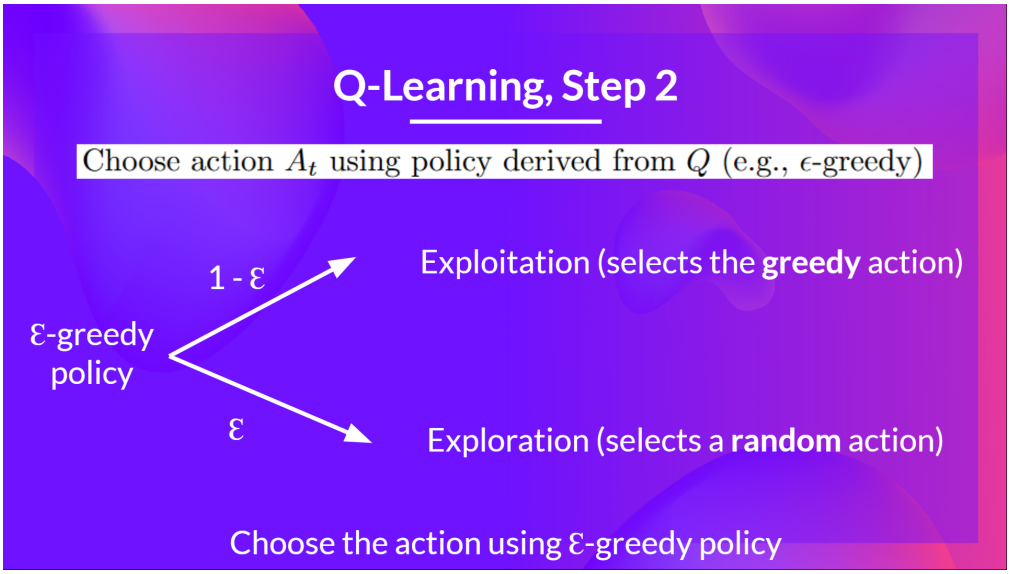


Рис. 52: ε-жадная стратегия: исследование и использование полученных знаний [оригинал]

ε-жадная стратегия — это политика, обеспечивающая компромисс между исследованием среды (exploration) и использованием известных знаний (exploitation).

Идея состоит в следующем. При начальном значении $\varepsilon = 1.0$:

- *С вероятностью* $1 - \varepsilon$*:* выполняется **использование знаний (exploitation)** — агент выбирает действие с наибольшей ценностью пары «состояние–действие».
- *С вероятностью* ε*:* выполняется **исследование (exploration)** — выбирается случайное действие.

В начале обучения **вероятность исследования велика, поскольку** ε **близко к единице, и агент большую часть времени исследует среду.** По мере обучения **Q-таблица всё точнее оценивает действия, поэтому мы постепенно уменьшаем** ε: необходимость исследования снижается, а использование накопленных знаний становится важнее.

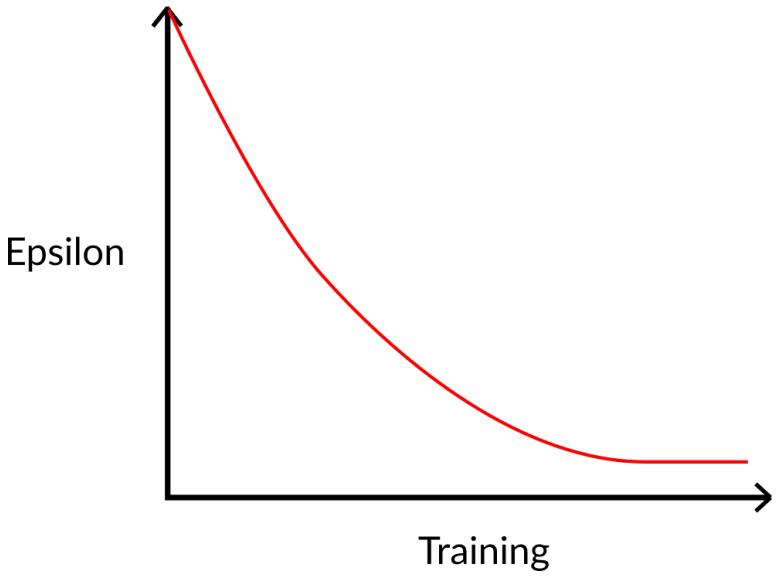


Рис. 53: Q-обучение: выполнение действия и наблюдение перехода [оригинал]

Шаг 3. Выполняем действие A_t, получаем вознаграждение R_{t+1} и следующее состояние S_{t+1}

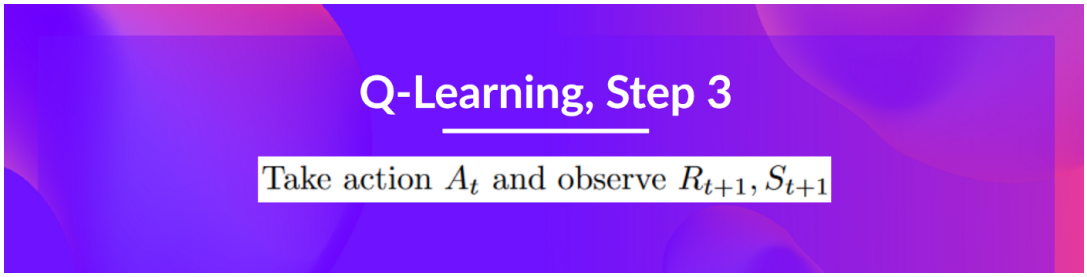


Рис. 54: Q-обучение: обновление ценности действия по TD-цели [оригинал]

Шаг 4. Обновляем $Q(S_t, A_t)$

Напомним, что при TD-обучении мы обновляем политику или функцию ценности (в зависимости от выбранного RL-подхода) **после одного шага взаимодействия со средой.**

Для формирования TD-цели **мы использовали непосредственное вознаграждение R_{t+1} и дисконтированную ценность следующего состояния**, которую вычисляем, находя действие, максимизирующее текущую Q-функцию в следующем состоянии. Это и есть bootstrap-оценивание.

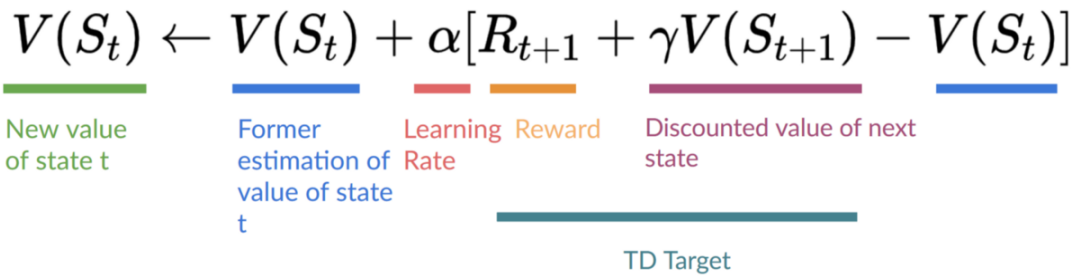


Рис. 55: Q-обучение: повторение обновлений на нескольких эпизодах [оригинал]

Следовательно, **формула обновления $Q(S_t, A_t)$ имеет вид:**

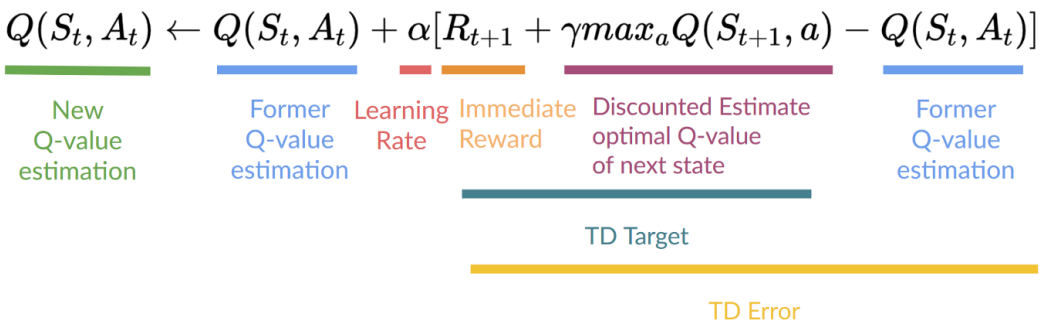


Рис. 56: Q-обучение: общая схема алгоритма [оригинал]

Для обновления $Q(S_t, A_t)$:

- Нам нужны $S_t, A_t, R_{t+1}, S_{t+1}$.
- Для обновления Q-значения пары «состояние–действие» мы используем TD-цель.

Как формируется TD-цель?

1. После выполнения действия A_t мы получаем вознаграждение R_{t+1}.

2. Чтобы получить **наилучшую ценность пары «состояние–действие»** в следующем состоянии, используем жадную политику, выбирающую оптимальное следующее действие. Обратите внимание: это не ε-жадная политика, она всегда выбирает действие с наибольшим Q-значением.

После обновления Q-значения мы переходим в новое состояние и снова выбираем действие **с помощью ε-жадной политики.**

Именно поэтому Q-обучение называют off-policy алгоритмом.

8.3 Off-policy и on-policy: различия

Различие довольно тонкое:

- *Off-policy:* для **выбора действий (взаимодействия со средой) и обновления при обучении используются разные политики.**

Например, в Q-обучении ε-жадная политика выбора действий отличается от жадной политики, **используемой для нахождения наилучшего действия в следующем состоянии при обновлении Q-значения.**

Choose action A_t using policy derived from Q (e.g., ϵ-greedy)

Рис. 57: Off-policy: политика поведения [оригинал]

Она отличается от политики, используемой при обновлении в ходе обучения:

$$\gamma \max_a Q(S_{t+1}, a)$$

Рис. 58: Off-policy: политика обновления [оригинал]

- *On-policy:* **для выбора действий и обновления используется одна и та же политика.**

Например, в алгоритме Sarsa — другом методе на основе ценности — **следующую пару «состояние–действие» выбирает ε-жадная, а не жадная политика.**

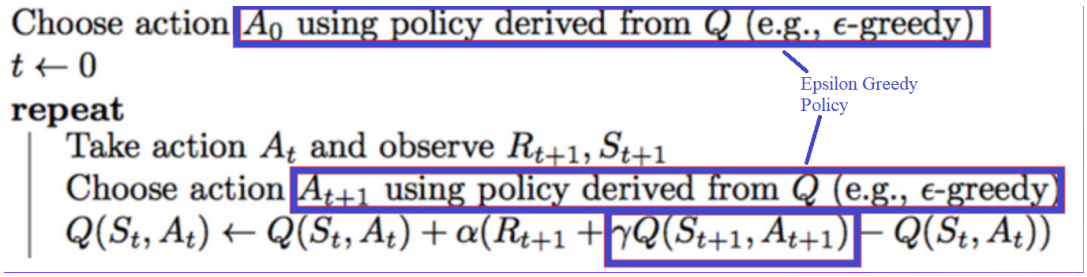


Рис. 59: On-policy: одинаковая политика при взаимодействии и обновлении [оригинал]

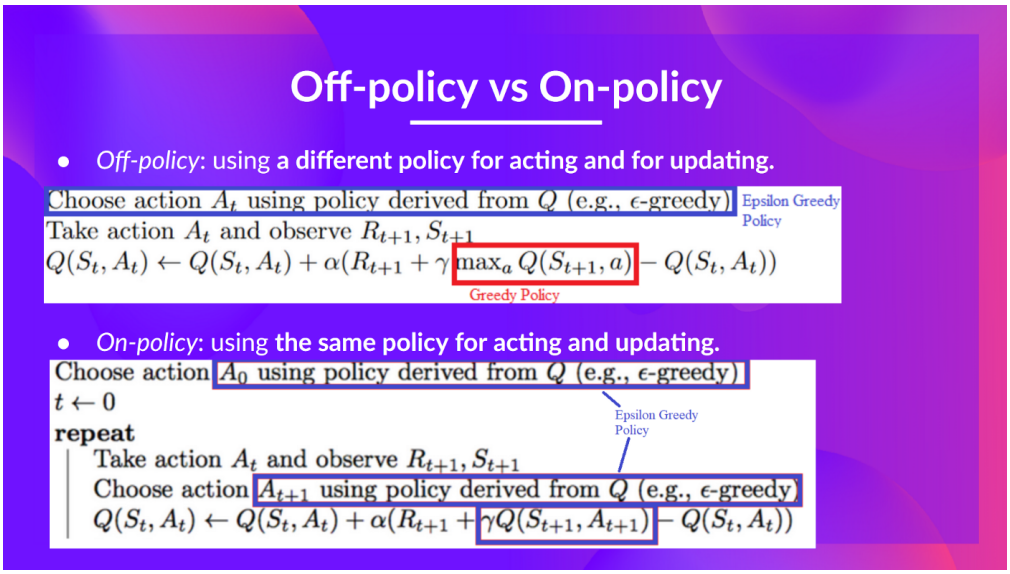


Рис. 60: Сравнение методов off-policy и on-policy [оригинал]

9 Пример Q-обучения

Чтобы лучше понять Q-обучение, рассмотрим простой пример.



Рис. 61: Лабиринт с мышью и сыром: правила вознаграждений и завершения эпизода [оригинал]

- Вы — мышь в небольшом лабиринте. Вы всегда **начинаете из одной и той же точки.**
- Цель — **добраться до большого куска сыра в правом нижнем углу** и избежать яда. Ведь кто не любит сыр?
- Эпизод заканчивается, если мышь съест яд, **большой кусок сыра** или сделает больше пяти шагов.
- Скорость обучения равна 0.1.
- Коэффициент дисконтирования $\gamma = 0.99$.

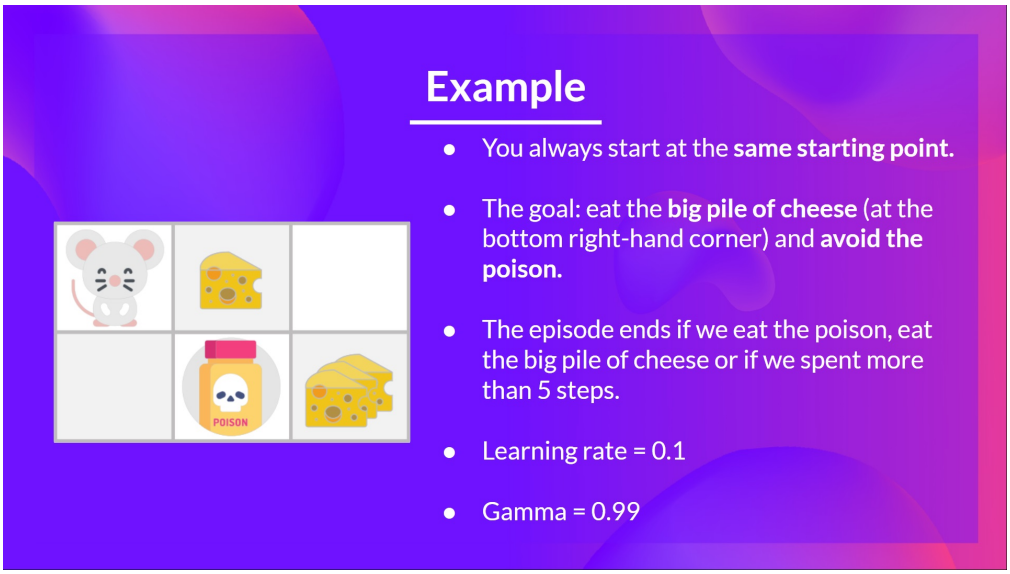


Рис. 62: Пример Q-обучения: переход и обновление Q-таблицы (шаг 1) [оригинал]

Функция вознаграждения устроена так:

- **+0:** переход в состояние, где нет сыра.
- **+1:** переход в состояние с маленьким кусочком сыра.
- **+10:** переход в состояние с большим куском сыра.
- **-10:** переход в состояние с ядом и гибель мыши.
- **+0:** если мышь сделала больше пяти шагов.

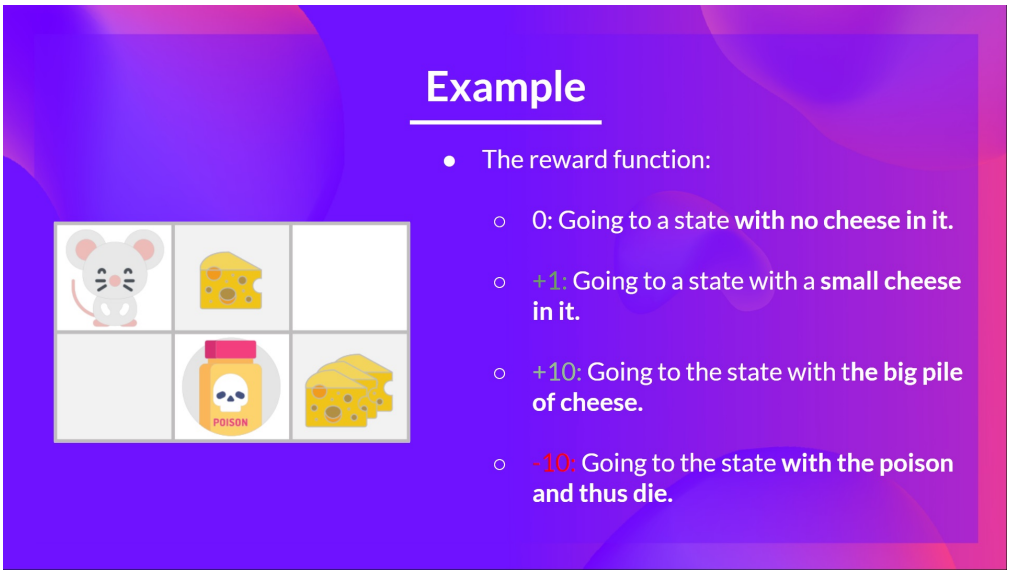


Рис. 63: Пример Q-обучения: переход и обновление Q-таблицы (шаг 2) [оригинал]

Чтобы обучить агента оптимальной политике (то есть действиям вправо, вправо, вниз), **мы применим алгоритм Q-обучения**.

9.1 Шаг 1. Инициализируем Q-таблицу



Рис. 64: Пример лабиринта [оригинал]

Итак, пока **наша Q-таблица бесполезна**. Необходимо **обучить Q-функцию алгоритмом Q-обучения.**

Выполним два шага обучения:

Шаг обучения 1:

9.2 Шаг 2. Выбираем действие с помощью ε-жадной стратегии

Поскольку ε велико ($= 1.0$), я выбираю случайное действие. В данном случае иду вправо.

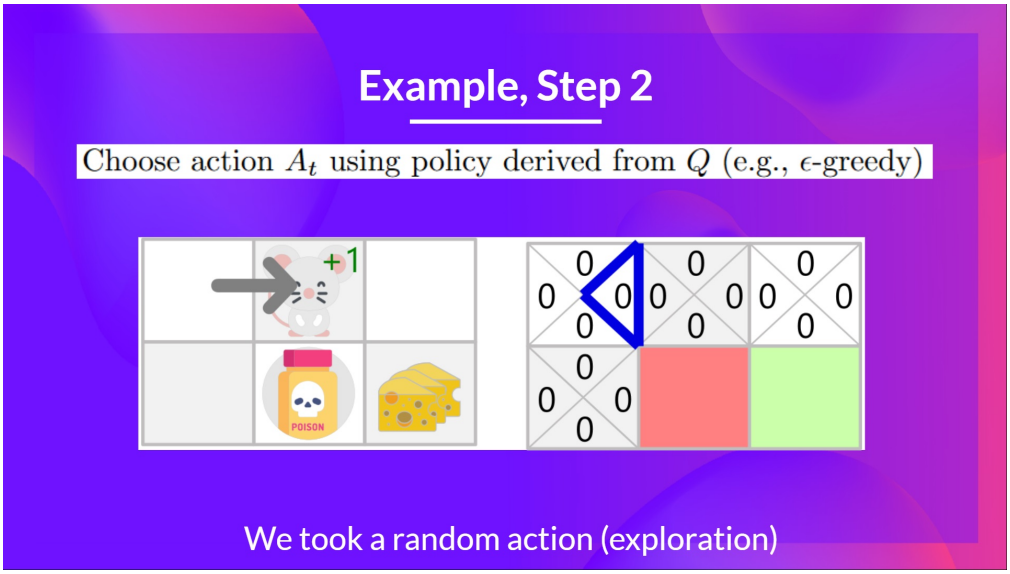


Рис. 65: Пример Q-обучения: переход и обновление Q-таблицы (шаг 3) [оригинал]

9.3 Шаг 3. Выполняем A_t, получаем R_{t+1} и S_{t+1}

Двигаясь вправо, я нахожу маленький кусочек сыра, поэтому $R_{t+1} = 1$, и оказываюсь в новом состоянии.

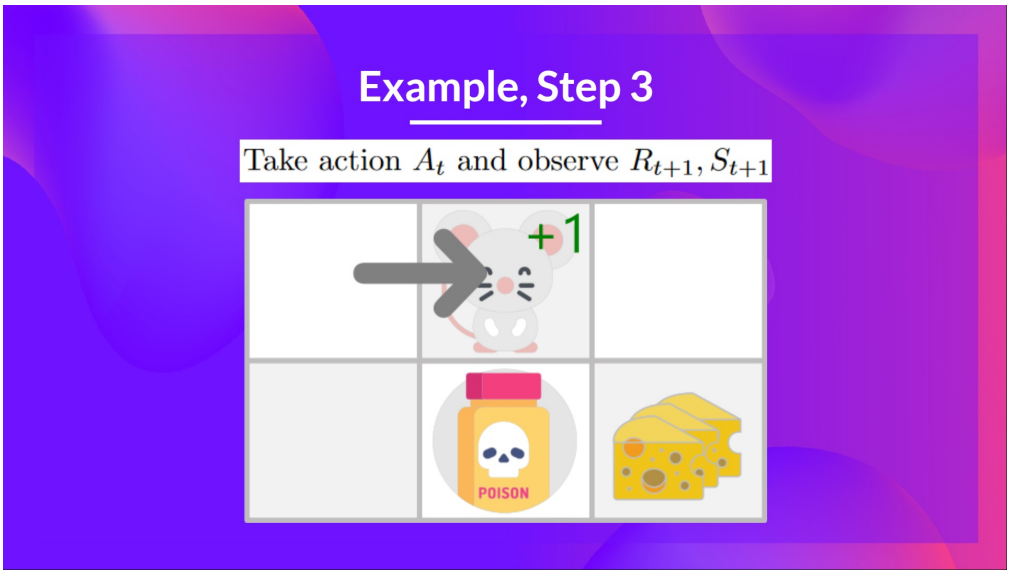


Рис. 66: Пример Q-обучения: переход и обновление Q-таблицы (шаг 4) [оригинал]

9.4 Шаг 4. Обновляем $Q(S_t, A_t)$

Теперь мы можем обновить $Q(S_t, A_t)$ по нашей формуле.

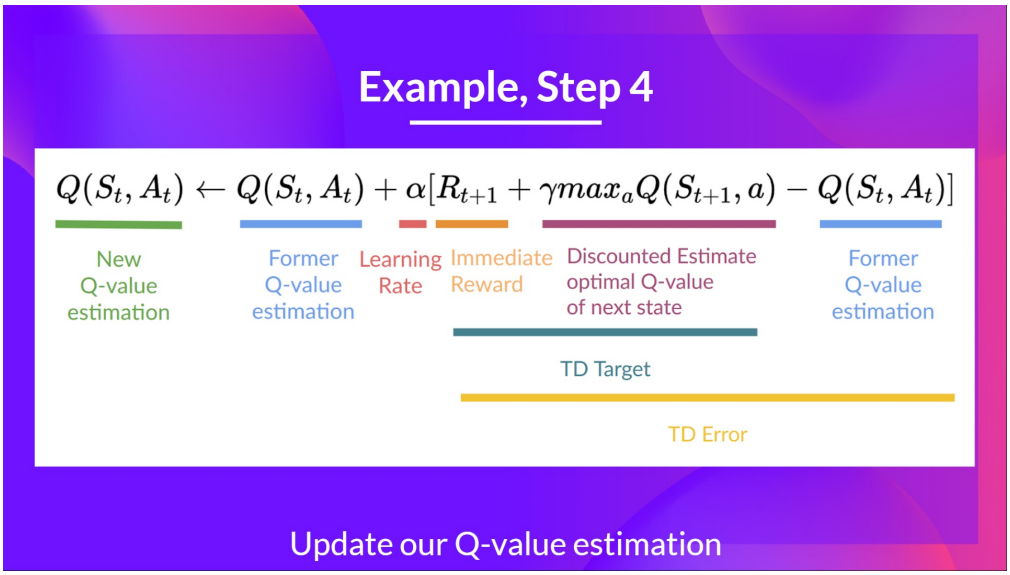


Рис. 67: Пример Q-обучения: переход и обновление Q-таблицы (шаг 5) [оригинал]

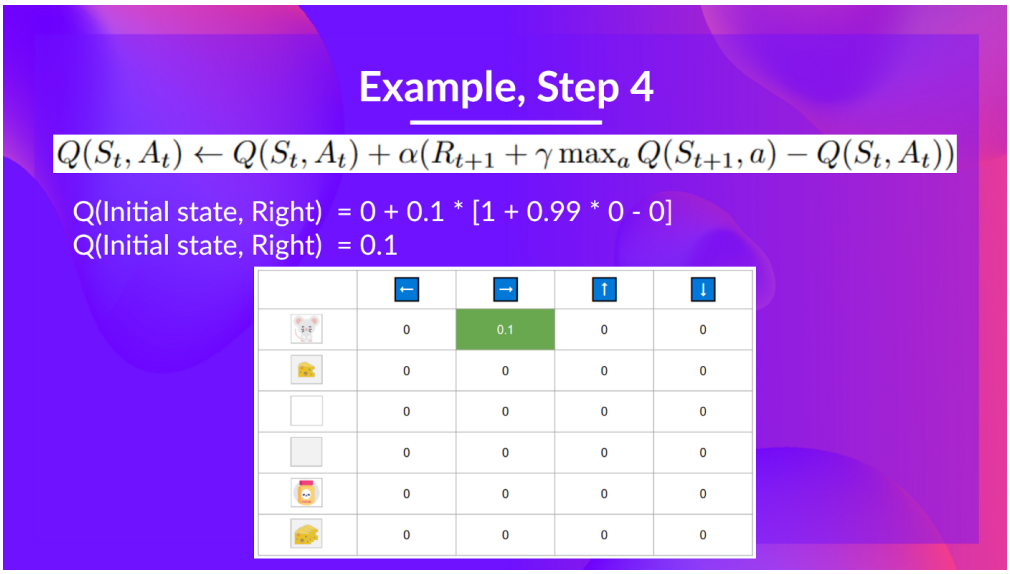


Рис. 68: Пример лабиринта после обновления [оригинал]

Шаг обучения 2:

9.5 Шаг 2. Выбираем действие с помощью ε-жадной стратегии

Я снова выбираю случайное действие, поскольку $\varepsilon = 0.99$ всё ещё велико. (Заметьте, что мы немного уменьшили ε, ведь по мере обучения хотим исследовать среду всё реже.)

Я выбрал действие «вниз». **Это неудачное действие, потому что оно приводит к яду.**

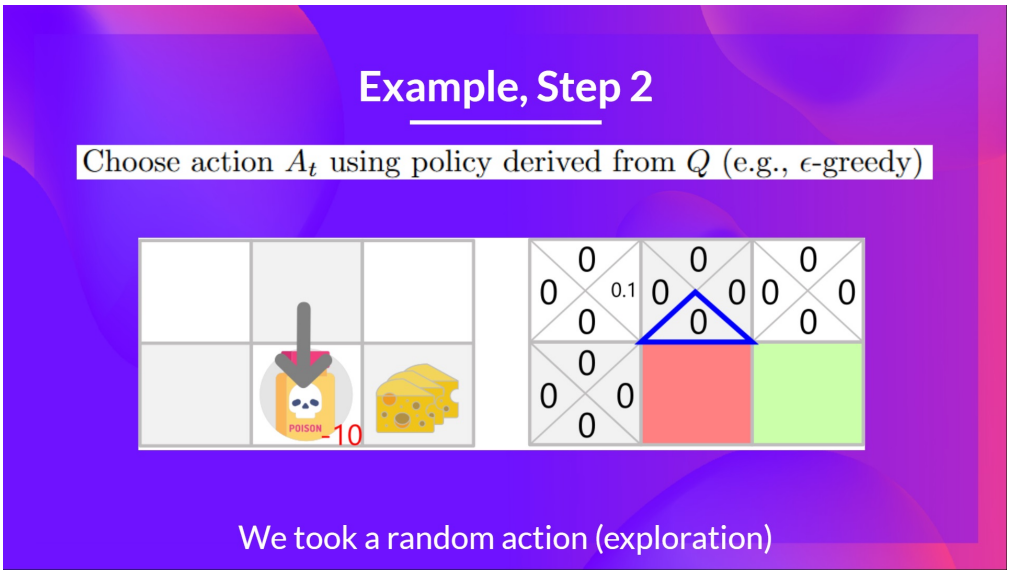


Рис. 69: Пример Q-обучения: переход и обновление Q-таблицы (шаг 6) [оригинал]

9.6 Шаг 3. Выполняем A_t, получаем R_{t+1} и S_{t+1}

Поскольку я съел яд, **получаю $R_{t+1} = -10$ и погибаю.**

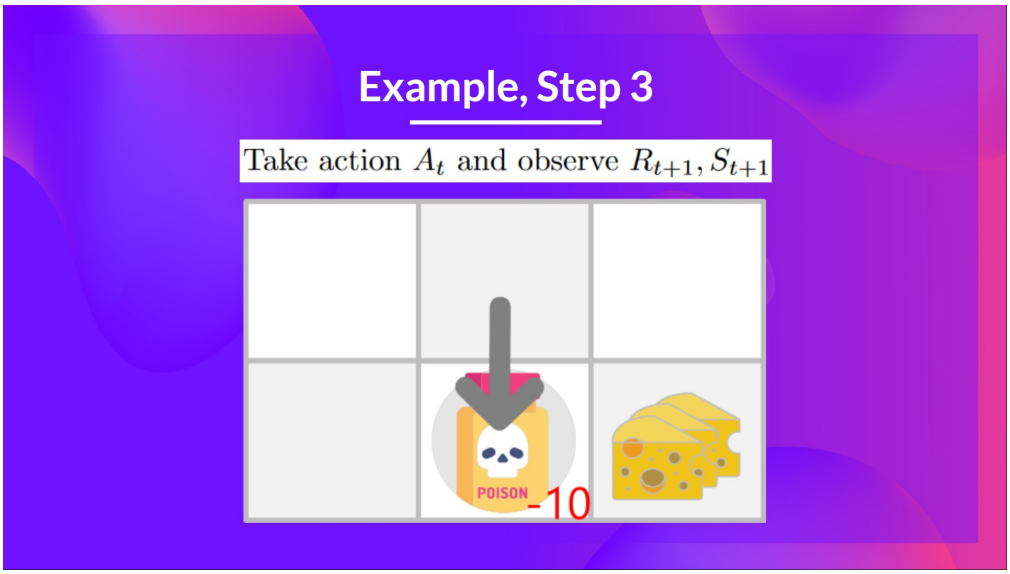


Рис. 70: Пример Q-обучения: переход и обновление Q-таблицы (шаг 7) [оригинал]

9.7 Шаг 4. Обновляем $Q(S_t, A_t)$

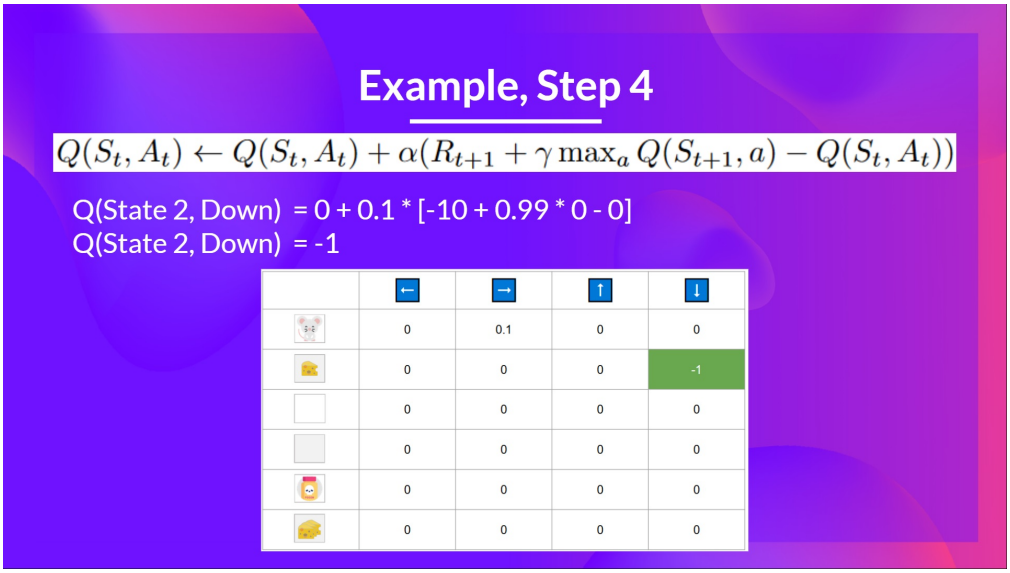


Рис. 71: Пример Q-обучения: переход и обновление Q-таблицы (шаг 8) [оригинал]

После гибели начинается новый эпизод. Однако уже здесь видно, что **всего за два шага исследования мой агент стал умнее.**

Продолжая исследовать среду и использовать накопленные знания, обновляя Q-значения по TD-целям, мы получим **всё более точную Q-таблицу. По окончании обучения она будет приближать оптимальную Q-функцию.**

10 Повторение: Q-обучение

Q-обучение — это **алгоритм RL, который:**

- Обучает *Q-функцию* — **функцию ценности действий**, представленную в памяти *Q-таблицей*, **содержащей ценности всех пар «состояние–действие»**.
- По заданному состоянию и действию **находит в Q-таблице соответствующее значение**.

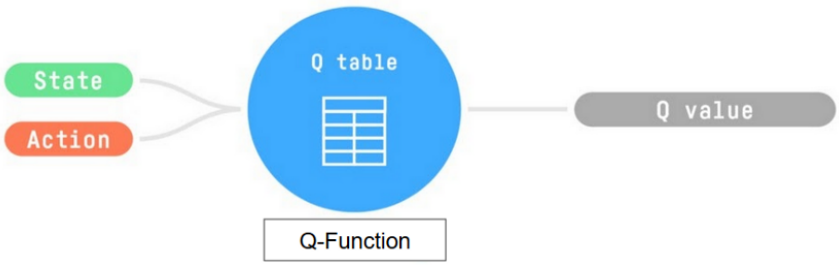


Рис. 72: Поиск ценности пары «состояние–действие» в Q-таблице [оригинал]

- После обучения **получается оптимальная Q-функция или, что эквивалентно, оптимальная Q-таблица.**
- А если **у нас есть оптимальная Q-функция**, то

получается и оптимальная политика: для каждого состояния **нам известно наилучшее действие**.

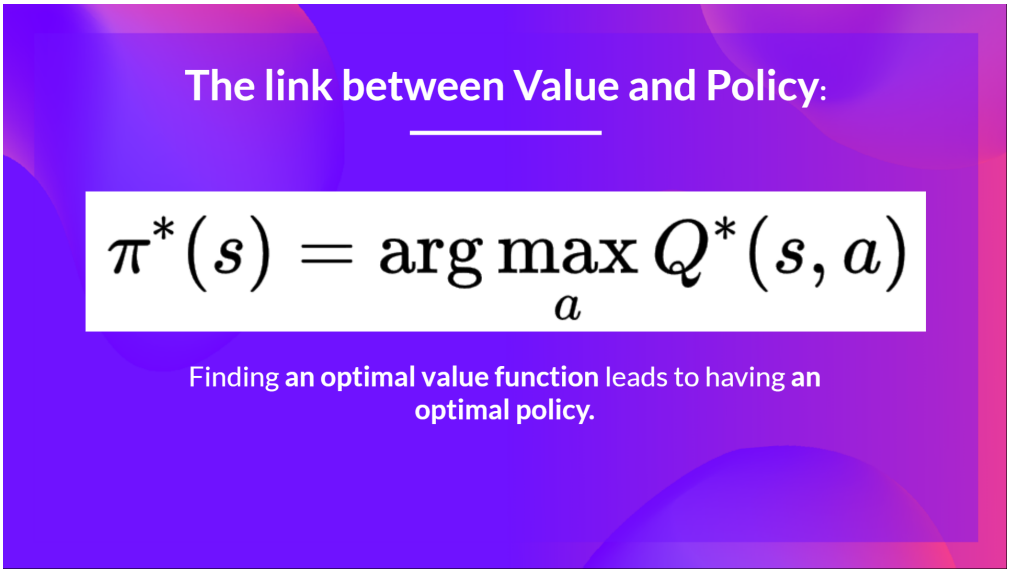


Рис. 73: Восстановление оптимальной политики из оптимальной функции ценности действий [оригинал]

Но в начале **Q-таблица бесполезна, поскольку содержит произвольные значения для каждой пары «состояние–действие» (обычно Q-таблица инициализируется нулями)**. По мере исследования среды и обновления таблицы она даёт всё более точные оценки.

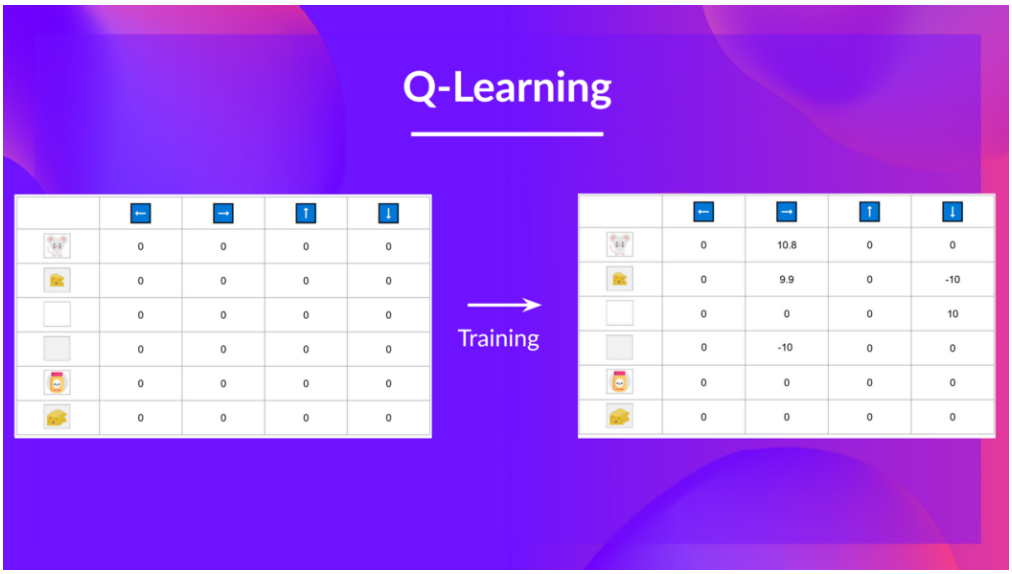


Рис. 74: Обучение Q-таблицы: от нулевой инициализации к полезной политике [оригинал]

Вот псевдокод Q-обучения:

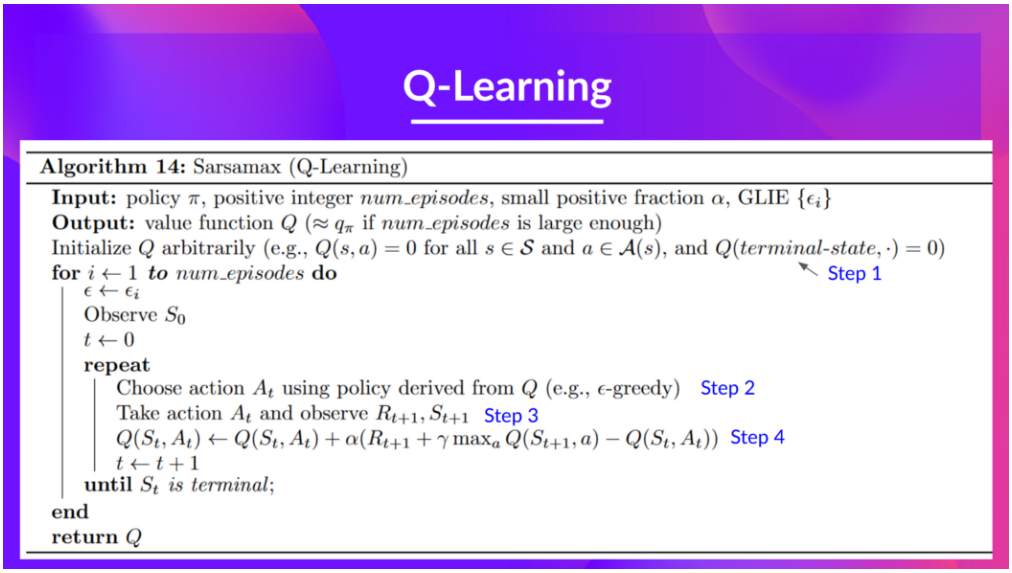


Рис. 75: Алгоритм Q-обучения и псевдокод обновления [оригинал]

11 Практическая часть

Официальный практический ноутбук: https://colab.research.google.com/github/huggingface/deep-rl-class/blob/main/notebooks/unit2/unit2.ipynb

Помощь по курсу: http://hf.co/join/discord

Теперь, когда мы изучили алгоритм Q-обучения, реализуем его с нуля и обучим Q-агента в двух средах:

1. FrozenLake-v1 (без скольжения и со скольжением): агенту нужно **добраться от начального состояния (S) до цели (G)**, перемещаясь только по замёрзшим клеткам (F) и избегая прорубей (H).
2. Автономное такси должно **научиться ориентироваться** в городе, чтобы **перевозить пассажиров из пункта A в пункт B.**

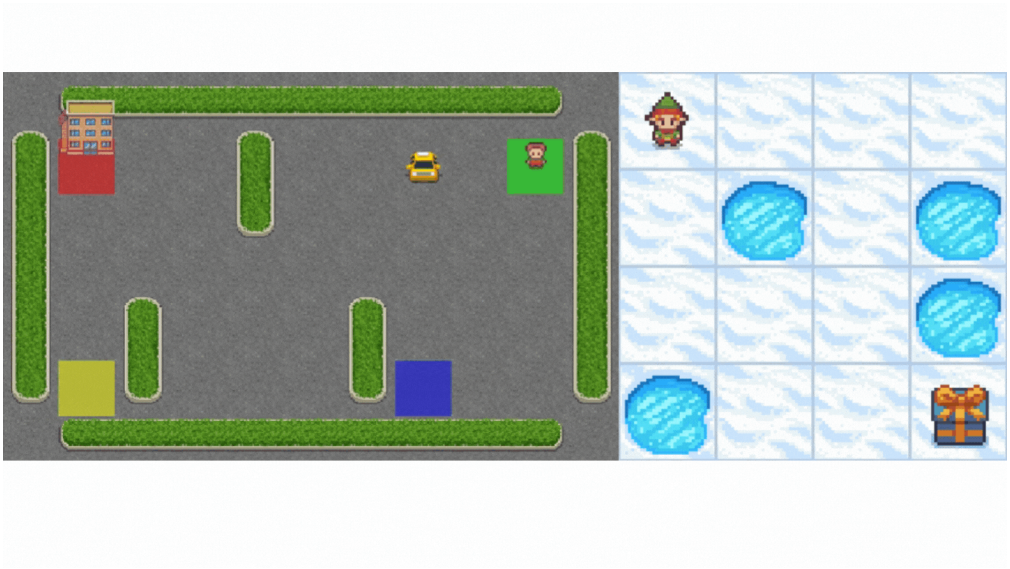


Рис. 76: Демонстрация сред FrozenLake и Taxi [оригинал]

Благодаря таблице лидеров можно сравнивать свои результаты с результатами других участников и делиться лучшими приёмами для улучшения качества агентов. Кто победит в испытании второго модуля?

Чтобы зачесть практическую часть в рамках сертификации, нужно загрузить обученную модель Taxi в Hub и **получить результат не ниже 4.5**.

Чтобы узнать результат, откройте таблицу лидеров и найдите свою модель; **результат = mean_reward - стандартное отклонение вознаграждения**.

Подробнее о сертификации читайте в разделе https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process.

Проверить прогресс можно здесь: https : / / huggingface . co / spaces / ThomasSimonini / Check-my-progress-Deep-RL-Course.

Чтобы начать практику, нажмите кнопку Open In Colab:

Открыть исходный ноутбук в Google Colab

Мы настоятельно **рекомендуем выполнять практические упражнения в Google Colab**, а не запускать их на личных компьютерах.

Google Colab позволяет **сосредоточиться на изучении и экспериментах, не отвлекаясь на техническую настройку** окружения.

12 Модуль 2: Q-обучение для FrozenLake-v1 и Taxi-v3

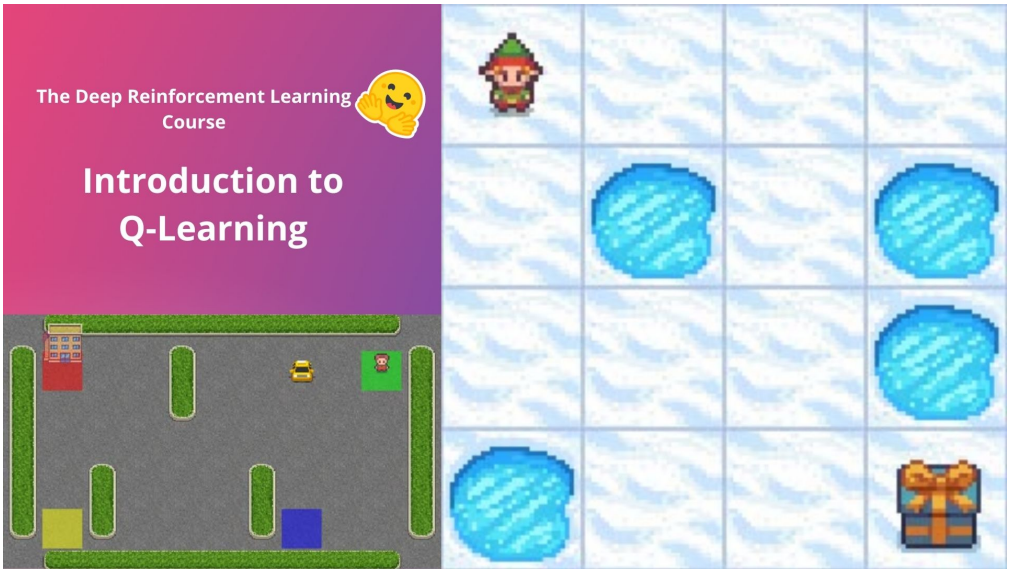


Рис. 77: Обложка модуля 2 [оригинал]

В этом ноутбуке **вы с нуля напишете своего первого RL-агента**, который обучается играть в FrozenLake с помощью Q-обучения, поделитесь результатом с сообществом и поэкспериментируете с настройками.

Вот пример того, **чего можно добиться буквально за несколько минут.**



Рис. 78: Демонстрация сред FrozenLake и Taxi [оригинал]

Среды:

- FrozenLake-v1
- Taxi-v3

Библиотеки для RL:

- Python и NumPy
- Gymnasium

Мы постоянно стараемся улучшать учебные материалы, поэтому **если вы найдёте ошибку в этом ноутбуке**, пожалуйста, создайте issue в GitHub-репозитории.

12.1 Цели этого ноутбука

К концу работы с ноутбуком вы сможете:

- Использовать библиотеку сред **Gymnasium**.
- Самостоятельно реализовать агента на основе Q-обучения.
- **Загрузить обученного агента и его код в Hub**, добавив красивую видеозапись работы агента и оценку его качества.

12.2 Этот ноутбук относится к курсу Deep Reinforcement Learning



Рис. 79: Иллюстрация курса Deep RL [оригинал]

В этом бесплатном курсе вы:

- Изучите глубокое обучение с подкреплением **теоретически и практически**.
- Научитесь **использовать известные библиотеки Deep RL**, например Stable-Baselines3, RL Baselines3 Zoo, CleanRL и Sample Factory 2.0.
- Обучите **агентов в разнообразных средах**.

И многое другое: смотрите https://simoninithomas.github.io/deep-rl-course.

Не забудьте **зарегистрироваться на курс** (мы собираем электронные адреса, чтобы **отправлять вам ссылки при публикации новых модулей, а также новости об испытаниях и обновлениях**).

Лучший способ оставаться на связи — присоединиться к нашему Discord-серверу, где можно общаться с авторами и сообществом: https://discord.gg/ydHrjt3WP5.

12.3 Предварительные требования

Перед началом работы с ноутбуком необходимо:

Изучить Q-обучение, прочитав теорию модуля 2.

12.4 Краткое повторение Q-обучения

Q-обучение — это **алгоритм RL, который:**

- Обучает *Q-функцию* — **функцию ценности действий**, которая хранится в памяти в виде *Q-таблицы*, **содержащей значения всех пар «состояние–действие»**.
- По состоянию и действию **находит в Q-таблице соответствующее значение**.

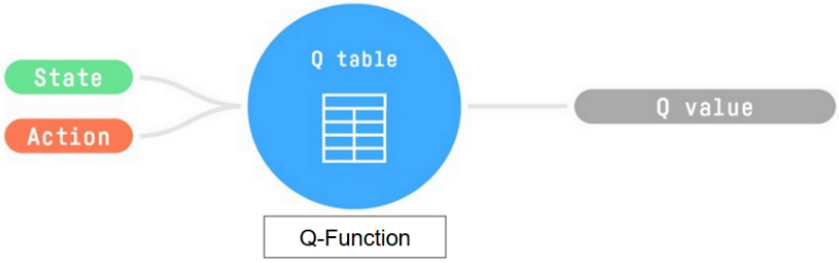


Рис. 80: Поиск ценности пары «состояние–действие» в Q-таблице [оригинал]

- После обучения **мы получаем оптимальную Q-функцию, а значит, оптимальную Q-таблицу**.
- Если **Q-функция оптимальна**, то

оптимальна и политика, поскольку **мы знаем наилучшее действие для каждого состояния**.

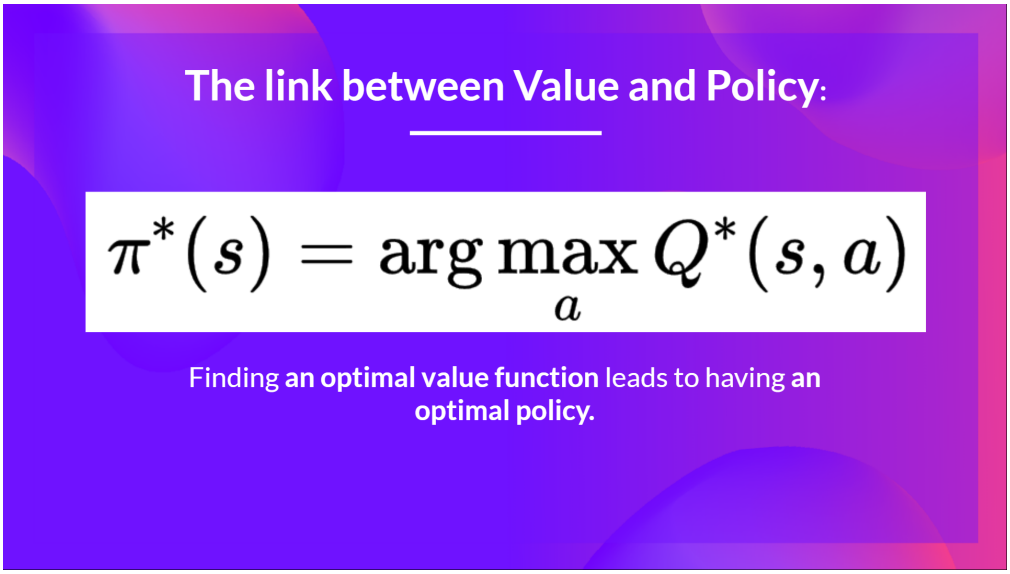


Рис. 81: Восстановление оптимальной политики из оптимальной функции ценности действий [оригинал]

В начале **Q-таблица бесполезна, поскольку содержит произвольные значения для всех пар «состояние–действие» (обычно её инициализируют нулями)**. Но по мере исследования среды и обновления Q-таблицы оценки становятся всё точнее.

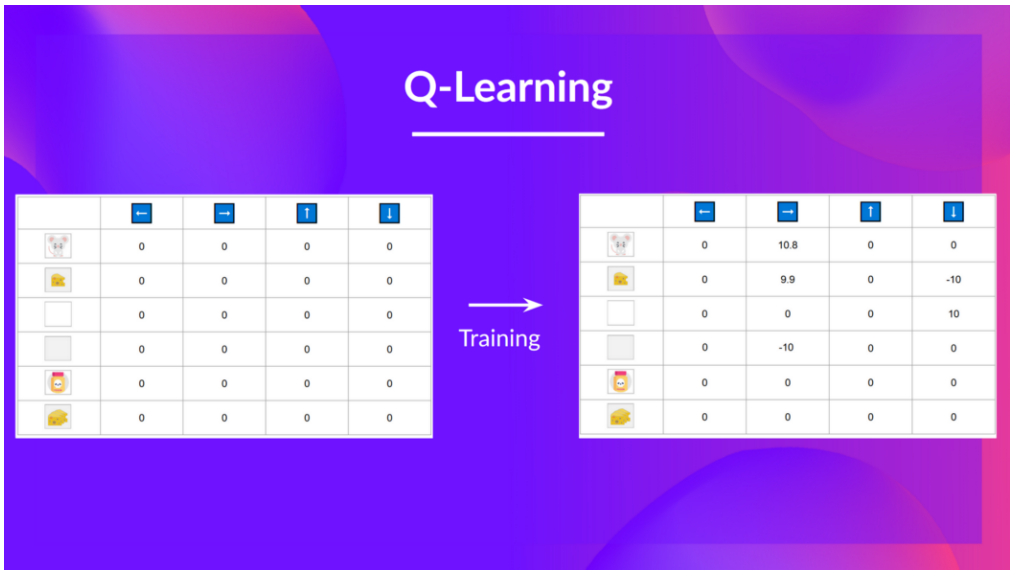


Рис. 82: Обучение Q-таблицы: от нулевой инициализации к полезной политике [оригинал]

Вот псевдокод Q-обучения:

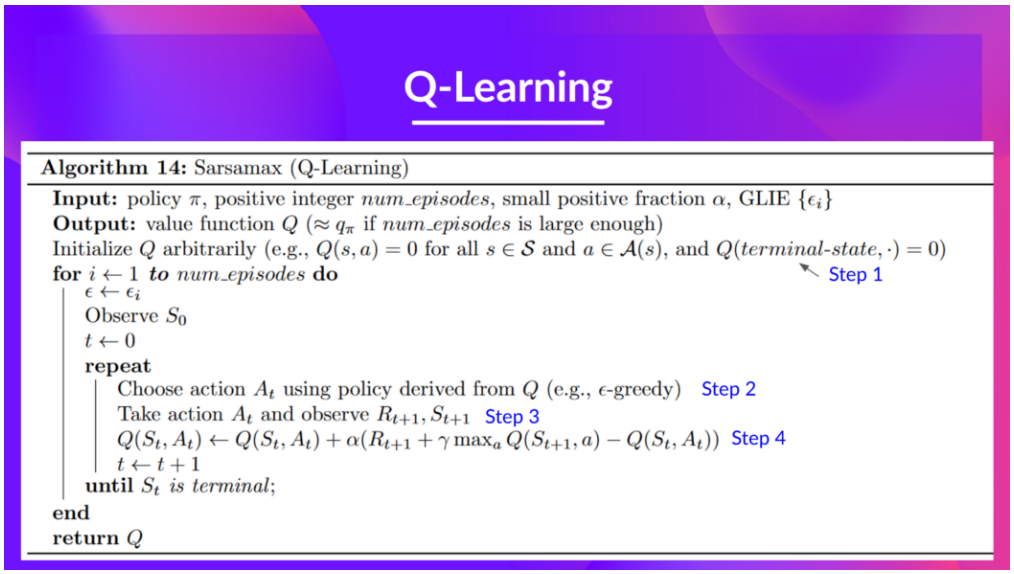


Рис. 83: Алгоритм Q-обучения и псевдокод обновления [оригинал]

13 Давайте напишем наш первый алгоритм обучения с подкреплением

Для зачёта этой практики в рамках сертификации нужно загрузить обученную модель Taxi в Hub и **получить результат не ниже 4.5**.

Чтобы узнать результат, откройте таблицу лидеров и найдите свою модель; **результат = mean_reward - стандартное отклонение вознаграждения**.

Подробнее о сертификации читайте здесь: https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process.

13.1 Установка зависимостей и создание виртуального экрана

В ноутбуке нам потребуется создать видеозапись работы агента. Для этого в Colab **необходимо создать виртуальный экран для отрисовки среды** (и, соответственно, записи кадров).

Следующая ячейка устанавливает библиотеки, создаёт и запускает виртуальный экран.

Мы установим несколько пакетов:

- `gymnasium`: содержит среды FrozenLake-v1 и Taxi-v3.
- `pygame`: используется для интерфейса FrozenLake-v1 и Taxi-v3.
- `numpy`: используется для работы с Q-таблицей.

Hugging Face Hub — это централизованная платформа, на которой любой пользователь может публиковать и изучать модели и датасеты. Поддерживаются версионирование, метрики, визуализации и другие средства для совместной работы.

Здесь можно посмотреть все доступные модели Deep RL, использующие Q-Learning: https://huggingface.co/models?other=q-learning.

```
pip install -r https://raw.githubusercontent.com/huggingface/deep-rl-class/main/notebooks/unit2/
    requirements-unit2.txt
```

```
sudo apt-get update
sudo apt-get install -y python3-opengl
apt install ffmpeg xvfb
pip3 install pyvirtualdisplay
```

Чтобы гарантировать использование вновь установленных библиотек, **иногда требуется перезапустить среду выполнения ноутбука**. Следующая ячейка намеренно **вызовет аварийное завершение среды; после этого нужно подключиться заново и продолжить выполнение с этого места**. Благодаря этому **мы сможем использовать виртуальный экран**.

```
import os

os.kill(os.getpid(), 9)
```

```
# Virtual display
from pyvirtualdisplay import Display

virtual_display = Display(visible=0, size=(1400, 900))
virtual_display.start()
```

13.2 Импорт библиотек

Помимо установленных библиотек, используем:

- `random`: для генерации случайных чисел (нужны для ε-жадной политики).
- `imageio`: для создания видеозаписи работы агента.

```
import numpy as np
import gymnasium as gym
import random
import imageio
import os
import tqdm

import pickle5 as pickle
from tqdm.notebook import tqdm
```

Теперь всё готово к написанию алгоритма Q-обучения.

14 Часть 1. FrozenLake без скольжения

14.1 Создание и изучение среды FrozenLake

При начале работы с новой средой полезно сначала изучить её документацию.

https://gymnasium.farama.org/environments/toy_text/frozen_lake/

Мы обучим Q-агента **переходить из начального состояния (S) в целевое (G), передвигаясь по замёрзшим клеткам (F) и избегая прорубей (H)**.

Доступно два размера среды:

- `map_name="4x4"`: сетка 4 на 4.
- `map_name="8x8"`: сетка 8 на 8.

У среды есть два режима:

- `is_slippery=False`: агент всегда движется **в выбранном направлении**, поскольку лёд не скользкий (детерминированный режим).
- `is_slippery=True`: из-за скольжения агент **не всегда перемещается в заданном направлении** (стохастический режим).

Пока выберем самый простой вариант: карту 4 на 4 без скольжения. Дополнительно зададим параметр `render_mode`, определяющий способ визуализации среды. Поскольку **в конце нам нужно записать видео, следует указать render_mode = rgb_array**.

Как сказано в документации, режим “rgb_array” возвращает один кадр текущего состояния среды. Кадр представлен массивом np.ndarray формы (x, y, 3) с RGB-значениями пикселей изображения размером x на y.

```
# Create the FrozenLake-v1 environment using 4x4 map and non-slippery version and render_mode="
    rgb_array"
env = gym.make()  # TODO use the correct parameters
```

Решение

```
env = gym.make("FrozenLake-v1", map_name="4x4", is_slippery=False, render_mode="rgb_array")
```

Можно создать собственную сетку следующим образом:

```
desc=["SFFF", "FHFH", "FFFH", "HFFG"]
gym.make('FrozenLake-v1', desc=desc, is_slippery=True)
```

Но пока будем использовать среду по умолчанию.

Посмотрим, как выглядит среда:

```
# We create our environment with gym.make("<name_of_the_environment>")- `is_slippery=False`: The
    agent always moves in the intended direction due to the non-slippery nature of the frozen lake
    (deterministic).
print("_____OBSERVATION SPACE_____ \n")
print("Observation Space", env.observation_space)
print("Sample observation", env.observation_space.sample())  # Get a random observation
```

По выводу `Observation Space Shape Discrete(16)` видно, что наблюдение — целое число, представляющее **текущую позицию агента как current_row * ncols + current_col (индексы строк и столбцов начинаются с нуля)**.

Например, позиция цели на карте 4 на 4 рассчитывается как $3 * 4 + 3 = 15$. Число возможных наблюдений зависит от размера карты. **Для сетки 4 на 4 существует 16 возможных наблюдений.**

Вот как выглядит состояние 0:



Рис. 84: Сетка FrozenLake, действия и типы клеток [оригинал]

```
print("\n _____ACTION SPACE_____ \n")
print("Action Space Shape", env.action_space.n)
print("Action Space Sample", env.action_space.sample())  # Take a random action
```

Пространство действий (набор доступных действий) дискретно и содержит четыре варианта:

- 0: ДВИЖЕНИЕ ВЛЕВО
- 1: ДВИЖЕНИЕ ВНИЗ
- 2: ДВИЖЕНИЕ ВПРАВО
- 3: GO UP

Функция вознаграждения:

- Достижение цели: +1.
- Попадание в прорубь: 0.
- Попадание на замёрзшую клетку: 0.

14.2 Создание и инициализация Q-таблицы

(Шаг 1 псевдокода.)

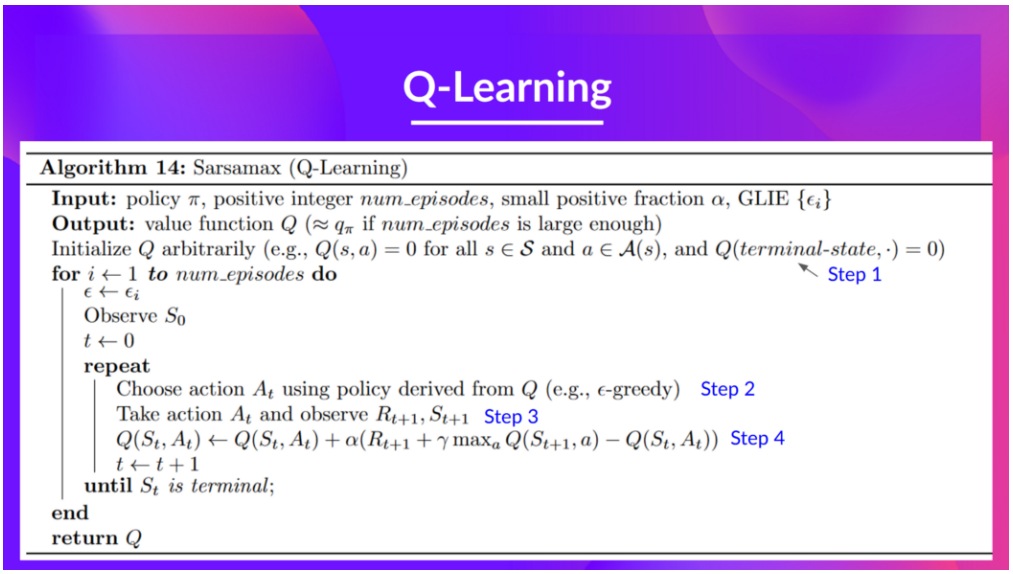


Рис. 85: Алгоритм Q-обучения и псевдокод обновления [оригинал]

Пора инициализировать Q-таблицу! Чтобы определить число строк (состояний) и столбцов (действий), нужно знать пространства наблюдений и действий. Их размеры мы уже выяснили, но лучше получать их программно, чтобы алгоритм работал с разными средами. В Gym для этого существуют `env.action_space.n` и `env.observation_space.n`.

```
state_space =
print("There are ", state_space, " possible states")

action_space =
print("There are ", action_space, " possible actions")
```

```
# Let's create our Qtable of size (state_space, action_space) and initialized each values at 0
    using np.zeros. np.zeros needs a tuple (a,b)
def initialize_q_table(state_space, action_space):
  Qtable =
  return Qtable
```

```
Qtable_frozenlake = initialize_q_table(state_space, action_space)
```

Решение

```
state_space = env.observation_space.n
print("There are ", state_space, " possible states")

action_space = env.action_space.n
print("There are ", action_space, " possible actions")
```

```
# Let's create our Qtable of size (state_space, action_space) and initialized each values at 0
    using np.zeros
def initialize_q_table(state_space, action_space):
    Qtable = np.zeros((state_space, action_space))
    return Qtable
```

```
Qtable_frozenlake = initialize_q_table(state_space, action_space)
```

14.3 Определение жадной политики

Напомним, что Q-обучение — **off-policy** алгоритм, поэтому мы используем **разные политики для выбора действий и обновления функции ценности**.

- ε-жадная политика (политика поведения).
- Жадная политика (политика обновления).

Жадная политика также станет итоговой политикой после завершения обучения Q-агента. Она выбирает действие на основе Q-таблицы.

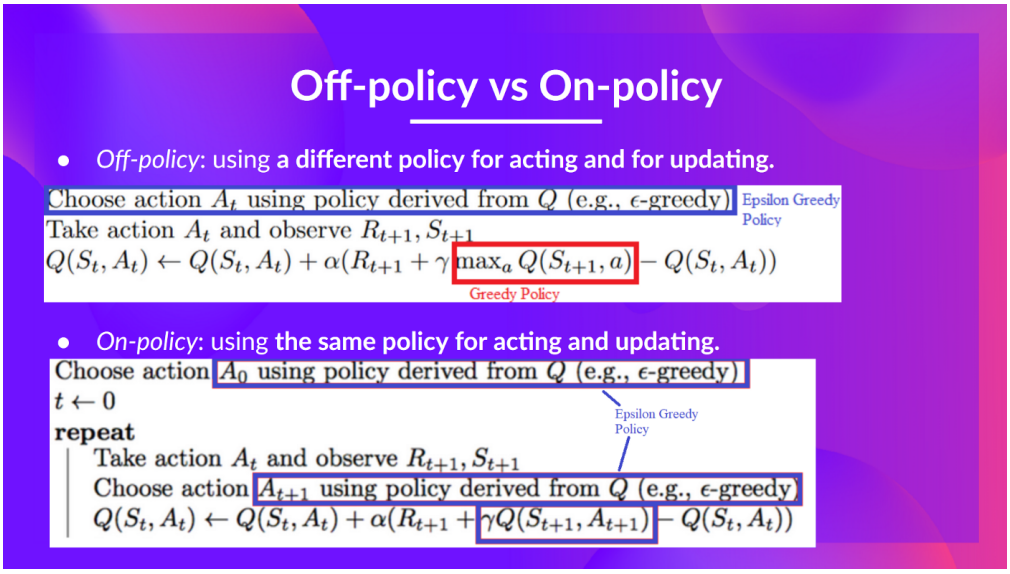


Рис. 86: Сравнение методов off-policy и on-policy [оригинал]

```
def greedy_policy(Qtable, state):
  # Exploitation: take the action with the highest state, action value
  action =

  return action
```

Решение

```
def greedy_policy(Qtable, state):
    # Exploitation: take the action with the highest state, action value
    action = np.argmax(Qtable[state][:])

    return action
```

14.4 Определение ε-жадной политики

ε-жадная политика применяется при обучении и обеспечивает компромисс между исследованием среды и использованием уже известных знаний.

Идея ε-жадной стратегии:

- С *вероятностью* $1 - \varepsilon$: **используем знания (exploitation)** — агент выбирает действие с максимальной ценностью пары «состояние–действие».
- С *вероятностью* ε: **исследуем среду (exploration)** — выбираем случайное действие.

По мере обучения мы постепенно **уменьшаем ε, поскольку исследования среды требуется всё меньше, а использования уже полученных знаний — всё больше.**

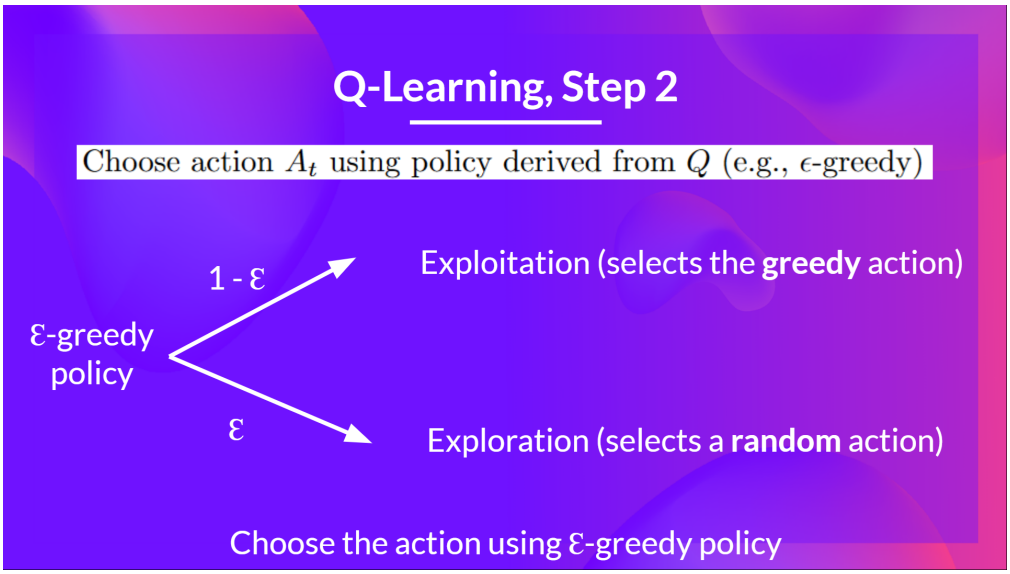


Рис. 87: ε-жадная стратегия: исследование и использование знаний [оригинал]

```
def epsilon_greedy_policy(Qtable, state, epsilon):
  # Randomly generate a number between 0 and 1
  random_num =
  # if random_num > greater than epsilon --> exploitation
  if random_num > epsilon:
    # Take the action with the highest value given a state
    # np.argmax can be useful here
    action =
  # else --> exploration
  else:
    action = # Take a random action

  return action
```

Решение

```
def epsilon_greedy_policy(Qtable, state, epsilon):
    # Randomly generate a number between 0 and 1
    random_num = random.uniform(0, 1)
    # if random_num > greater than epsilon --> exploitation
    if random_num > epsilon:
        # Take the action with the highest value given a state
        # np.argmax can be useful here
        action = greedy_policy(Qtable, state)
    # else --> exploration
    else:
        action = env.action_space.sample()

    return action
```

14.5 Определение гиперпараметров

Гиперпараметры, связанные с исследованием среды, относятся к наиболее важным.

- Нужно удостовериться, что агент **достаточно хорошо исследует пространство состояний**, чтобы получить качественную оценку ценности. Для этого ε следует уменьшать постепенно.
- Если уменьшать ε слишком быстро (слишком большой decay_rate), **возникает риск, что обучение агента зайдёт в тупик**: он не исследует достаточную часть пространства состояний и потому не сможет решить задачу.

```
# Training parameters
n_training_episodes = 10000  # Total training episodes
learning_rate = 0.7  # Learning rate

# Evaluation parameters
n_eval_episodes = 100  # Total number of test episodes

# Environment parameters
env_id = "FrozenLake-v1"  # Name of the environment
max_steps = 99  # Max steps per episode
gamma = 0.95  # Discounting rate
eval_seed = []  # The evaluation seed of the environment

# Exploration parameters
max_epsilon = 1.0  # Exploration probability at start
min_epsilon = 0.05  # Minimum exploration probability
decay_rate = 0.0005  # Exponential decay rate for exploration prob
```

14.6 Создание цикла обучения

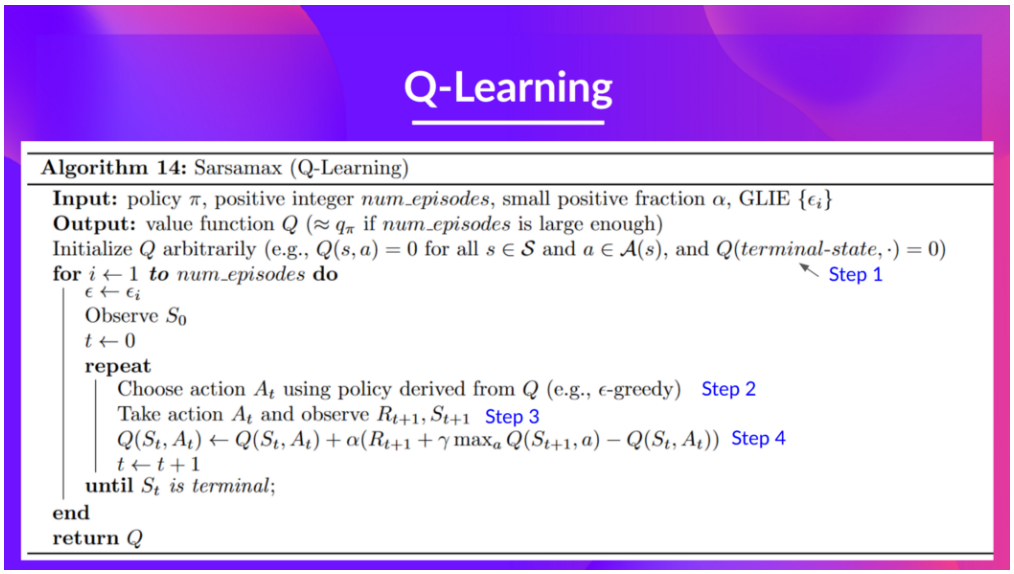


Рис. 88: Алгоритм Q-обучения и псевдокод обновления [оригинал]

Цикл обучения устроен следующим образом:

```
For episode in the total of training episodes:

Reduce epsilon (since we need less and less exploration)
Reset the environment

  For step in max timesteps:
    Choose the action At using epsilon greedy policy
    Take the action (a) and observe the outcome state(s') and reward (r)
    Update the Q-value Q(s,a) using Bellman equation Q(s,a) + lr [R(s,a) + gamma * max Q(s',a') - Q
    (s,a)]
    If done, finish the episode
    Our next state is the new state
```

```
def train(n_training_episodes, min_epsilon, max_epsilon, decay_rate, env, max_steps, Qtable):
  for episode in tqdm(range(n_training_episodes)):
    # Reduce epsilon (because we need less and less exploration)
    epsilon = min_epsilon + (max_epsilon - min_epsilon)*np.exp(-decay_rate*episode)
```

```
    # Reset the environment
    state, info = env.reset()
    step = 0
    terminated = False
    truncated = False

    # repeat
    for step in range(max_steps):
      # Choose the action At using epsilon greedy policy
      action =

      # Take action At and observe Rt+1 and St+1
      # Take the action (a) and observe the outcome state(s') and reward (r)
      new_state, reward, terminated, truncated, info =

      # Update Q(s,a):= Q(s,a) + lr [R(s,a) + gamma * max Q(s',a') - Q(s,a)]
      Qtable[state][action] =

      # If terminated or truncated finish the episode
      if terminated or truncated:
        break

      # Our next state is the new state
      state = new_state
  return Qtable
```

Решение

```
def train(n_training_episodes, min_epsilon, max_epsilon, decay_rate, env, max_steps, Qtable):
    for episode in tqdm(range(n_training_episodes)):
        # Reduce epsilon (because we need less and less exploration)
        epsilon = min_epsilon + (max_epsilon - min_epsilon) * np.exp(-decay_rate * episode)
        # Reset the environment
        state, info = env.reset()
        step = 0
        terminated = False
        truncated = False

        # repeat
        for step in range(max_steps):
            # Choose the action At using epsilon greedy policy
            action = epsilon_greedy_policy(Qtable, state, epsilon)

            # Take action At and observe Rt+1 and St+1
            # Take the action (a) and observe the outcome state(s') and reward (r)
            new_state, reward, terminated, truncated, info = env.step(action)

            # Update Q(s,a):= Q(s,a) + lr [R(s,a) + gamma * max Q(s',a') - Q(s,a)]
            Qtable[state][action] = Qtable[state][action] + learning_rate * (
                reward + gamma * np.max(Qtable[new_state]) - Qtable[state][action]
            )

            # If terminated or truncated finish the episode
            if terminated or truncated:
                break

            # Our next state is the new state
            state = new_state
    return Qtable
```

14.7 Обучение Q-агента

```
Qtable_frozenlake = train(n_training_episodes, min_epsilon, max_epsilon, decay_rate, env, max_steps,
     Qtable_frozenlake)
```

14.8 Посмотрим на получившуюся Q-таблицу

```
Qtable_frozenlake
```

14.9 Метод оценки качества

- Мы определили метод оценки, который будем использовать для проверки нашего Q-агента.

```
def evaluate_agent(env, max_steps, n_eval_episodes, Q, seed):
    """
    Evaluate the agent for ``n_eval_episodes`` episodes and returns average reward and std of
    reward.
    :param env: The evaluation environment
    :param n_eval_episodes: Number of episode to evaluate the agent
    :param Q: The Q-table
    :param seed: The evaluation seed array (for taxi-v3)
    """
    episode_rewards = []
    for episode in tqdm(range(n_eval_episodes)):
        if seed:
            state, info = env.reset(seed=seed[episode])
        else:
            state, info = env.reset()
        step = 0
        truncated = False
        terminated = False
        total_rewards_ep = 0

        for step in range(max_steps):
            # Take the action (index) that have the maximum expected future reward given that state
            action = greedy_policy(Q, state)
            new_state, reward, terminated, truncated, info = env.step(action)
            total_rewards_ep += reward

            if terminated or truncated:
                break
            state = new_state
        episode_rewards.append(total_rewards_ep)
    mean_reward = np.mean(episode_rewards)
    std_reward = np.std(episode_rewards)

    return mean_reward, std_reward
```

14.10 Оценка качества Q-агента

- Обычно среднее вознаграждение должно быть равно 1.0.
- **Эта среда относительно простая**, потому что пространство состояний очень маленькое (16). В качестве усложнения попробуйте версию со скольжением: она вносит стохастичность, делая среду сложнее.

```
# Evaluate our Agent
mean_reward, std_reward = evaluate_agent(env, max_steps, n_eval_episodes, Qtable_frozenlake,
    eval_seed)
print(f"Mean_reward={mean_reward:.2f} +/- {std_reward:.2f}")
```

14.11 Публикация обученной модели в Hub

Теперь, когда обучение дало хорошие результаты, **мы можем опубликовать обученную модель в Hub одной строкой кода**.

Вот пример карточки модели (Model Card):

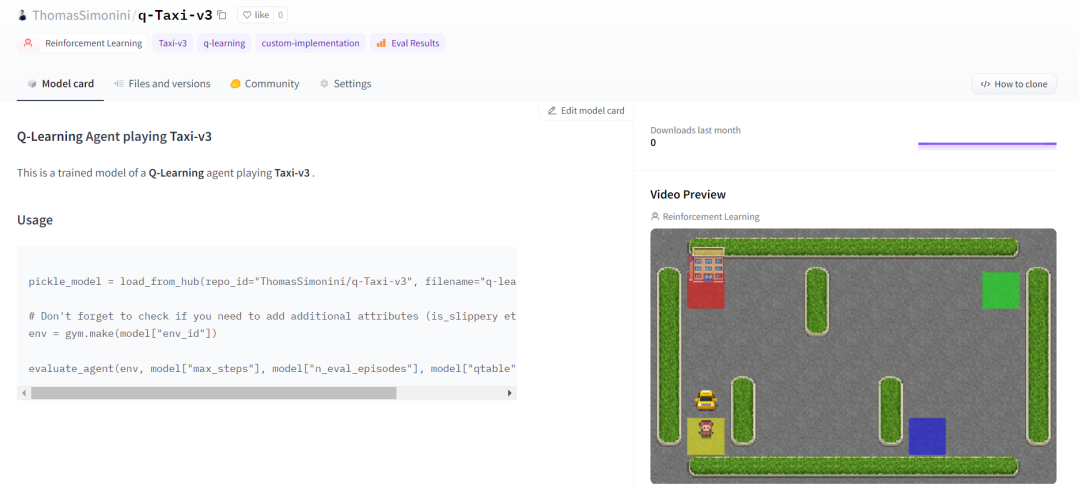


Рис. 89: Карточка модели на Hugging Face Hub [оригинал]

Внутри Hub использует репозитории на основе Git (ничего страшного, если вы пока не знакомы с Git). Это означает, что по мере экспериментов и улучшения агента модель можно обновлять и публиковать новые версии.

Не изменяйте этот код

```
from huggingface_hub import HfApi, snapshot_download
from huggingface_hub.repocard import metadata_eval_result, metadata_save

from pathlib import Path
import datetime
import json
```

```
def record_video(env, Qtable, out_directory, fps=1):
    """
    Generate a replay video of the agent
    :param env
    :param Qtable: Qtable of our agent
    :param out_directory
    :param fps: how many frame per seconds (with taxi-v3 and frozenlake-v1 we use 1)
    """
    images = []
    terminated = False
    truncated = False
    state, info = env.reset(seed=random.randint(0, 500))
    img = env.render()
    images.append(img)
    while not terminated or truncated:
        # Take the action (index) that have the maximum expected future reward given that state
        action = np.argmax(Qtable[state][:])
        state, reward, terminated, truncated, info = env.step(
            action
        )  # We directly put next_state = state for recording logic
        img = env.render()
        images.append(img)
    imageio.mimsave(out_directory, [np.array(img) for i, img in enumerate(images)], fps=fps)
```

```
def push_to_hub(repo_id, model, env, video_fps=1, local_repo_path="hub"):
    """
    Evaluate, Generate a video and Upload a model to Hugging Face Hub.
    This method does the complete pipeline:
    - It evaluates the model
    - It generates the model card
    - It generates a replay video of the agent
    - It pushes everything to the Hub

    :param repo_id: repo_id: id of the model repository from the Hugging Face Hub
    :param env
    :param video_fps: how many frame per seconds to record our video replay
    (with taxi-v3 and frozenlake-v1 we use 1)
    :param local_repo_path: where the local repository is
    """
    _, repo_name = repo_id.split("/")

    eval_env = env
    api = HfApi()

    # Step 1: Create the repo
    repo_url = api.create_repo(
        repo_id=repo_id,
        exist_ok=True,
    )

    # Step 2: Download files
    repo_local_path = Path(snapshot_download(repo_id=repo_id))

    # Step 3: Save the model
    if env.spec.kwargs.get("map_name"):
        model["map_name"] = env.spec.kwargs.get("map_name")
        if env.spec.kwargs.get("is_slippery", "") == False:
            model["slippery"] = False

    # Pickle the model
    with open((repo_local_path) / "q-learning.pkl", "wb") as f:
        pickle.dump(model, f)

    # Step 4: Evaluate the model and build JSON with evaluation metrics
    mean_reward, std_reward = evaluate_agent(
        eval_env, model["max_steps"], model["n_eval_episodes"], model["qtable"], model["eval_seed"]
    )

    evaluate_data = {
        "env_id": model["env_id"],
        "mean_reward": mean_reward,
        "n_eval_episodes": model["n_eval_episodes"],
        "eval_datetime": datetime.datetime.now().isoformat(),
    }

    # Write a JSON file called "results.json" that will contain the
    # evaluation results
    with open(repo_local_path / "results.json", "w") as outfile:
        json.dump(evaluate_data, outfile)

    # Step 5: Create the model card
    env_name = model["env_id"]
    if env.spec.kwargs.get("map_name"):
        env_name += "-" + env.spec.kwargs.get("map_name")

    if env.spec.kwargs.get("is_slippery", "") == False:
        env_name += "-" + "no_slippery"
```

```
  metadata = {}
  metadata["tags"] = [env_name, "q-learning", "reinforcement-learning", "custom-implementation"]

  # Add metrics
  eval = metadata_eval_result(
      model_pretty_name=repo_name,
      task_pretty_name="reinforcement-learning",
      task_id="reinforcement-learning",
      metrics_pretty_name="mean_reward",
      metrics_id="mean_reward",
      metrics_value=f"{mean_reward:.2f} +/- {std_reward:.2f}",
      dataset_pretty_name=env_name,
      dataset_id=env_name,
  )

  # Merges both dictionaries
  metadata = {**metadata, **eval}

  model_card = f"""
# **Q-Learning** Agent playing1 **{env_id}**
This is a trained model of a **Q-Learning** agent playing **{env_id}** .

## Usage

model = load_from_hub(repo_id="{repo_id}", filename="q-learning.pkl")

# Don't forget to check if you need to add additional attributes (is_slippery=False etc)
env = gym.make(model["env_id"])
"""

  evaluate_agent(env, model["max_steps"], model["n_eval_episodes"], model["qtable"], model["
  eval_seed"])

  readme_path = repo_local_path / "README.md"
  readme = ""
  print(readme_path.exists())
  if readme_path.exists():
      with readme_path.open("r", encoding="utf8") as f:
          readme = f.read()
  else:
      readme = model_card

  with readme_path.open("w", encoding="utf-8") as f:
      f.write(readme)

  # Save our metrics to Readme metadata
  metadata_save(readme_path, metadata)

  # Step 6: Record a video
  video_path = repo_local_path / "replay.mp4"
  record_video(env, model["qtable"], video_path, video_fps)

  # Step 7. Push everything to the Hub
  api.upload_folder(
      repo_id=repo_id,
      folder_path=repo_local_path,
      path_in_repo=".",
  )

  print("Your model is pushed to the Hub. You can view your model here: ", repo_url)
```

.

С помощью `push_to_hub` **вы оцениваете агента, записываете видео его работы, генерируете карточку модели и публикуете его в Hub**.

Это позволяет:

- **Продемонстрировать свою работу**.
- **Увидеть, как играет агент**.
- **Поделиться агентом с сообществом**, чтобы им могли пользоваться другие.
- **Посмотреть в таблице лидеров, насколько хорошо ваш агент работает по сравнению с агентами других участников**: https://huggingface.co/spaces/huggingface-projects/Deep-Reinforcement-Learning-Leaderboard.

Для публикации модели в сообществе нужно выполнить ещё три шага:

1. Если вы ещё не зарегистрированы, создайте аккаунт Hugging Face: https://huggingface.co/join.

2. Войдите в аккаунт и сохраните токен аутентификации, полученный на сайте Hugging Face.

- Создайте новый токен (https://huggingface.co/settings/tokens) **с правом записи (write)**.

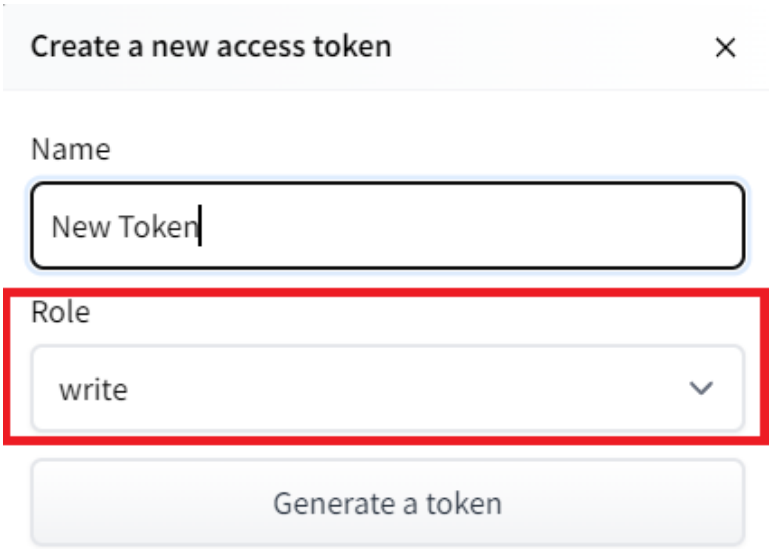


Рис. 90: Создание токена доступа к Hugging Face Hub [оригинал]

```
from huggingface_hub import notebook_login

notebook_login()
```

Если вы работаете не в Google Colab или Jupyter Notebook, используйте вместо этого команду `huggingface-cli login` (или `login`).

3. Теперь можно загрузить обученного агента в Hub с помощью функции `push_to_hub()`.

- Создадим **словарь модели, содержащий гиперпараметры и Q_table**.

```
model = {
    "env_id": env_id,
    "max_steps": max_steps,
    "n_training_episodes": n_training_episodes,
    "n_eval_episodes": n_eval_episodes,
    "eval_seed": eval_seed,
    "learning_rate": learning_rate,
    "gamma": gamma,
```

```
    "max_epsilon": max_epsilon,
    "min_epsilon": min_epsilon,
    "decay_rate": decay_rate,
    "qtable": Qtable_frozenlake,
}
```

Заполним аргументы функции `push_to_hub`:

- `repo_id`: имя репозитория Hugging Face Hub, который будет создан или обновлён.

(`repo_id = {username}/{repo_name}`). Хорошее имя репозитория: `{username}/q-{env_id}`.

- `model`: наш словарь модели с гиперпараметрами и Q-таблицей.
- `env`: среда.
- `commit_message`: сообщение коммита.

```
model
```

```
username = ""  # FILL THIS
repo_name = "q-FrozenLake-v1-4x4-noSlippery"
push_to_hub(repo_id=f"{username}/{repo_name}", model=model, env=env)
```

Поздравляем! Вы только что самостоятельно реализовали, обучили и загрузили своего первого RL-агента. FrozenLake-v1 без скольжения — очень простая среда; теперь попробуем более сложную.

15 Часть 2. Taxi-v3

15.1 Создание и изучение среды Taxi-v3

Перед использованием новой среды полезно изучить её документацию.

https://gymnasium.farama.org/environments/toy_text/taxi/

В `Taxi-v3` есть четыре специально обозначенных места на карте: R (красное), G (зелёное), Y (жёлтое) и B (синее).

В начале эпизода **такси находится в случайной клетке**, а пассажир — в случайной точке. Такси едет к пассажиру, **забирает его**, доставляет к пункту назначения (ещё одной из четырёх обозначенных точек) и **высаживает**. После высадки пассажира эпизод заканчивается.

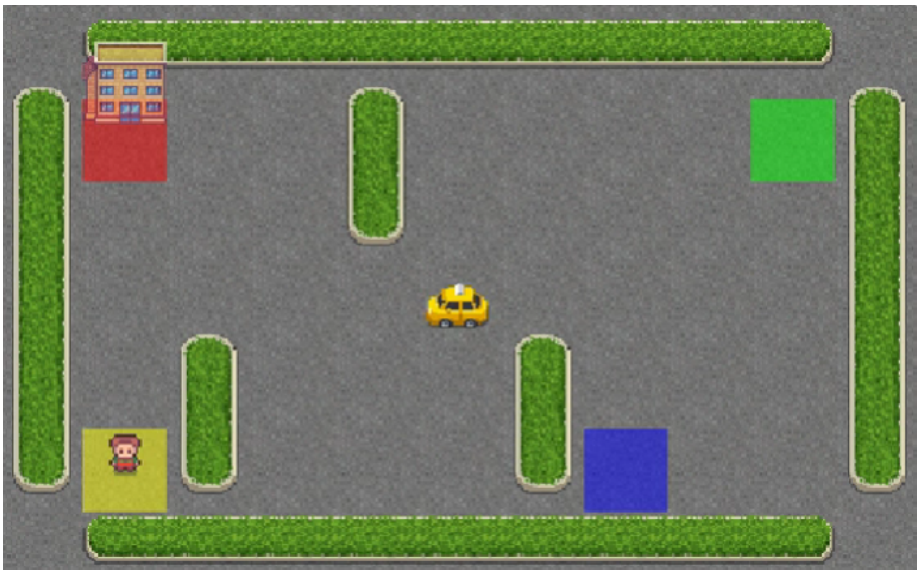


Рис. 91: Среда Taxi для Q-обучения [оригинал]

```
env = gym.make("Taxi-v3", render_mode="rgb_array")
```

Всего существует **500 дискретных состояний: 25 положений такси, 5 возможных положений пассажира** (включая нахождение в такси) и **4 пункта назначения**.

```
state_space = env.observation_space.n
print("There are ", state_space, " possible states")
```

```
action_space = env.action_space.n
print("There are ", action_space, " possible actions")
```

Пространство действий (множество возможных действий агента) дискретно и содержит **6 действий**:

- 0: движение на юг
- 1: движение на север
- 2: движение на восток
- 3: движение на запад
- 4: посадка пассажира
- 5: высадка пассажира

Функция вознаграждения:

- -1 за каждый шаг, если не сработало другое вознаграждение.
- +20 за доставку пассажира.
- -10 за недопустимое действие «посадка» или «высадка».

```
# Create our Q table with state_size rows and action_size columns (500x6)
Qtable_taxi = initialize_q_table(state_space, action_space)
print(Qtable_taxi)
print("Q-table shape: ", Qtable_taxi.shape)
```

15.2 Определение гиперпараметров

НЕ ИЗМЕНЯЙТЕ EVAL_SEED: массив eval_seed **позволяет оценивать агентов всех участников при одинаковых исходных положениях такси**.

```
# Training parameters
n_training_episodes = 25000  # Total training episodes
learning_rate = 0.7  # Learning rate

# Evaluation parameters
n_eval_episodes = 100  # Total number of test episodes

# DO NOT MODIFY EVAL_SEED
eval_seed = [
    16,
    54,
    165,
    177,
    191,
    191,
    120,
    80,
    149,
    178,
    48,
    38,
    6,
    125,
    174,
    73,
    50,
    172,
    100,
    148,
    146,
    6,
    25,
    40,
    68,
    148,
    49,
    167,
    9,
    97,
    164,
    176,
    61,
    7,
    54,
    55,
    161,
    131,
    184,
    51,
    170,
    12,
    120,
    113,
    95,
    126,
    51,
    98,
    36,
    135,
    54,
```

```
    82,
    45,
    95,
    89,
    59,
    95,
    124,
    9,
    113,
    58,
    85,
    51,
    134,
    121,
    169,
    105,
    21,
    30,
    11,
    50,
    65,
    12,
    43,
    82,
    145,
    152,
    97,
    106,
    55,
    31,
    85,
    38,
    112,
    102,
    168,
    123,
    97,
    21,
    83,
    158,
    26,
    80,
    63,
    5,
    81,
    32,
    11,
    28,
    148,
]  # Evaluation seed, this ensures that all classmates agents are trained on the same taxi starting
     position
# Each seed has a specific starting state
# Environment parameters
env_id = "Taxi-v3"  # Name of the environment
max_steps = 99  # Max steps per episode
gamma = 0.95  # Discounting rate

# Exploration parameters
max_epsilon = 1.0  # Exploration probability at start
min_epsilon = 0.05  # Minimum exploration probability
decay_rate = 0.005  # Exponential decay rate for exploration prob
```

15.3 Обучение нашего Q-агента

```
Qtable_taxi = train(n_training_episodes, min_epsilon, max_epsilon, decay_rate, env, max_steps,
    Qtable_taxi)
Qtable_taxi
```

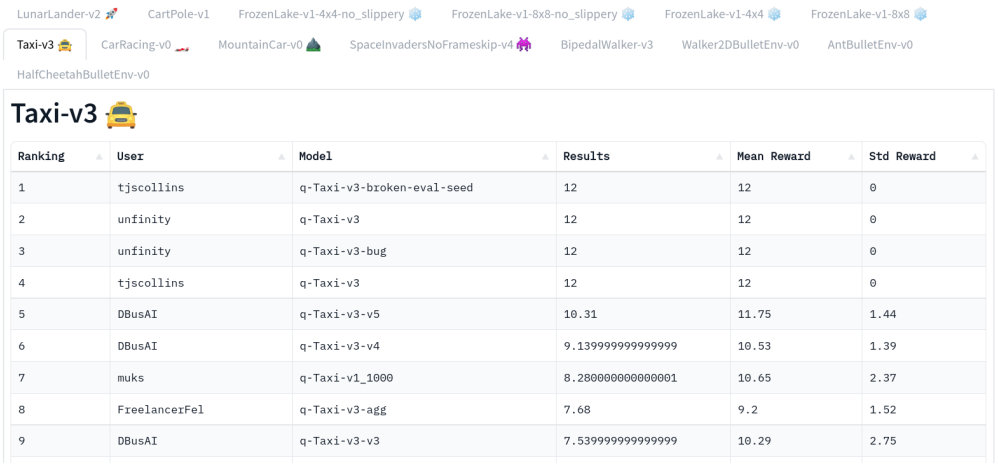
15.4 Создание словаря модели и публикация обученной модели в Hub

- Создадим словарь модели, содержащий все гиперпараметры обучения для воспроизводимости результатов, а также Q-таблицу.

```
model = {
    "env_id": env_id,
    "max_steps": max_steps,
    "n_training_episodes": n_training_episodes,
    "n_eval_episodes": n_eval_episodes,
    "eval_seed": eval_seed,
    "learning_rate": learning_rate,
    "gamma": gamma,
    "max_epsilon": max_epsilon,
    "min_epsilon": min_epsilon,
    "decay_rate": decay_rate,
    "qtable": Qtable_taxi,
}
```

```
username = ""  # FILL THIS
repo_name = ""  # FILL THIS
push_to_hub(repo_id=f"{username}/{repo_name}", model=model, env=env)
```

Теперь, когда модель опубликована в Hub, можно сравнить результаты своего агента Taxi-v3 с результатами других участников в таблице лидеров.



Ranking	User	Model	Results	Mean Reward	Std Reward
1	tjscollins	q-Taxi-v3-broken-eval-seed	12	12	0
2	unfinity	q-Taxi-v3	12	12	0
3	unfinity	q-Taxi-v3-bug	12	12	0
4	tjscollins	q-Taxi-v3	12	12	0
5	DBusAI	q-Taxi-v3-v5	10.31	11.75	1.44
6	DBusAI	q-Taxi-v3-v4	9.139999999999999	10.53	1.39
7	muks	q-Taxi-v1_1000	8.280000000000001	10.65	2.37
8	FreelancerFel	q-Taxi-v3-agg	7.68	9.2	1.52
9	DBusAI	q-Taxi-v3-v3	7.539999999999999	10.29	2.75

Рис. 92: Таблица лидеров для среды Taxi [оригинал]

16 Часть 3. Загрузка модели из Hub

Одна из замечательных возможностей Hugging Face Hub — простая загрузка мощных моделей, опубликованных сообществом.

Загрузить сохранённую модель из Hub очень просто:

1. Откройте https : / / huggingface . co / models ? other = q-learning, чтобы увидеть список сохранённых моделей Q-обучения.

2. Выберите модель и скопируйте её repo_id.

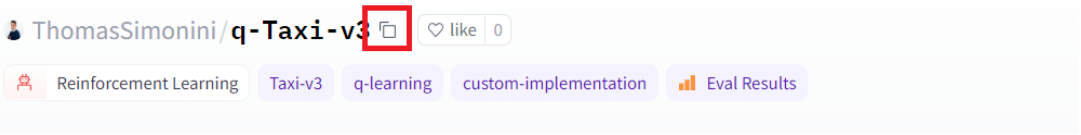


Рис. 93: Пример поиска идентификатора репозитория модели на Hugging Face Hub [оригинал]

3. Затем нужно вызвать `load_from_hub`, указав:

- Идентификатор repo_id.
- Имя файла: сохранённая модель внутри репозитория.

Не изменяйте этот код

```
from urllib.error import HTTPError

from huggingface_hub import hf_hub_download


def load_from_hub(repo_id: str, filename: str) -> str:
    """
    Download a model from Hugging Face Hub.
    :param repo_id: id of the model repository from the Hugging Face Hub
    :param filename: name of the model zip file from the repository
    """
    # Get the model from the Hub, download and cache the model on your local disk
    pickle_model = hf_hub_download(repo_id=repo_id, filename=filename)

    with open(pickle_model, "rb") as f:
        downloaded_model_file = pickle.load(f)

    return downloaded_model_file
```

.

```
model = load_from_hub(repo_id="ThomasSimonini/q-Taxi-v3", filename="q-learning.pkl")  # Try to use
    another model

print(model)
env = gym.make(model["env_id"])

evaluate_agent(env, model["max_steps"], model["n_eval_episodes"], model["qtable"], model["eval_seed
    "])
```

```
model = load_from_hub(
    repo_id="ThomasSimonini/q-FrozenLake-v1-no-slippery", filename="q-learning.pkl"
)  # Try to use another model

env = gym.make(model["env_id"], is_slippery=False)

evaluate_agent(env, model["max_steps"], model["n_eval_episodes"], model["qtable"], model["eval_seed
    "])
```

16.1 Дополнительные задания

Лучший способ учиться — **экспериментировать самостоятельно**! Как вы заметили, текущий агент пока работает не слишком хорошо. Для начала можно увеличить число шагов обучения.

На миллионе шагов нам удалось получить отличные результаты!

В таблице лидеров вы найдёте своих агентов. Получится ли выйти на первое место?

Несколько идей для улучшения позиции в таблице:

- Увеличьте число шагов обучения.
- Попробуйте другие гиперпараметры, ориентируясь на опыт других участников.
- **Опубликуйте новую обученную модель** в Hub.

Кажется, что ходить по льду и управлять такси скучно? Попробуйте **сменить среду**, например использовать FrozenLake-v1 со скольжением. Изучите, как работают другие среды, по документации Gymnasium, и экспериментируйте с удовольствием.

Поздравляем! Вы только что реализовали, обучили и опубликовали своего первого агента обучения с подкреплением.

Понимание Q-обучения — **важный шаг к пониманию методов на основе ценности.**

В следующем модуле мы изучим Deep Q-Learning и увидим, что создание и обновление Q-таблицы — хорошая стратегия, **но она плохо масштабируется.**

Например, представьте, что вы создаёте агента, который учится играть в Doom.

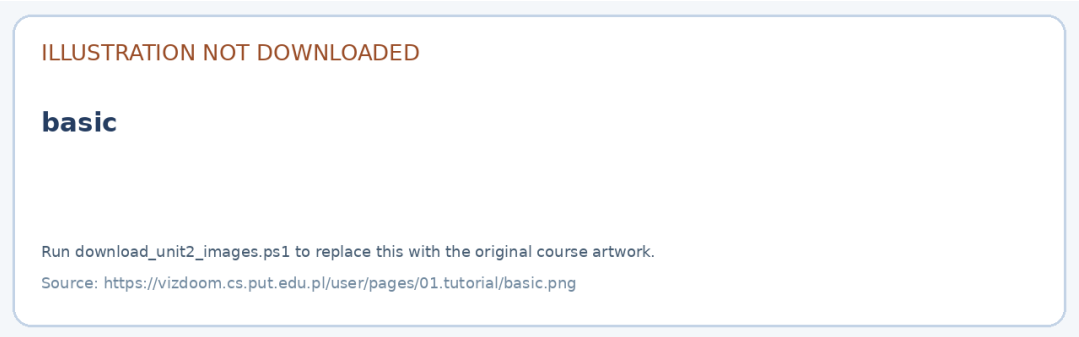


Рис. 94: Игра Doom [оригинал]

Doom — большая среда с огромным пространством состояний (миллионы различных состояний). Создавать и обновлять Q-таблицу для такой среды неэффективно.

Поэтому в следующем модуле мы изучим Deep Q-Learning — алгоритм, **в котором нейронная сеть по заданному состоянию приближённо вычисляет Q-значения доступных действий.**

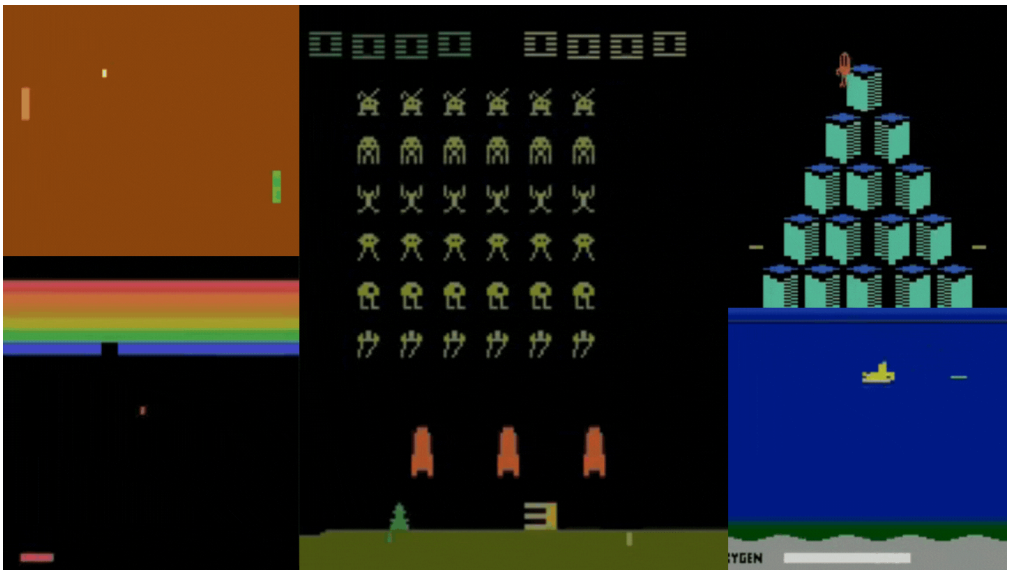


Рис. 95: Игровые среды Atari [оригинал]

Увидимся в третьем модуле!

16.2 Продолжайте учиться и развиваться

17 Второй тест

Лучший способ учиться и избежать иллюзии компетентности — **проверять себя**. Это помогает понять, **какие знания требуют закрепления**.

Вопрос 1. Что такое Q-обучение?

Выберите все подходящие ответы.

- **A.** Алгоритм, с помощью которого мы обучаем Q-функцию.
- **B.** Функция ценности.
- **C.** Алгоритм, определяющий ценность нахождения в заданном состоянии и выполнения в нём конкретного действия.
- **D.** Таблица.

Вопрос 2. Что такое Q-таблица?

Выберите все подходящие ответы.

- **A.** Алгоритм, который мы используем в Q-обучении.
- **B.** Q-таблица — внутренняя память нашего агента.
- **C.** Каждая ячейка Q-таблицы соответствует ценности состояния.

Вопрос 3. Почему наличие оптимальной Q-функции Q^* означает наличие оптимальной политики?

Вопрос 4. Можете ли вы объяснить, что такое ε-жадная стратегия?

Вопрос 5. Как обновляется Q-значение пары «состояние–действие»?

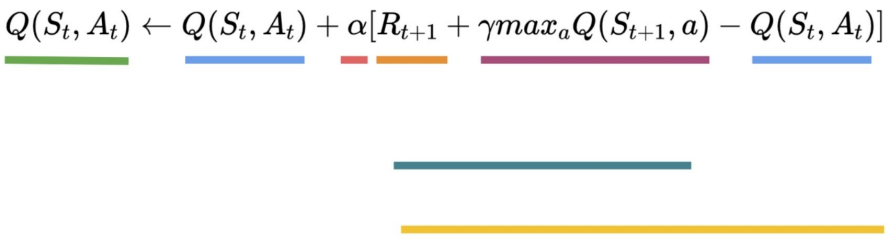


Рис. 96: Тест: определите компоненты формулы обновления Q-значения [оригинал]

Вопрос 6. В чём различие между on-policy и off-policy?

17.1 Ответы и пояснения

Вопрос 1. Что такое Q-обучение?

Правильный ответ: A

- **Вариант B:** Это функция ценности действий, поскольку она определяет ценность нахождения в заданном состоянии и выполнения в нём конкретного действия.
- **Вариант C:** Q-функция определяет ценность нахождения в конкретном состоянии и выполнения в нём конкретного действия.
- **Вариант D:** Q-обучение — не Q-таблица. Q-функция — это алгоритм, формирующий значения для Q-таблицы.

Вопрос 2. Что такое Q-таблица?

Правильный ответ: B

- **Вариант C:** Каждая ячейка соответствует ценности пары «состояние–действие», а не отдельного состояния.

Вопрос 3. Почему наличие оптимальной Q-функции Q^* означает наличие оптимальной политики?

Потому что при наличии оптимальной Q-функции есть и оптимальная политика: для каждого состояния мы знаем, какое действие является наилучшим.

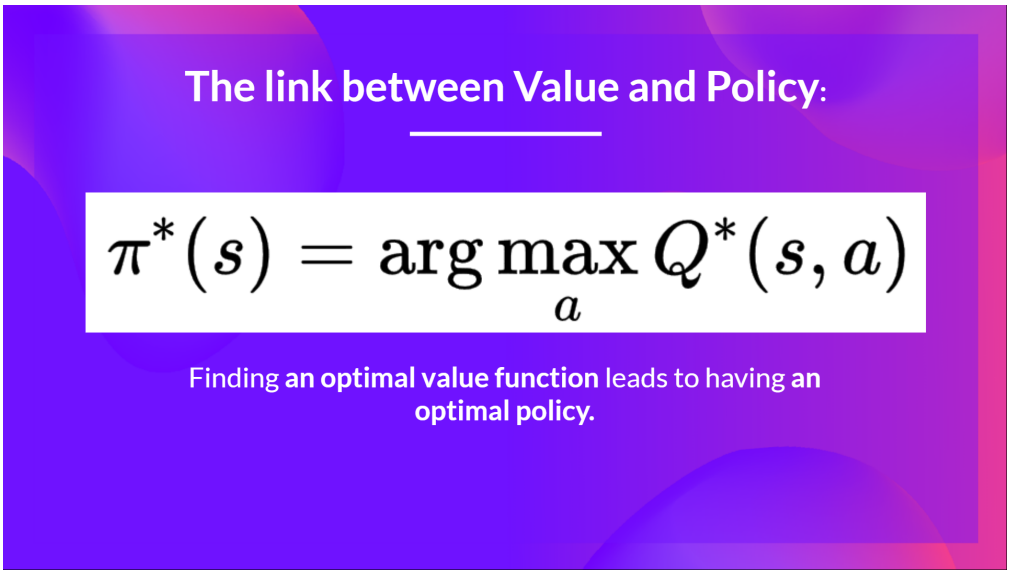


Рис. 97: Восстановление оптимальной политики из оптимальной функции ценности действий [оригинал]

Вопрос 4. Можете ли вы объяснить ε-жадную стратегию?

ε-жадная стратегия — это политика, обеспечивающая компромисс между исследованием среды и использованием накопленных знаний.

Идея состоит в том, что мы задаём $\varepsilon = 1.0$:

- С *вероятностью* $1 - \varepsilon$ используем знания (агент выбирает действие с максимальной ценностью пары «состояние–действие»).
- С *вероятностью* ε исследуем среду (пробуем случайное действие).

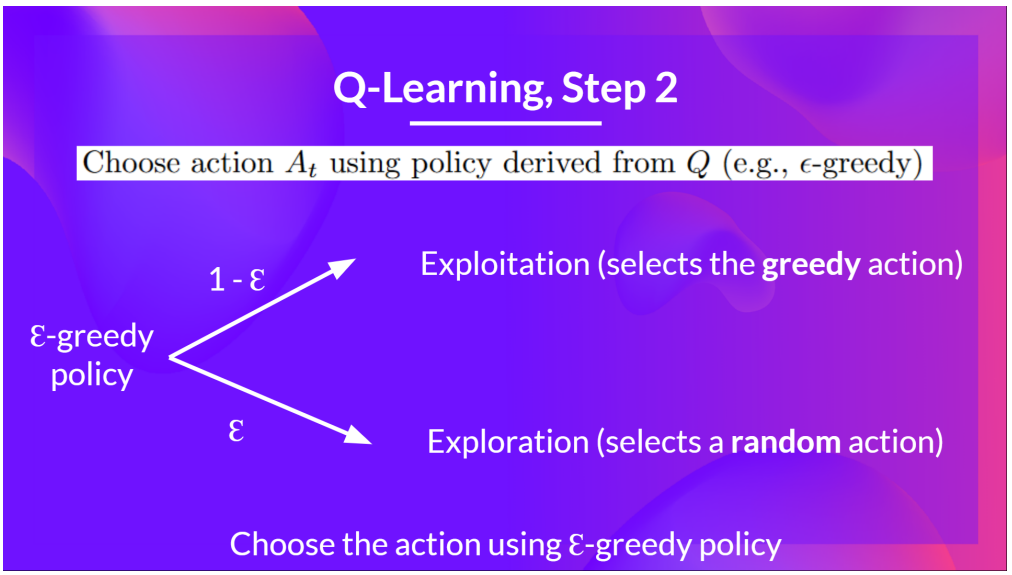


Рис. 98: ε-жадная стратегия: исследование и использование знаний [оригинал]

Вопрос 5. Как обновляется Q-значение пары «состояние–действие»?

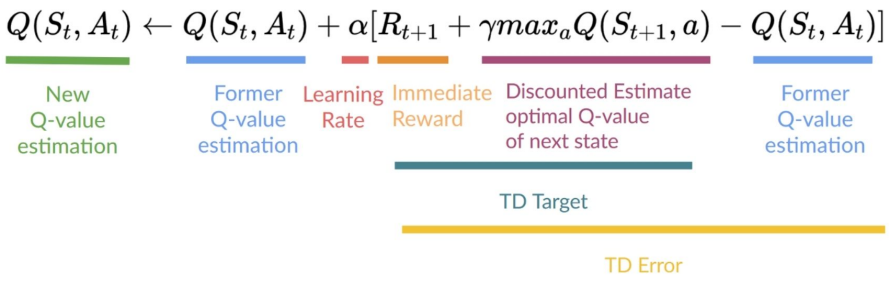


Рис. 99: Решение: компоненты формулы обновления Q-значения [оригинал]

Вопрос 6. В чём различие между on-policy и off-policy?

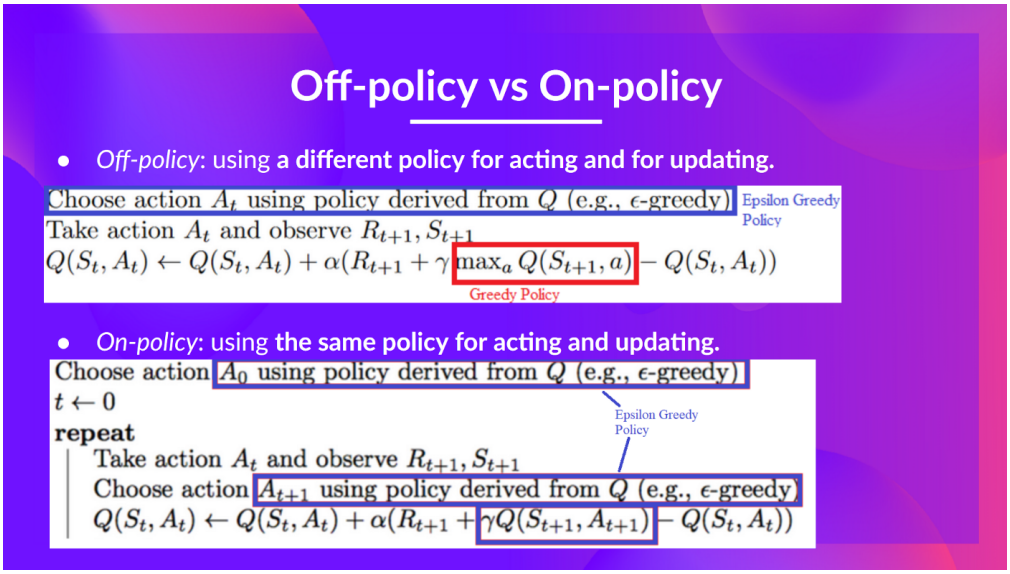


Рис. 100: Сравнение методов off-policy и on-policy [оригинал]

Поздравляем с прохождением теста! Если какие-то вопросы вызвали затруднения, вернитесь к материалу главы, чтобы закрепить знания.

18 Глоссарий

Этот глоссарий составлен сообществом. Мы будем рады вашим дополнениям!

Стратегии поиска оптимальной политики

- **Методы на основе политики (policy-based).** Обычно политика обучается с помощью нейронной сети, выбирающей действие по заданному состоянию. В этом случае именно нейронная сеть возвращает действие, которое должен выполнить агент, а не функция ценности. На основании опыта взаимодействия со средой параметры нейронной сети корректируются, и она начинает предлагать всё более подходящие действия.

- **Методы на основе ценности (value-based).** В этом случае обучается функция ценности, выдающая оценку состояния или пары «состояние–действие», на основе которой строится политика. Однако само значение не указывает, какое действие нужно выполнить. Поэтому необходимо отдельно задать правила поведения агента по выходу функции ценности. Например, можно выбрать политику, всегда выполняющую действие с максимальным ожидаемым вознаграждением (жадную политику). Таким образом, политика — это жадная стратегия (или любая другая заданная пользователем), использующая значения функции ценности для выбора действий.

В методах на основе ценности различают два основных типа функций

- **Функция ценности состояния.** Для каждого состояния выдаёт ожидаемую отдачу, если агент начинает из него и следует политике до конца.
- **Функция ценности действия.** В отличие от функции ценности состояния, для каждой пары «состояние–действие» вычисляет ожидаемую отдачу, если агент начинает из заданного состояния, выполняет заданное действие, а затем всегда следует политике.

ε-жадная стратегия

- Распространённая стратегия обучения с подкреплением, которая балансирует исследование среды и использование известных знаний.
- С вероятностью $1 - \varepsilon$ выбирает действие с максимальным ожидаемым вознаграждением.
- С вероятностью ε выбирает случайное действие.
- Обычно ε со временем уменьшают, постепенно смещая акцент в сторону использования накопленных знаний.

Жадная стратегия

- Предполагает постоянный выбор действия, которое, согласно текущим знаниям о среде, должно принести наибольшее вознаграждение (только использование известных знаний).
- Всегда выбирает действие с максимальным ожидаемым вознаграждением.
- Не предусматривает исследования среды.
- Может быть неэффективной в условиях неопределённости или когда оптимальные действия неизвестны.

Алгоритмы off-policy и on-policy

- **Off-policy алгоритмы:** при обучении и применении модели используются разные политики.
- **On-policy алгоритмы:** при обучении и применении модели используется одна и та же политика.

Стратегии обучения Монте-Карло и с временными разностями

- **Монте-Карло (MC):** обучение в конце эпизода. Метод ждёт завершения эпизода, после чего по всему эпизоду обновляет функцию ценности (или политику).
- **Временные разности (TD):** обучение на каждом шаге. Функция ценности (или политика) обновляется после каждого шага, не дожидаясь завершения эпизода.

Если хотите помочь улучшить курс, можете создать Pull Request.

Этот глоссарий был подготовлен благодаря:

- Ramón Rueda
- Hasarindu Perera
- Arkady Arkhangorodsky

19 Заключение

Поздравляем с завершением главы! Мы разобрали много материала. И поздравляем с окончанием практики: вы только что самостоятельно реализовали первого RL-агента и поделились им в Hub.

При изучении новой архитектуры **важно реализовать её самостоятельно с нуля, чтобы понять принцип работы.**

Совершенно нормально, если какие-то детали пока остаются непонятными. Со мной и со всеми, кто изучал RL, происходило то же самое.

Не спешите: прежде чем двигаться дальше, постарайтесь действительно понять материал.

В следующей главе мы глубже изучим первый алгоритм глубокого обучения с подкреплением на базе Q-обучения: Deep Q-Learning. А затем вы обучите **DQN-агента с помощью RL-Baselines3 Zoo играть в игры Atari**.

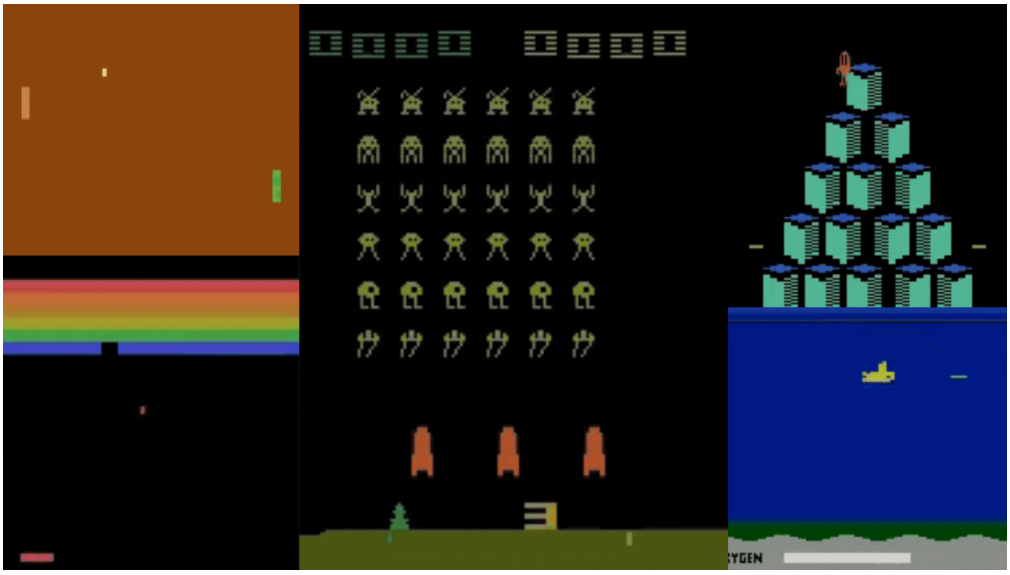


Рис. 101: Игровые среды Atari [оригинал]

Наконец, нам очень важно **узнать ваше мнение о курсе и о том, как его улучшить**. Если у вас есть предложения, пожалуйста, заполните эту форму.

Продолжайте учиться и развиваться

20 Дополнительная литература

Эти материалы **необязательны** и предназначены для желающих изучить тему глубже.

20.1 Обучение Монте-Карло и с временными разностями

Для более глубокого изучения методов Монте-Карло и TD:

- Почему методы временных разностей (TD) имеют меньшую дисперсию, чем методы Монте-Карло?
- В каких случаях методы Монте-Карло предпочтительнее методов временных разностей?

20.2 Q-обучение

- «Reinforcement Learning: An Introduction», Ричард Саттон и Эндрю Барто, главы 5, 6 и 7
- Цикл лекций «Foundations of Deep RL», лекция 2 о Deep Q-Learning, Питер Аббил